Data Engineering

Distributed Systems, Scalable Processing,
and Modern Architectures

Rokan Uddin Faruqui

Professor, Department of Computer Science & Engineering
University of Chittagong

2026

First Edition, 2026.

Contents

Part I

The Data Engineering Landscape

1

The Big Data Revolution: Scale, Velocity, and Variety

"The world's most valuable resource is no longer oil, but data."

The Economist, 2017

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why traditional relational database systems cannot handle modern data workloads at scale, and identify the specific limits that are exceeded.
2. Define the three original dimensions of big data (Volume, Velocity, Variety) and the two additional dimensions (Veracity, Value) that form the extended 5Vs framework.
3. Classify any given data source as structured, semi-structured, or unstructured, and identify the appropriate storage and query tool for each category.
4. Distinguish between vertical and horizontal scaling strategies, and explain why distributed, horizontally scaled systems dominate big data engineering.
5. State the CAP theorem, explain the meaning of each property, and

identify how real-world systems (HBase, Cassandra, PostgreSQL) position themselves in the CAP triangle.
6. Compare batch and stream processing models and select the appropriate model for a given engineering requirement.

Every 60 seconds in 2024, users upload 500 hours of video to YouTube, execute more than 5 million Google searches, send over one million WhatsApp messages, and publish approximately 6,000 posts on X (formerly Twitter). These figures are not curiosities; they represent a fundamental transformation in the scale, speed, and diversity of data that computing infrastructure is expected to handle. This chapter establishes the conceptual and technical foundation for the entire book. We begin by examining what makes data "big," explore how that bigness is formally characterized, and introduce the core theoretical constraints—the CAP theorem in particular—that shape every architectural decision in data engineering.

1.1 Why Data Engineering Matters

In 2018, the International Data Corporation (IDC) projected that the global ***datasphere***—the total volume of data generated, captured, copied, and consumed worldwide—would reach **175 zettabytes** by 2025, representing a 23-fold increase over the 7.2 ZB that existed in 2018 (Reinsel, Gantz, and Rydning 2018). To appreciate this scale, consider that one zettabyte equals 10^{21} bytes, or roughly 250 billion standard 4 TB hard drives—more drives than there are stars visible to the naked eye from any single point on Earth.

Traditional ***relational database management systems*** (RDBMS)—the dominant paradigm since Edgar Codd's foundational work in 1970—are extraordinary tools for managing well-structured, bounded datasets. A modern relational database running on a well-provisioned server can comfortably manage tens of terabytes and sustain thousands of transactions per second with full ACID guarantees. For a bank's transaction ledger, a hospital's patient registry, or an airline's reservation system, a carefully tuned relational system remains the correct choice. The problem emerges

Table 1.1: Data volume units and their relationship to everyday storage

Unit	Bytes	Illustrative equivalent
Kilobyte (KB)	10^3	A short email (plain text)
Megabyte (MB)	10^6	A 3-minute MP3 audio track
Gigabyte (GB)	10^9	A feature-length HD film
Terabyte (TB)	10^{12}	~310,000 photos from a 12 MP camera
Petabyte (PB)	10^{15}	All US academic research libraries combined
Exabyte (EB)	10^{18}	All words ever spoken by humans (estimated)
Zettabyte (ZB)	**10^{21}**	**2025 global datasphere**
Yottabyte (YB)	10^{24}	Theoretical future scale

when data characteristics exceed what any single machine can handle: when datasets are measured in petabytes rather than terabytes; when data arrives not from a single transactional application but from millions of concurrent event streams; and when the data itself is not neatly organized into rows and columns but exists as images, audio recordings, sensor telemetry, and free-form text. In these conditions, the fundamental assumptions of relational database design break down.

Data engineering is the discipline concerned with the design, construction, and maintenance of systems that collect, store, transform, and serve large-scale data for downstream consumers—analysts, machine learning models, business intelligence dashboards, and operational applications. A data engineer builds the infrastructure that makes raw data useful. This field draws on distributed systems, software engineering, database theory, and applied computer science. The tools of modern data engineering—distributed file systems, parallel batch processors, in-memory computation engines, stream processing frameworks, and pipeline orchestrators—are the subjects of this book.

Key Insight | The Data Engineering Mandate

Data engineering is not about *generating* insight—that is the domain of data science and analytics. Data engineering is about making data *available, reliable, and efficiently accessible* at any scale. The data engineer builds the plumbing; analysts and scientists use the water.

1.2 Defining Big Data: The 3Vs Framework

The term "big data" gained formal structure in 2001 when Doug Laney, then a research analyst at the META Group, published a brief but enormously influential research note identifying three dimensions along which data management challenges had outpaced conventional tools (Laney 2001). He labeled these dimensions *Volume*, *Velocity*, and *Variety*—the ***3Vs of big data***. Eleven years later, Gartner (which had acquired META Group) codified these dimensions into a formal definition:

Definition | Big Data (Gartner/Beyer & Laney, 2012)

"Big data is high-volume, high-velocity and/or high-variety information assets that demand cost-effective, innovative forms of information processing that enable enhanced insight, decision making, and process automation."

— Beyer & Laney, Gartner Research, 2012 (Beyer and Laney 2012)

Each of the three dimensions corresponds to a distinct class of engineering challenge. The McKinsey Global Institute, in its landmark 2011 study, reinforced this framing: big data refers to datasets "whose size is beyond the ability of typical database software tools to capture, store, manage and analyze" (Manyika et al. 2011). Let us examine each dimension in turn.

1.2.1 Volume: The Scale Dimension

Volume refers to the sheer quantity of data that is generated, accumulated, and must be stored and processed. Volume is the most intuitively obvious characteristic of big data: a dataset is voluminous when it exceeds the capacity of conventional tools—typically, when a single machine cannot

store and process it within an acceptable elapsed time. The following production examples illustrate current operational scales:

Table 1.2: Volume at production scale (representative 2023–2024 estimates)

Organisation	Volume characteristic
Meta (Facebook)	500 TB of new data ingested per day across all platforms
Google	>100 PB of data processed internally per day (estimated)
Walmart	>2.5 PB of customer transaction data in analytical systems
CERN (LHC)	≈15 PB of particle collision data generated per year
Netflix	>1 PB of user interaction events per day
NYSE	≈1 TB of trading records generated on a peak day

From an engineering perspective, volume creates several concrete challenges. First, no single server—even the highest-specification available in 2024, with up to 64 TB of RAM—can hold a petabyte-scale dataset in memory. Second, sequential reads from a single high-speed SSD array proceed at approximately 7 GB/s; reading one petabyte at that speed requires roughly 41 hours. Third, ingesting 500 TB per day means sustained write throughput of approximately 5.8 GB/s, which no single storage device can sustain reliably. The engineering response is *horizontal distribution*: partition data into blocks and spread those blocks across a cluster of hundreds or thousands of commodity machines—the approach formalized by the Google File System (Ghemawat, Gobioff, and Leung 2003) and, in open-source form, by Hadoop's HDFS (discussed in Chapter 3).

1.2.2 Velocity: The Speed Dimension

Velocity refers to the speed at which data is generated, arrives at a system, and must be processed or acted upon. High-velocity data is not merely large; it arrives *continuously* and often demands responses on timescales far

shorter than batch processing can provide.

Consider a few representative velocities:

- The Visa payment network processes approximately 24,000 transactions per second at peak load.
- Global IoT deployments generate over one billion device sensor readings per hour across industrial, environmental, and consumer applications.
- A ride-sharing platform (such as Uber or Lyft) receives a GPS ping every three seconds from millions of simultaneously active drivers—at 5 million drivers, that is over 1.6 million location records per second.
- Modern stock exchanges publish quote updates for listed securities at rates exceeding one million messages per second during high-activity sessions.

The engineering challenge of velocity is distinct from that of volume: a system may need to respond to each event within milliseconds (fraud detection, payment authorization) or may tolerate latencies of seconds (real-time dashboards). The key constraint is that high-velocity data *cannot wait* for a nightly batch process—by the time the batch runs, the window for action has closed. This distinction drives the separation between *batch* and *stream* processing architectures, explored in Section 1.7 and in depth in Chapter **??**.

Table 1.3: The data processing spectrum by velocity requirement

Processing mode	Latency	Representative tools	Typical use case
Batch	Hours	Hadoop MapReduce, Hive	Nightly ETL, monthly billing
Micro-batch	Seconds	Spark Streaming	Near-real-time dashboards
Stream	Milliseconds	Apache Flink, Kafka Streams	Fraud detection, alerts
Real-time	Sub-ms	In-memory + Kafka	Payment authorization

1.2.3 Variety: The Diversity Dimension

Variety refers to the diversity of data formats, types, schemas, and sources that must be jointly ingested, stored, and analysed. Modern organisations do not operate a single data source with a uniform schema; they accumulate data from dozens of systems, each with its own format and semantics.

Research Spotlight | Laney (2001) — The Origin of the 3Vs

Citation: Laney, D. (2001). *3D Data Management: Controlling Data Volume, Velocity and Variety*. META Group Research Note 949, February 2001. (Laney 2001)

Key insight: Laney's one-page note, written in the context of e-commerce data challenges, framed data management as a three-dimensional problem—not merely a storage problem. He argued that organisations failing to address all three dimensions would face "increasing management headaches."

Technical contributions:

- Established the first formal vocabulary (*3Vs*) for characterising modern data management challenges, unifying what had previously been treated as unrelated operational problems.
- Distinguished *velocity* as a management dimension independent of volume—recognising that *how fast* data arrives is as important as *how much* arrives.
- Introduced *variety* as the hardest dimension to manage, predicting the rise of semi-structured and unstructured data as dominant forms.

Impact today: The 3Vs framework is taught at every major computer science programme and has been cited in thousands of academic papers and industry reports. It remains the canonical definition of big data, even as the framework has been extended to 5Vs and beyond. Gartner officially adopted and extended the definition in 2012.

Variety manifests along several axes. *Format variety* arises when data arrives as CSV files, JSON documents, XML feeds, binary Avro records, columnar Parquet files, DICOM medical images, and MP4 video simultaneously—all of which must be stored and queried together. *Source variety* arises when

the same analytical question requires joining data from relational databases, REST APIs, IoT sensor streams, social media firehoses, and internal event logs. *Semantic variety* arises when a field named `date` means the transaction date in one system, the record-entry date in another, and the customer birth date in a third.

The foundational distinction among data types—structured, semi-structured, and unstructured—is the key conceptual framework for managing variety, and we explore it in detail in Section 1.4.

1.3 The Extended 5Vs Framework

The 3Vs framework is powerful but incomplete. Two additional dimensions have gained wide acceptance, bringing the framework to five:

Veracity refers to the trustworthiness and quality of data. High-veracity data is accurate, consistent, and free of critical errors; low-veracity data is noisy, inconsistent, or of uncertain provenance. Veracity challenges arise in practical systems for many reasons: GPS sensors drift by tens of metres in dense urban environments; optical character recognition (OCR) on historical documents introduces transcription errors; social media platforms contain significant volumes of bot-generated content; and sensor networks may experience intermittent failures that produce spurious or missing readings. The consequences of low veracity can be severe—a machine learning model trained on inaccurate data will make inaccurate predictions, regardless of how sophisticated the model or how large the training set.

Value refers to the business or societal utility that can be extracted from data. It is the ultimate justification for the considerable engineering investment required to build big data systems. A dataset may score highly on all four other Vs—petabytes of high-velocity, highly varied, high-quality data—and still produce no actionable insight if the right analytical questions are never asked, or if the results are never operationalised. Value is the bridge between technical capability and organisational outcomes.

Research Spotlight | Quantifying the Value Dimension

Citation: Manyika, J. et al. (2011). *Big Data: The Next Frontier for Innovation, Competition, and Productivity*. McKinsey Global Institute, May 2011. (Manyika et al. 2011)

Key insight: The report argued that big data has become "a new factor of production," alongside capital and labour—that is, organisations with superior data capabilities will, over time, systematically outperform those without them, in the same way that access to capital separates competitive enterprises from struggling ones.

Technical and economic contributions:

- Quantified the value potential across five major sectors: healthcare ($300B+ annually in the US), public sector ($250B+ in Europe), retail (>60% operating margin improvement for early adopters), manufacturing, and personal location data.
- Identified the acute shortage of data-skilled talent as the primary constraint on value realisation—the report projected a shortfall of 140,000–190,000 professionals with deep analytical expertise in the US alone by 2018.
- Introduced the concept of "data as infrastructure," arguing that the economic benefits of big data investment are broadly distributed across the economy, much like roads or electricity grids.

Impact today: This report is one of the most cited documents in the history of data-driven business strategy. It directly influenced technology investment priorities at major corporations and governments worldwide, and it validated the creation of dedicated data engineering and data science functions within large organisations.

1.4 A Taxonomy of Digital Data

Managing the Variety dimension of big data requires a clear classification of data types. The field has converged on three primary categories—structured, semi-structured, and unstructured—distinguished principally

Table 1.4: The 5Vs framework with engineering implications

Dimension	Definition	Production example	Primary engineering challenge
Volume	Scale of data generated and stored	Meta: 500 TB/day ingested	Distributed storage (HDFS, S3)
Velocity	Speed of generation and processing	Visa: 24,000 tx/s	Stream processing (Kafka, Flink)
Variety	Diversity of formats and sources	CSV + JSON + DICOM + MP4	Schema integration, ETL/ELT
Veracity	Trustworthiness of data quality	GPS drift, OCR errors, bot tweets	Data quality pipelines, validation
Value	Business utility extracted from data	Fraud prevented, revenue gained	Feature engineering, ML pipelines

by the presence, flexibility, and location of their schema.

1.4.1 Structured Data

Definition | Structured Data

Data organised according to a **predefined, rigid schema** in which every record contains the same fields in the same order, with each field having a specified data type. The schema is defined and enforced before data is written (**schema-on-write**). Structured data is typically stored in relational database management systems and queryable directly with standard SQL.

Structured data represents the most mature and well-understood category. Relational database theory, developed by Codd (1970) and refined over

five decades, provides powerful tools for storing, querying, and maintaining the integrity of structured data. A bank's transaction ledger is a canonical example: every row has an account number, a timestamp, an amount, a currency code, and a transaction type. There are no optional fields, no embedded documents, no ambiguous formats. SQL can answer virtually any analytical question over such data with high efficiency.

The primary limitation of structured data is its rigidity. Adding a new column to a table with hundreds of millions of rows requires a schema migration—a costly, time-consuming, and potentially disruptive operation. More fundamentally, many of the most valuable data types in modern organisations—customer reviews, support transcripts, medical images, sensor streams—cannot be naturally represented in rows and columns without destructive simplification. Despite this, structured data remains the bedrock of transactional systems, and big data engineering tools like Apache Sqoop exist specifically to migrate structured data from RDBMS sources into distributed analytical systems (covered in Chapter **??**).

In terms of global prevalence, structured data accounts for only approximately 20% of all digital data—yet it represents the most well-governed, analysable slice of that data.

1.4.2 Semi-Structured Data

Definition | Semi-Structured Data

Data with a **flexible, self-describing schema** embedded within the data itself. Fields are identified by keys or tags that travel with the record, not by position in a fixed row. New fields can be added without modifying a central schema registry. Semi-structured data is typically stored as files or in NoSQL databases and processed with a **schema-on-read** approach: the schema is imposed at query time, not at write time.

Semi-structured data dominates modern web and application architectures. The two most prevalent formats are **JSON** (JavaScript Object Notation) and **XML** (Extensible Markup Language):

```
{
  "event_id":   "e-928471",
  "event_type": "page_view",
```

```
  "timestamp":  "2024-03-15T14:22:31Z",
  "user": {
    "id":       "u-441982",
    "tier":     "premium"
  },
  "page":       "/product/laptop-x500",
  "session_ms": 4820,
  "metadata": {
    "user_agent": "Mozilla/5.0 ...",
    "referrer":   "https://search.example.com"
  }
}
```

Listing 1.1: A JSON event record from a streaming API

Notice that the JSON record carries its own field names ("event_id", "timestamp", "user", etc.) directly within the data. A consumer reading this record does not need to consult an external schema definition to know what each field means. This *self-description* property makes semi-structured data highly flexible: a downstream service that only cares about `event_type` and `user.tier` can simply ignore all other fields. When a new field (e.g., `ab_test_variant`) must be added, it can be appended to future records without invalidating existing ones.

The schema-on-read approach enabled by semi-structured formats is central to the *data lake* architecture: data is ingested in its native format without transformation, and the schema is applied by the analytical tool at query time. This defers the cost of schema design but introduces the risk of schema drift—where the meaning or format of a field changes silently between producers, creating silent data quality failures. Managing this risk (through schema registries, contract testing, and data lineage tracking) is a significant part of production data engineering practice.

Web server access logs, IoT device telemetry published via MQTT, REST API responses, email headers, configuration files in YAML or TOML, and Apache Avro records with embedded schemas all belong to the semi-structured category. This category accounts for roughly 10% of global data by volume.

1.4.3 Unstructured Data

Definition | Unstructured Data

Data with **no predefined schema** and no embedded key-value structure that can be directly queried. The semantic content is encoded in the raw signal—pixels, audio samples, token sequences—and cannot be accessed by a relational or document query engine without substantial preprocessing (feature extraction, natural language processing, or computer vision).

Unstructured data is the dominant form of digital data, accounting for an estimated 70–80% of the global datasphere. It encompasses text documents (news articles, legal contracts, clinical notes, social media posts), images (medical scans, satellite photography, product photographs), audio recordings (call-centre conversations, podcasts, speech-enabled device logs), video (surveillance footage, user-generated content, manufacturing inspection recordings), and raw sensor streams (seismic signals, EEG waveforms, raw GPS trajectories).

The defining challenge of unstructured data is that extracting information requires *interpretation*—a step that structured and semi-structured data largely skip. A relational database can compute the average transaction amount in a single SQL aggregation; answering the question "what is the sentiment of customer reviews mentioning our new product?" requires tokenising text, building or loading a language model, running inference, and aggregating the model's output. This computational chain is far more complex and resource-intensive, which is why the tools introduced in later chapters—Spark MLlib, distributed deep learning frameworks, and specialised feature stores—are essential components of modern data engineering systems.

1.5 The Engineering Response to Scale

Given the 3Vs, the engineering question is: how do we build systems capable of storing petabytes, processing millions of events per second, and integrating dozens of heterogeneous data sources? The answer, which runs

Table 1.5: Comparison of data type characteristics and engineering implications

Dimension	Structured	Semi-structured	Unstructured
Schema	Predefined, rigid	Self-describing, flexible	Absent
Schema timing	On write	On read	N/A
Storage	RDBMS tables	Files, NoSQL, APIs	Object stores, HDFS
Query tool	SQL	JSONPath, Hive, Spark SQL	NLP, CV, Spark MLlib
Global share	~20%	~10%	~70–80%
Schema change cost	High	Low	N/A
Typical ingestion tool	Sqoop	Kafka, Flume	Custom pipelines
Examples	Bank ledger, ERP records	REST API JSON, server logs	MRI scan, audio call

through every chapter of this book, reduces to a small set of foundational principles.

Common Misconception | JSON is Semi-Structured, Not Unstructured

A frequent and consequential error: classifying JSON or XML as "unstructured" because it does not look like a database table. JSON and XML carry an embedded schema in the form of key names and hierarchical nesting. A query engine can navigate this structure directly (using JSONPath, XPath, or Hive's schema-on-read). An MRI scan, a JPEG image, or an MP3 recording carries *no* such navigable schema—its semantic content is encoded in the binary signal itself, requiring a domain-specific model (computer vision, speech recognition) to interpret. The distinction matters architecturally: semi-structured data can be stored and queried in a data lake with minimal transformation; unstructured data requires a preprocessing pipeline before it becomes

analytically useful.

1.5.1 Vertical vs. Horizontal Scaling

There are two ways to make a computing system more powerful. ***Vertical scaling*** (scaling up) means adding more resources to a single machine: a faster processor, more RAM, a larger or faster storage array. Vertical scaling is simple—it introduces no distributed-systems complexity—but it has a hard physical ceiling. The most powerful single-node servers available in 2024 top out at approximately 64 TB of RAM and a handful of high-throughput storage controllers. Above this limit, no amount of money can buy a single machine large enough to hold the dataset. Vertical scaling is also exponentially expensive: a server with twice the RAM costs approximately eight times as much.

Horizontal scaling (scaling out) means adding more machines to a cluster and distributing the data and computation across all of them. Each individual machine is a commodity server costing between three and eight thousand dollars. The power of the cluster grows nearly linearly with the number of nodes added: doubling the number of nodes approximately doubles the throughput. Failure of a single node—which is expected, not exceptional, in a large cluster—does not halt the system, because data is replicated across multiple nodes.

1.5.2 The Commodity Hardware Insight

The observation that underpins all of distributed data engineering is deceptively simple: *commodity hardware will fail, and systems must be designed with this expectation from the start*. In a cluster of 1,000 commodity servers, with each server having a disk failure rate of 1% per year, we expect approximately 10 disk failures per year—about one every five weeks. At 10,000 nodes (the scale of Yahoo!'s first production Hadoop cluster in 2008), this becomes roughly one disk failure per day. The solution is not to prevent failures—that is neither economically nor physically feasible—but to design for them. Every piece of data is stored in multiple copies on different nodes; if one node fails, the data remains available from the surviving copies, and the system automatically creates a new replica on a healthy node. This

Table 1.6: Vertical vs. horizontal scaling: a systematic comparison

Attribute	Vertical (Scale Up)	Horizontal (Scale Out)
Scalability ceiling	Hard physical limit (~64 TB RAM)	Near-unlimited (add nodes)
Cost scaling	Superlinear (exponential)	Near-linear per node
Fault model	Single point of failure	Tolerates individual node failures
Complexity	Low (no distributed systems)	High (consistency, coordination)
Latency for small queries	Low	Higher (network overhead)
Typical cost example	Oracle Exadata: >$500K/yr	1,000 nodes × $5K = $5M (one-time)
Used by	Traditional RDBMS, data warehouses	Hadoop, Spark, Cassandra, Kafka

design principle, first articulated for distributed file systems by Google's engineers (Ghemawat, Gobioff, and Leung 2003), is the cornerstone of HDFS, Kafka, and Cassandra alike.

1.5.3 Moving Computation to Data

The second foundational principle of distributed data processing inverts the traditional computing model. In a conventional system, data is moved to the processor: a query is issued, data is fetched from storage into RAM, and the CPU processes it. This model fails at petabyte scale because the network bandwidth required to move large amounts of data to a central processor becomes the bottleneck—the network simply cannot move data fast enough to keep the processor busy.

The solution, pioneered by Google's MapReduce system (Dean and Ghemawat 2004) and implemented in every major big data processing framework, is to *move computation to the data*: send the processing logic

(a function or query) to the node where the relevant data blocks reside, execute the computation locally, and collect only the (smaller) results. This principle—**data locality**—drastically reduces network traffic and enables the aggregate CPU capacity of an entire cluster to be applied in parallel to a large dataset.

System Design Note | Data Locality in Practice

In HDFS (Chapter 3), a 128 MB data block is stored on three nodes. When a MapReduce or Spark job must process that block, the scheduler first attempts to assign the task to one of the three nodes already holding a replica (task-local scheduling). If those nodes are busy, it next tries a node on the same physical rack (rack-local scheduling). Only if no rack-local node is available does it resort to sending the data over the network. In production clusters, over 90% of tasks execute with data locality, meaning almost all processing is local I/O rather than network transfer.

1.6 The CAP Theorem

Building a distributed system—one that spans multiple machines connected by a network—introduces a fundamental theoretical constraint that shapes every storage and database design decision in data engineering. This constraint is captured by the ***CAP theorem***.

1.6.1 Statement and Definitions

The CAP theorem was conjectured by Eric Brewer in his keynote address at the ACM Symposium on Principles of Distributed Computing in 2000 (Brewer 2000) and formally proved by Gilbert and Lynch two years later (Gilbert and Lynch 2002). It states:

Definition | The CAP Theorem (Brewer, 2000; Gilbert & Lynch, 2002)

A distributed data system can simultaneously guarantee **at most two** of the following three properties:

1. **Consistency (C):** Every read operation returns the most recent successfully written value, or an error. All nodes in the system see the same data at the same time.
2. **Availability (A):** Every request to a non-failed node receives a response—not guaranteed to be the most recent write, but a response nonetheless.
3. **Partition Tolerance (P):** The system continues to operate correctly even if an arbitrary number of messages between nodes are lost or delayed (a *network partition*).

Understanding what each property means in operational terms is essential for reasoning about system design. *Consistency*, in the CAP sense, is not the same as the C in ACID transactions (which refers to maintaining database invariants); CAP consistency is *linearisability*— the guarantee that every read sees the most recent write, as if all operations occurred on a single machine. *Availability* means every request gets a response; it does not mean the response contains current data. *Partition tolerance* means the system continues to function when the network splits into disconnected sub-clusters.

1.6.2 The Practical Implication: Choosing C vs. A

The critical practical insight is that network partitions in distributed systems are not theoretical edge cases—they are *inevitable*. Any system operating across multiple machines connected by a real network will, at some point, experience a partition: a switch failure, a misconfigured firewall rule, a miscabling in a data centre, or a brief network congestion event that causes messages to be delayed or dropped. Therefore, in any real distributed system, partition tolerance (P) is not optional. A system that cannot tolerate partitions is effectively a single-machine system.

Given that P is always required, the CAP theorem reduces to a binary choice: when a partition occurs, should the system prioritise **consistency** (refuse requests that cannot be answered with current data, guaranteeing accuracy) or **availability** (answer every request with possibly stale data, guaranteeing responsiveness)? This choice is one of the most important architectural decisions in data engineering.

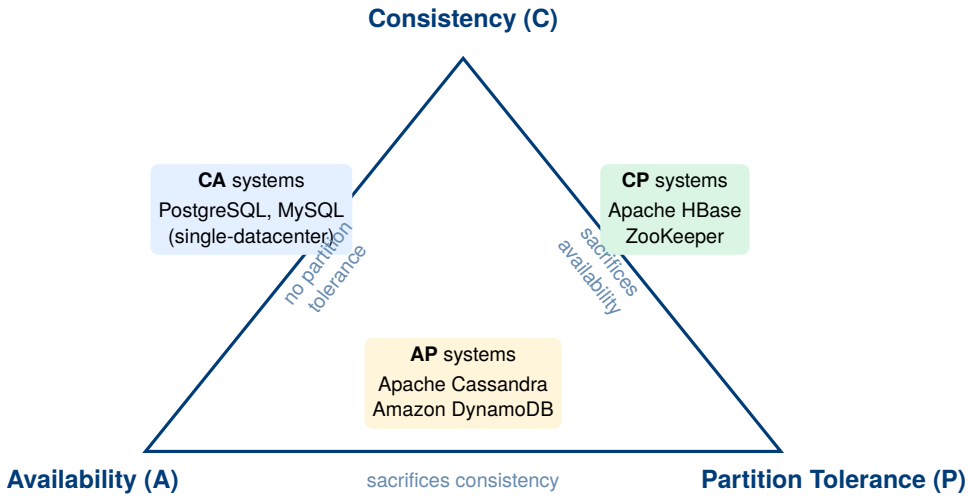

Figure 1.1: The CAP triangle: every distributed system chooses two of the three properties. Because network partitions are unavoidable, real systems choose between CP and AP.

Research Spotlight | Brewer (2000) and Gilbert & Lynch (2002) — The CAP Theorem

Citations:

- Brewer, E.A. (2000). *Towards Robust Distributed Systems*. Keynote, PODC 2000. (Brewer 2000)
- Gilbert, S. & Lynch, N.A. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2):51–59. (Gilbert and Lynch 2002)

Key insight: Brewer's conjecture, presented as intuition in 2000, was that three desirable properties of distributed systems could not all be simultaneously guaranteed. Gilbert and Lynch provided a formal proof two years later, elevating the observation from engineering folklore to a mathematically rigorous theorem.

Technical contributions:

- Provided the formal definitions of consistency (linearisability), availability, and partition tolerance used in distributed systems theory.
- Proved that no distributed system can achieve all three properties simultaneously using an asynchronous network model.

Table 1.7: CAP positioning of representative data systems

System	CAP	Design rationale
PostgreSQL, MySQL	CA	Designed for single-node or tightly coupled clusters; cannot tolerate true network partitions
Apache HBase	CP	Chooses consistency: if a partition occurs, HBase may become unavailable rather than serve stale data
Apache ZooKeeper	CP	Coordination service where serving stale configuration data would be catastrophic
Apache Cassandra	AP	Chooses availability: always responds, using tunable eventual consistency (read your own writes)
Amazon DynamoDB	AP	Optimised for always-on e-commerce workloads where availability is paramount

- Established the vocabulary (CA, CP, AP) that data architects use to describe and compare distributed systems.

Nuance and evolution: The CAP theorem is often oversimplified. Eric Brewer himself, in a 2012 retrospective, noted that the choice is not always a binary C-vs-A decision: systems like DynamoDB offer tunable consistency levels, allowing different operations to trade consistency for availability at the operation level. The PACELC model (Patterson et al., 2012) extends CAP by noting that even without a partition, systems must trade latency for consistency—a trade-off CAP does not capture. Despite these refinements, the CAP theorem remains the most widely cited result in distributed systems theory.

1.7 Batch vs. Stream Processing

The Velocity dimension of big data—the need to process data that arrives continuously and at high rates—has driven the development of two fundamentally different processing paradigms that data engineers must understand and choose between.

1.7.1 Batch Processing

Batch processing operates on a fixed, bounded dataset accumulated over a period of time. The data is first stored in full (on disk, in a distributed file system, or in a data warehouse), and then a computation—the *batch job*—is run over the entire stored dataset. Classic examples include nightly financial reconciliation, end-of-month billing, weekly machine learning model retraining, and annual census analysis.

Batch processing has several important properties. First, it achieves very high *throughput*: because the entire dataset is available before processing begins, the job can be optimised for sequential I/O, sorted reads, and massive parallelism. Second, batch jobs are inherently *reproducible*: re-running the same job on the same input data will always produce the same output—a property critical for auditing and debugging. Third, batch jobs can be *fault-tolerant* in a simple way: if a task fails, it can be restarted from the last checkpoint without affecting the rest of the job. Fourth, and most importantly for our purposes, batch processing has *latency measured in hours*: a nightly ETL job that begins at 02:00 and finishes at 06:00 delivers results four hours after the data cutoff. For applications that need to respond to data within seconds or milliseconds, this latency is unacceptable.

1.7.2 Stream Processing

Stream processing operates on an unbounded, continuously arriving sequence of data records. Rather than accumulating data and running a batch job, a stream processing engine processes each record (or a small micro-batch of records) as it arrives, maintaining running state across the stream. The key characteristic is *low latency*: a stream processor can emit results within milliseconds to seconds of each event's arrival.

Stream processing is the appropriate model for applications where the value of a response decays rapidly with time: real-time fraud detection (a stolen card must be blocked before the second transaction clears, not after nightly batch analysis); live stock market monitoring (an algorithmic trading system needs the current quote, not yesterday's close); operational dashboards (a call-centre manager needs to see current queue depth, not yesterday's average); and anomaly detection in industrial systems (a turbine failure signal is valuable in real time, not eight hours later).

Stream processing introduces engineering challenges that batch processing avoids. Records may arrive *out of order* (a mobile device's network connection may be intermittent, causing events to be delayed and arrive after later events). Events may carry an *event time* (when something happened) that differs from *processing time* (when the event arrived at the system). Stateful computations—for example, counting the number of page views per user in the past 60 minutes—must be maintained across an infinite stream, requiring careful memory management and state backends. Fault recovery requires mechanisms like *exactly-once processing* semantics, which add significant implementation complexity.

Key Insight | Lambda and Kappa Architectures

Many production systems need both low latency *and* high accuracy—properties that are difficult to achieve simultaneously. The **Lambda architecture** (Nathan Marz, 2011) addresses this by running both a batch layer (for accurate, comprehensive historical analysis) and a speed layer (for low-latency approximate results) in parallel, merging their outputs in a serving layer. The cost is engineering complexity: every transformation must be implemented twice, in different frameworks. The **Kappa architecture** (Jay Kreps, 2014) simplifies this by replacing the batch layer with a replay of the full event log through the stream processor—using Apache Kafka's message retention to make historical reprocessing possible through the streaming path. Both architectures are examined in depth in Chapter **??**.

1.8 Case Study: Walmart's Retail Analytics Platform

Table 1.8: Batch vs. stream processing: a systematic comparison

Attribute	Batch processing	Stream processing
Data model	Bounded, stored dataset	Unbounded, continuously arriving events
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high	Moderate (per-event overhead)
Fault recovery	Re-run from checkpoint	Exactly-once semantics required
State management	Simple (no persistent state)	Complex (stateful operators, windows)
Out-of-order data	Not a concern	Central challenge (watermarks)
Debugging	Easy (fixed inputs/outputs)	Hard (live, non-reproducible)
Typical tools	Hadoop MapReduce, Hive, Spark Batch	Kafka, Apache Flink, Spark Streaming
Use cases	ETL, ML training, billing	Fraud detection, monitoring, alerts

Case Study | Walmart — Petabyte-Scale Retail Intelligence

Background. Walmart is the world's largest retailer by revenue, operating over 10,000 stores across 27 countries and serving approximately 230 million customers per week. The scale of its data generation is correspondingly vast: every store transaction, supply chain event, website interaction, and third-party demand signal contributes to what is one of the largest private-sector data repositories in existence.

Scale. Walmart's internal data analytics environment holds over **2.5 petabytes** of customer transaction data in its core analytics systems, with an additional multi-petabyte tier of raw, unstructured data in its data lake. The company processes roughly one million customer transactions *per hour*—approximately 280 transactions per second on a typical day, with significant spikes during seasonal events (Thanksgiving weekend in the United States generates transaction volumes three to four times the average daily rate).

Data variety. Walmart's analytical systems must integrate across all three data categories described in this chapter:

- **Structured:** Point-of-sale transaction records (item, quantity, price, timestamp, store ID, payment method) from 10,000+ stores, all originally stored in Oracle RDBMS systems.
- **Semi-structured:** Web and mobile clickstream events in JSON format (product views, search queries, cart additions, checkout abandonment events) from Walmart.com.
- **Unstructured:** Customer product reviews, call-centre transcript audio, and supplier-submitted product imagery for catalogue enrichment.

Engineering challenges and solutions. The 3Vs framework maps directly onto Walmart's operational realities:

- **Volume:** Transaction data accumulated over decades cannot fit in any single RDBMS. Walmart migrated its analytical workloads to a Hadoop-based data lake, using HDFS for distributed storage and Apache Hive for SQL-style querying over PB-scale datasets.
- **Velocity:** Supply chain decisions—whether to redistribute inventory across distribution centres—must respond to real-time demand signals, not yesterday's batch report. Walmart deployed Apache Kafka for real-time event streaming and uses stream processing for demand sensing.
- **Variety:** Integrating transaction records, clickstream events, weather data (a known driver of consumer behaviour), and social media sentiment requires a data integration pipeline that handles schema mismatches, different temporal granularities, and inconsistent entity

representations (the same product may have different identifiers in the RDBMS and the web catalogue).

Business outcomes. Walmart's investment in big data engineering has enabled demand forecasting accuracy that reduced food waste by 15–20% in its grocery divisions (reducing spoilage by anticipating demand more precisely), and dynamic pricing algorithms that run across its full product catalogue multiple times per day. During the COVID-19 pandemic, real-time analytics over social media and purchase patterns allowed Walmart to anticipate demand surges for sanitisers, masks, and canned goods several days before they peaked—enabling pre-positioning of inventory that proved critical when supply chains became constrained.

Lesson. Walmart's evolution from single-RDBMS analytics to a multi-petabyte distributed data platform illustrates the core thesis of this book: the 3Vs are not abstract concepts but concrete engineering constraints that force specific architectural choices. Every tool introduced in subsequent chapters exists because organisations like Walmart encountered these constraints and could not solve them with conventional technology.

Chapter Summary

This chapter established the conceptual and theoretical foundations for the rest of the book. The key points are:

- The global datasphere is projected to reach 175 ZB by 2025, driven by social media, IoT, commerce, and scientific instrumentation. Traditional RDBMS cannot manage data at this scale.
- **Data engineering** is the discipline of building infrastructure that makes large-scale data available, reliable, and efficiently queryable for downstream consumers.
- The **3Vs framework** (Laney, 2001) characterises big data along three dimensions: *Volume* (scale), *Velocity* (speed), and *Variety* (diversity). The extended **5Vs** adds *Veracity* (data quality) and *Value* (business utility).

- Digital data falls into three categories: **structured** (predefined schema, schema-on-write, ~20% of global data), **semi-structured** (self-describing schema, schema-on-read, ~10%), and **unstructured** (no schema, requires ML/NLP/CV to interpret, ~70–80%).
- **Horizontal scaling** (adding commodity nodes) is the dominant strategy for big data infrastructure because it is cost-effective, fault-tolerant, and has no hard ceiling. Moving **computation to data** (data locality) is the principle that makes distributed computation efficient.
- The **CAP theorem** proves that any distributed system can guarantee at most two of: Consistency, Availability, and Partition Tolerance. Since partitions are unavoidable, the real design choice is CP vs. AP.
- **Batch processing** offers high throughput and simplicity for bounded datasets but incurs high latency. **Stream processing** offers low latency for continuously arriving data but adds complexity. Production systems often combine both paradigms (Lambda or Kappa architectures).

Exercises

Short Questions

SQ1.1. State the 3Vs of big data and give one production-scale numerical example for each dimension.

SQ1.2. What is the CAP theorem? List the three properties and explain what each means in operational terms.

SQ1.3. A single high-end server in 2024 has approximately 64 TB of RAM. Is a 100 PB dataset "big data" by the Volume definition? Justify your answer quantitatively.

SQ1.4. Classify each of the following as structured, semi-structured, or unstructured. Justify each answer in one sentence.

(a) A table of stock prices in a PostgreSQL database.

(b) A REST API response in JSON format from a weather service.

(c) An MP3 recording of a customer service call.

(d) An XML document containing a purchase order.

(e) A satellite image of an agricultural field.

SQ1.5. What is "schema-on-read"? How does it differ from "schema-on-write," and in which type of data is each used?

SQ1.6. Explain the concept of "data locality" in distributed processing. Why does moving computation to data (rather than data to computation) improve performance at petabyte scale?

SQ1.7. An organisation's fraud detection system must decide whether to approve a payment within 300 milliseconds. Should it use batch or stream processing? Explain why.

SQ1.8. Apache Cassandra is described as an AP system. What does this mean concretely? Give a scenario where choosing Cassandra over a CP system (like HBase) is the correct engineering decision, and a scenario where it is not.

SQ1.9. What is the difference between *event time* and *processing time* in stream processing? Why does this distinction matter for computing aggregations over a stream?

SQ1.10. Identify the primary V (Volume, Velocity, or Variety) that dominates each scenario:

(a) A genomics research lab accumulating 200 TB of whole-genome sequencing data per month.

(b) A payment network processing 24,000 card swipes per second.

(c) An enterprise integrating data from 40 departments, each using a different database vendor and schema.

SQ1.11. What is the Veracity dimension of the 5Vs framework? Give two concrete examples of low-veracity data from different domains.

SQ1.12. Compare the cost-scaling behaviour of vertical and horizontal scaling. Why is vertical scaling said to be "superlinear" in cost?

SQ1.13. What is the Lambda architecture? What problem does it solve, and what is its primary drawback?

SQ1.14. True or false, with a one-sentence justification: "A relational

database system can always be made to handle big data workloads by upgrading to a more powerful server."

Analytical Problems

AP1.1. Data Volume Estimation. A financial services company ingests stock market quote updates at a rate of 1.2 million messages per second during trading hours (6.5 hours per day). Each message is 256 bytes on average, and HDFS stores three replicas of all data. Calculate:

(a) The raw volume of quote data generated per trading day (in GB).

(b) The total HDFS storage consumed per trading day (with 3× replication).

(c) The annual storage requirement (250 trading days per year), in TB.

(d) If each HDFS DataNode has 48 TB of usable disk capacity, what is the minimum number of DataNodes needed to store one year of data?

AP1.2. CAP Theorem Analysis. For each of the following systems, determine whether its designers should choose CP or AP, and explain the reasoning in terms of what happens when a network partition occurs and the system must choose between C and A.

(a) An inventory management system for an online retailer. Showing a product as "in stock" when it is actually sold out results in order cancellations and customer dissatisfaction; showing it as "out of stock" when it is available causes lost sales.

(b) A distributed configuration registry used by microservices to discover the addresses of other services. Serving a stale configuration might cause a service to connect to a decommissioned endpoint.

(c) A social media "like" counter. The precise count at any given millisecond is not critical; users expect the counter to update within a few seconds.

AP1.3. Batch vs. Stream Latency Analysis. A credit card company runs a fraud detection system. The fraudster's behaviour pattern (multiple small test transactions followed by a large transaction) is detectable within a 15-minute time window. The company currently runs a batch fraud detection

job that starts at midnight and finishes by 05:00. Show, with a timeline analysis, why this batch job cannot prevent in-session fraud. Estimate the maximum percentage of daily fraud that could be caught by a streaming system with a 10-second detection latency, assuming fraud transactions are uniformly distributed throughout the day.

AP1.4. Scaling Economics. A company must build a system to store and query a 500 TB analytical dataset. Option A is a vertically scaled Oracle Exadata appliance (licensed at $1.2M/year, maximum capacity 500 TB). Option B is a horizontally scaled HDFS cluster of commodity servers at $6,000 per node (12 TB usable per node), with HDFS's 3× replication factor.

(a) How many DataNodes does Option B require?

(b) What is the hardware capital cost of Option B?

(c) If the dataset is expected to grow at 30% per year, how many years before Option A requires a hardware replacement (assume it cannot be expanded beyond 500 TB), and what would Option B's cost be to scale to the equivalent capacity?

(d) What non-monetary factors also influence this architectural choice?

Design Problems

DP1.1. Architecture Selection for a Global E-commerce Platform. You are the lead data engineer at a mid-sized e-commerce company with operations in 12 countries. The company processes 85 million orders per year and must support the following data workloads:

- Monthly financial reconciliation reports (all orders for the month).
- Real-time fraud scoring for each checkout event (must respond within 500 ms).
- A product recommendation engine that retrains weekly on full order and clickstream history.
- Live operational dashboards showing current order throughput and error rates by country.

For each workload, identify: (a) the dominant V dimension it stresses, (b)

whether batch or stream processing is appropriate, and (c) which CAP trade-off the underlying storage must make. Then propose a single integrated architecture (a sketch is sufficient) that serves all four workloads, and identify the single component that is the highest engineering risk.

DP1.2. Data Classification and Integration Strategy. A national weather service wants to build an analytics platform integrating the following data sources:

- Hourly temperature and humidity readings from 12,000 automated weather stations (CSV format, streamed via SFTP).
- Real-time radar imagery in TIFF format, updated every 6 minutes.
- Historical weather observations from 1950 to present, stored in a legacy Oracle database with a proprietary schema.
- Citizen-submitted weather reports via a mobile app (JSON, free-text description field included).
- Satellite cloud-cover imagery (binary NetCDF format).

For each source: (a) classify the data type (structured/semi-structured/unstructured), (b) identify the dominant V dimension, and (c) propose an ingestion mechanism. Then identify the two most difficult integration challenges when combining these sources into a unified query layer, and propose mitigations for each.

Programming Exercises

PE1.1. Data Type Classifier (Python). The following list of data source descriptions must be classified as `structured`, `semi_structured`, or `unstructured`. Implement the `classify_source()` function using keyword-based heuristics. Then extend your solution to print the recommended primary storage technology for each category.

```
sources = [
    {"name": "Customer transaction records in PostgreSQL",
     "format": "relational_table", "schema": "predefined"},
    {"name": "HTTP server access log",
     "format": "text",   "schema": "implicit"},
    {"name": "Satellite multispectral imagery (GeoTIFF)",
```

```
     "format": "binary", "schema": "none"},
    {"name": "Product catalog (XML)",
     "format": "xml",    "schema": "self_describing"},
    {"name": "Customer service call recording (MP3)",
     "format": "audio",  "schema": "none"},
    {"name": "E-commerce clickstream events (JSON)",
     "format": "json",   "schema": "self_describing"},
    {"name": "Annual census data (CSV with fixed columns)",
     "format": "csv",    "schema": "predefined"},
]

STORAGE_MAP = {
    "structured":       "Relational RDBMS (PostgreSQL, MySQL)
        or columnar DW",
    "semi_structured":  "Data lake (HDFS/S3) + Apache Hive /
        Spark SQL",
    "unstructured":     "Object store (S3, Azure Blob) + ML
        feature pipeline",
}

def classify_source(source):
    """
    Returns 'structured', 'semi_structured', or 'unstructured'.
    Hint: use source['format'] and source['schema'] fields.
    """
    # TODO: implement classification logic
    pass

for s in sources:
    label = classify_source(s)
    print(f"Source  : {s['name']}")
    print(f"Category: {label}")
    print(f"Storage : {STORAGE_MAP.get(label, 'Unknown')}")
    print()
```

Listing 1.2: PE1.1: Data type classifier

PE1.2. CAP Trade-off Decision Matrix (Python). Given a set of system requirements, write a function that recommends the appropriate CAP trade-off and a supporting rationale. Then simulate what happens under a network partition for each choice.

```
requirements = [
    {"system": "Banking ledger (account balances)",
```

```
     "stale_read_acceptable": False,
     "unavailability_acceptable": True,
     "description": "Must never show incorrect balance"},
    {"system": "Social media post counter (likes)",
     "stale_read_acceptable": True,
     "unavailability_acceptable": False,
     "description": "Slight delay in count update is tolerable"
         },
    {"system": "Distributed lock manager for microservices",
     "stale_read_acceptable": False,
     "unavailability_acceptable": True,
     "description": "Two services must never acquire the same
         lock"},
    {"system": "Product search index",
     "stale_read_acceptable": True,
     "unavailability_acceptable": False,
     "description": "Returning yesterday's index is acceptable"
         },
]

def cap_recommendation(req):
    """
    Returns a tuple: (cap_choice, rationale_string)
    where cap_choice is 'CP' or 'AP'.

    Logic:
    - If stale reads are NOT acceptable -> CP
      (system must be consistent even if it becomes unavailable
          )
    - If stale reads ARE acceptable     -> AP
      (system must stay available even with possibly stale data
          )
    """
    # TODO: implement decision logic and rationale string
    pass

def simulate_partition(cap_choice, system_name):
    """
    Print what happens to the system during a network partition
        .
    """
    if cap_choice == "CP":
        print(f"  Partition occurs in {system_name}:")
        print(f"  -> System refuses requests on isolated nodes.
```

```
            ")
        print(f"  -> Availability drops; consistency maintained.")
    elif cap_choice == "AP":
        print(f"  Partition occurs in {system_name}:")
        print(f"  -> System continues serving requests on isolated nodes.")
        print(f"  -> Data may be stale; eventual consistency applies.")

for req in requirements:
    choice, rationale = cap_recommendation(req)
    print(f"System : {req['system']}")
    print(f"Choice : {choice}  |  Rationale: {rationale}")
    simulate_partition(choice, req['system'])
    print()
```

Listing 1.3: PE1.2: CAP trade-off recommender

PE1.3. Storage Volume Estimator (Python). Implement a function that estimates the raw and replicated HDFS storage requirements for a streaming data source, and classifies the result as "conventional" (manageable with a single RDBMS), "big data" (requiring distributed storage), or "hyperscale" (requiring cloud object storage tiers in addition to HDFS).

```
def estimate_hdfs_storage(records_per_second, avg_record_bytes,
                          retention_days, replication_factor=3):
    """
    Parameters
    ----------
    records_per_second  : float  -- sustained ingestion rate
    avg_record_bytes    : int    -- average size of one record
    retention_days      : int    -- how long to retain data
    replication_factor  : int    -- HDFS replication (default 3)

    Returns
    -------
    dict with keys:
      raw_tb            : raw data in terabytes
      replicated_tb     : total HDFS storage in terabytes (with replication)
```

```
      replicated_pb     : total HDFS storage in petabytes
      classification    : 'conventional' | 'big_data' | '
          hyperscale'
    """
    BYTES_PER_TB = 1e12
    BYTES_PER_PB = 1e15

    # TODO: implement computation
    # Steps:
    # 1. Compute bytes per second = records_per_second *
        avg_record_bytes
    # 2. Compute bytes per day = bytes_per_second * 86400
    # 3. Compute raw_bytes = bytes_per_day * retention_days
    # 4. Compute replicated_bytes = raw_bytes *
        replication_factor
    # 5. Convert to TB and PB
    # 6. Classify:
    #    < 50 TB   -> 'conventional'
    #    50 TB - 10 PB -> 'big_data'
    #    > 10 PB   -> 'hyperscale'
    pass

scenarios = [
    {"name": "IoT temperature sensors (500K devices, 1 reading/
        min)",
     "rps": 500_000 / 60, "bytes": 64, "days": 365},
    {"name": "E-commerce clickstream (peak 200K events/s, 90-
        day retention)",
     "rps": 200_000,      "bytes": 512, "days": 90},
    {"name": "Payment network (25K tx/s, 7-year regulatory
        retention)",
     "rps": 25_000,       "bytes": 256, "days": 2555},
    {"name": "Video streaming events (50K concurrent sessions,
        1 event/s)",
     "rps": 50_000,       "bytes": 1024, "days": 180},
]

for s in scenarios:
    result = estimate_hdfs_storage(s["rps"], s["bytes"], s["
        days"])
    if result:
        print(f"Scenario     : {s['name']}")
        print(f"Raw storage  : {result['raw_tb']:.1f} TB")
        print(f"With 3x rep. : {result['replicated_pb']:.3f} PB
```

```
            ")
        print(f"Class        : {result['classification']}")
        print()
```

Listing 1.4: PE1.3: HDFS storage volume estimator

2

Data Integration: Heterogeneity, Quality, and Provenance

"The hardest part of data science isn't machine learning or statistics. It's getting data from different places to agree with each other."

Anonymous data engineer

Learning Objectives

After completing this chapter, you will be able to:

1. Explain the data silo problem and articulate why integration complexity—not storage cost—is the primary barrier to enterprise analytics.
2. Define the three fundamental operations of data integration: schema mapping, data exchange, and entity resolution.
3. Distinguish among the four dimensions of data heterogeneity—structural, semantic, syntactic, and scale—and give a concrete example of each.
4. Describe the six dimensions of data quality and identify appropriate remediation strategies for each.
5. Explain what data provenance is, why it is required by regulations such as GDPR and HIPAA, and how lineage tracking systems capture

it.

6. Compare the ETL and ELT paradigms: their architectures, trade-offs, and appropriate use cases.
7. Identify the right tool (Sqoop, Kafka, Airbyte, dbt) for each stage of a modern integration pipeline.

Chapter 1 established the 3Vs as the defining characteristics of big data: Volume, Velocity, and Variety. Volume and Velocity receive the most attention in the popular press because they are easy to quantify—petabytes of storage, thousands of events per second. But *Variety* is arguably the hardest engineering challenge. Data arrives from dozens of independent sources, each with its own schema, format, quality level, and update cadence. Before a single analytical query can be answered, all of this heterogeneous data must be identified, mapped, cleaned, and unified. This process is called ***data integration***, and it is the subject of this chapter.

A 2019 Forrester Research survey found that 73% of enterprise data is never analysed. The primary reason is not a shortage of storage or processing power—it is integration complexity. Data that cannot be found, joined, or trusted cannot be analysed. Data engineering at production scale is, to a greater degree than is often acknowledged, the art of making heterogeneous data coherent.

2.1 The Data Silo Problem

Large organisations accumulate data in ***data silos***: independent systems built by separate teams, at different times, for different purposes, using different technology stacks. Each silo does its job well in isolation, but no silo has visibility into the others. The result is an organisation that is simultaneously data-rich and insight-poor.

Consider a representative global retail bank. A customer who has held an account for five years may appear as a distinct record in at least seven separate systems: the core banking RDBMS (account balances, transactions), the mobile application database (session events, in-app behaviour), the loan

origination system (credit applications, repayment history), the customer relationship management platform (support tickets, call transcripts), the anti-money-laundering engine (transaction flags, compliance cases), the marketing automation platform (campaign responses, email open rates), and the regulatory reporting database (filings with supervisory authorities). In each system, the customer may be identified by a different key—an internal account number, a mobile device identifier, a CRM contact ID, a government-issued reference number—and the same attribute (the customer's full name, for example) may be spelled differently, encoded differently, or stored at different granularities.

The consequence is not merely an engineering inconvenience. It is a fundamental barrier to analytical insight: a data scientist wishing to understand the relationship between a customer's loan repayment behaviour and their mobile application engagement cannot combine these datasets without first resolving which records in the loan system correspond to which records in the mobile database—a non-trivial operation even for a single bank with thousands of customer records, let alone for a global institution with hundreds of millions.

Key Insight | Integration is the Critical Path

In most data engineering projects, the time spent on data integration—schema mapping, entity resolution, quality cleaning, and pipeline construction—exceeds the time spent on the analytical computation itself. Studies of data science workflows consistently find that engineers spend 60–80% of project time on data preparation rather than on modelling or analysis. Integration quality directly determines analytical quality: a machine learning model trained on incorrectly joined data will encode the integration error into its predictions.

2.2 A Formal Framework for Data Integration

The academic study of data integration has a rich history spanning decades of database research. Halevy, Rajaraman, and Ordille's landmark 2006 survey—titled "Data Integration: The Teenage Years"—synthesised thirty

years of work and established the formal vocabulary that the field still uses today (Halevy, Rajaraman, and Ordille 2006).

Definition | Data Integration

Given a set of heterogeneous, autonomous data sources $S_1, S_2, \ldots, S_n$, each with its own schema Σ_i and content D_i, **data integration** is the problem of providing a unified query interface over a **mediated (global) schema** Σ_G that gives users a single coherent view of all data, regardless of which source it originated from.

This definition makes three important properties explicit. First, data sources are *heterogeneous*: they differ in schema, format, vocabulary, and update cadence. Second, they are *autonomous*: the integration layer has no authority to modify the source systems—it can only read from them. Third, the goal is a *unified query interface*: users should be able to ask a single query against the global schema without knowing which source holds the relevant data or in what format.

Achieving this goal requires three core operations, each of which is a research area in its own right.

2.2.1 Schema Mapping

Schema mapping is the process of defining a formal correspondence between the schema of a source system and the global mediated schema. A schema mapping $\mathcal{M}_{i \to G}$ specifies, for each attribute in the global schema, which attribute in source S_i provides the corresponding data, and what transformation (if any) is required to reconcile differences in naming, data type, units, or encoding.

Schema mappings are expressed as rules. For example:

$$\texttt{GlobalSchema.customer\_id} \leftarrow \texttt{CRM.contact\_id} \quad [\text{identity}] \tag{2.1}$$

$$\texttt{GlobalSchema.full\_name} \leftarrow \texttt{uppercase(ATM.CUST\_FNAME+" "+ATM.CUST\_LNAME)} \tag{2.2}$$

In practice, deriving accurate schema mappings is one of the most labour-intensive tasks in data engineering. Automated ***schema matching*** tools—which use string similarity, structural similarity, and machine learning

to propose candidate mappings—can reduce this effort significantly, but they always require human validation before production deployment.

2.2.2 Data Exchange

Data exchange is the process of *materialising* data from source schemas into the global schema. It is the implementation of a schema mapping: given source data D_i and a mapping $\mathcal{M}_{i \to G}$, produce a dataset D_G that conforms to the global schema Σ_G. Data exchange corresponds to the "Transform" step in an ETL pipeline (discussed in Section 2.7).

2.2.3 Entity Resolution

Entity resolution (also called ***record linkage***, ***deduplication***, or ***identity resolution***) is the process of determining whether two records from different sources (or from the same source) refer to the same real-world entity. It is necessary when different systems use different identifiers for the same object and when direct key-based joins are therefore impossible.

Entity resolution is examined in detail in Section 2.4.

Research Spotlight | Halevy, Rajaraman & Ordille (2006) — Data Integration: The Teenag

Citation: Halevy, A., Rajaraman, A., & Ordille, J. (2006). Data Integration: The Teenage Years. *Proceedings of the 32nd VLDB Conference*, pp. 9–16. (Halevy, Rajaraman, and Ordille 2006)

Key insight: The paper's title is a pointed metaphor: after three decades of research, data integration remained "in its teenage years"—having demonstrated proof of concept and produced fundamental theory, but not yet reaching the robust, deployable maturity that industry needed.

Technical contributions:

- Provided a comprehensive taxonomy of the data integration problem space, distinguishing virtual integration (query rewriting over live sources) from materialised integration (ETL into a data warehouse).
- Formalised the distinction between *local-as-view* (LAV) and *global-as-view* (GAV) approaches to schema mapping— still the canonical framework for integration system design.
- Identified entity resolution as the hardest open problem in data

integration, predicting (correctly) that it would require probabilistic and machine learning approaches at scale.

Impact today: The paper is required reading in database courses at MIT, Stanford, and CMU. Its taxonomy of integration challenges directly influenced the design of modern enterprise data integration platforms, including MuleSoft, Informatica, and Apache Atlas. The LAV/GAV distinction remains the theoretical foundation of modern data virtualisation products.

2.3 The Four Dimensions of Data Heterogeneity

Heterogeneity—the fundamental barrier to integration—manifests along four distinct dimensions. Understanding all four is essential because a pipeline that addresses only one or two will produce subtly incorrect results, often without any obvious indication of failure.

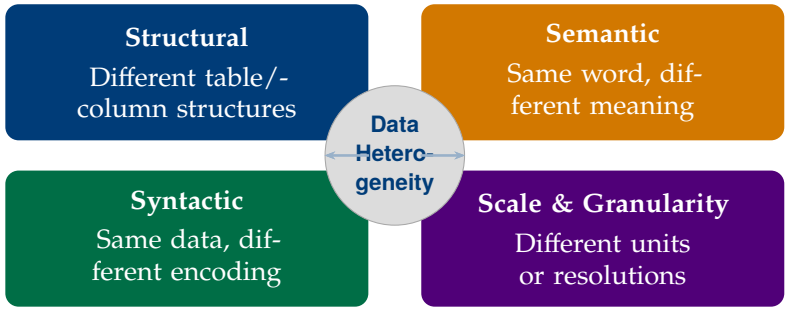

Figure 2.1: The four dimensions of data heterogeneity. A production integration pipeline must address all four simultaneously.

2.3.1 Structural Heterogeneity

Structural heterogeneity arises when two data sources represent the same real-world concept using different table organisations, column names, or normalisation levels. It is the most immediately obvious form of heterogeneity and the first that integration engineers typically address.

Naming conflicts occur when the same attribute has different names in different systems. A customer identifier might be called `cust_id` in the core banking system, `customer_number` in the loan system, and `contact_id` in the CRM. Without a mapping, a JOIN across these tables produces no results.

Structural conflicts occur when the same data is stored at different levels of normalisation. One system may store a customer's address as a single string (`"14 Oxford Street, London, W1D 1AU"`), while another uses four separate columns (`street_line_1`, `city`, `region`, `postcode`). Converting between these representations requires both parsing and composing—two operations that are error-prone when addresses do not conform to expected patterns.

Table 2.1: Structural heterogeneity: a customer record in three systems

Concept	Core Banking	CRM	ATM Logs
Customer ID	`cust_id`	`contact_id`	`CARD_HOLDER`
First name	`first_name`	`fname`	`FN`
Date of birth	`dob` (DATE)	`birth_date` (VARCHAR)	absent
Address	4-column normalised	Single string	absent
Account open date	`open_date`	`create_ts`	absent

2.3.2 Semantic Heterogeneity

Semantic heterogeneity—the most dangerous form—arises when two data sources use the same name to mean different things, or different names to mean the same thing. A schema mapping that resolves structural conflicts (renaming columns) will silently produce incorrect results if the underlying semantic conflict is not also resolved.

Classic examples are pervasive:

- The column `price` in the inventory system records the wholesale cost; the same column in the e-commerce system records the retail price. Joining these tables without disambiguation produces a dataset where "price"

has a different meaning for every row.

- The field `date` in an order management system could mean the order date, the shipping date, or the invoice date, depending on which team created the table. All three share the same column name in different tables.
- The concept of a "customer" means a natural person in the retail banking system, but can also mean a legal entity (company) in the corporate banking system. A JOIN on `customer_id` would mix individuals and corporations.

Semantic heterogeneity cannot be detected by examining schemas alone—it requires domain knowledge, documentation review, and often direct consultation with the teams that created each system. This is why data dictionaries and business glossaries are not administrative overhead but engineering necessities.

2.3.3 Syntactic Heterogeneity

Syntactic heterogeneity arises when the same data is encoded in different formats, character encodings, or data types across sources. The value and meaning are equivalent, but the byte-level representation differs. Syntactic heterogeneity must be resolved before any comparison or JOIN can succeed.

Representative examples include:

- Dates stored as `"2024-03-15"` (ISO 8601, universally sortable) in one system and as `"15/03/2024"` (UK format) or `"03-15-2024"` (US format) in another. A lexicographic sort on the date column produces incorrect ordering when formats are mixed.
- Boolean values represented as `true/false` (JSON), `1/0` (integer), `"Y"/"N"` (legacy COBOL systems), or `"yes"/"no"` (human-entered data).
- Currency amounts stored as 64-bit floating-point numbers in one system and as decimal strings (e.g., `"1,234.56"` or `"1.234,56"` for European locale) in another.
- Character encodings: legacy mainframe systems frequently use EBCDIC encoding; modern web applications use UTF-8; some regional systems use Windows-1252 or ISO-8859 variants.

2.3.4 Scale and Granularity Heterogeneity

Scale and granularity heterogeneity arises when different sources measure the same quantity at different units, resolutions, or temporal granularities. Even when structural, semantic, and syntactic conflicts have been resolved, a mismatch in granularity can invalidate comparisons.

A revenue report that joins monthly aggregates from one system with daily transaction records from another must aggregate the latter before joining; failing to do so produces a systematic distortion where each monthly figure appears 30 times. Similarly, a climate analytics system integrating temperature readings from stations reporting in Celsius and others in Fahrenheit must apply a unit conversion before any cross-station analysis; a system integrating GPS coordinates at metre precision with satellite imagery at 10-metre resolution must account for the resolution mismatch when attributing imagery to ground-truth measurements.

Production Lesson | Integration Failures Are Silent

Of the four heterogeneity dimensions, structural and syntactic failures are the easiest to detect: they typically produce explicit errors (column not found, type mismatch, parse failure). Semantic and granularity failures are far more dangerous because they produce *no error*—the pipeline succeeds, data flows, and the output looks superficially correct. A data warehouse that has silently mixed wholesale prices with retail prices will answer every price-related query without complaint, producing reports that are confidently and systematically wrong. Detecting semantic errors requires domain knowledge and statistical profiling of the integrated output, not just schema validation.

2.4 Entity Resolution

Even after schema mappings have been defined and all four heterogeneity dimensions addressed, a fundamental problem remains: how do we know that record r_1 from source S_1 and record r_2 from source S_2 refer to the same real-world entity? When a common key exists (both systems use a social security number or a product EAN barcode), the answer is trivial: join on

the key. When no common key exists—the common case—the problem is ***entity resolution***.

Definition | Entity Resolution

Given two sets of records R_1 and R_2 from sources S_1 and S_2 respectively, **entity resolution** is the task of identifying all pairs $(r_1, r_2) \in R_1 \times R_2$ that refer to the same real-world entity, using only the attribute values of the records (and no common key).

The naive approach—comparing every record in R_1 against every record in R_2—has complexity $O(|R_1| \times |R_2|)$. For datasets with millions of records, this is computationally infeasible. A dataset of 10 million customer records from two sources would require 100 trillion comparisons—at a rate of one billion comparisons per second, that is over 27 hours. At big data scale, the quadratic cost is prohibitive.

2.4.1 Blocking: Reducing the Candidate Space

Blocking is the standard technique for making entity resolution computationally tractable. Rather than comparing all possible pairs, blocking partitions records into ***blocks*** based on a simple attribute (or combination of attributes) that is likely to match for true duplicates. Only pairs within the same block are compared. A common blocking key is the first three characters of the last name combined with the year of birth; two records can only be a match if they share the same block key.

A well-designed blocking strategy must balance two competing objectives. *Recall* requires that almost all true duplicates fall in the same block—if a blocking key is too aggressive, it splits true duplicates into different blocks and they are never compared. *Efficiency* requires that blocks be small enough to make comparison tractable—if a blocking key is too lenient, blocks are large and the quadratic comparison cost returns. In practice, multiple blocking keys are used in parallel, and a pair is a candidate if it shares at least one blocking key.

2.4.2 Similarity Functions

Within each block, records are compared using ***similarity functions*** that assign a score $\sigma(r_1, r_2) \in [0, 1]$ to each candidate pair. Two commonly used functions are:

Jaccard similarity operates on sets of tokens. Given two strings s_1 and s_2 tokenised into sets of words or character n-grams T_1 and T_2:

$$J(T_1, T_2) = \frac{|T_1 \cap T_2|}{|T_1 \cup T_2|} \tag{2.3}$$

Jaccard similarity is robust to word reordering and partial matches, making it well-suited for company names and addresses.

Levenshtein (edit) distance measures the minimum number of single-character insertions, deletions, or substitutions needed to transform one string into another. Normalised edit similarity is:

$$\sigma_{\text{edit}}(s_1, s_2) = 1 - \frac{d_{\text{edit}}(s_1, s_2)}{\max(|s_1|, |s_2|)} \tag{2.4}$$

Edit distance handles typos, transpositions, and abbreviated names ("Corp." vs "Corporation").

A ***composite similarity score*** combines multiple attribute similarities with weights reflecting their discriminative power:

$$\sigma_{\text{composite}}(r_1, r_2) = w_{\text{name}} \cdot \sigma(r_1.\text{name},\ r_2.\text{name}) + w_{\text{addr}} \cdot \sigma(r_1.\text{addr},\ r_2.\text{addr}) + w_{\text{dob}} \cdot \sigma(r_1.\text{dob}, \tag{2.5}$$

where $w_{\text{name}} + w_{\text{addr}} + w_{\text{dob}} = 1$.

A pair (r_1, r_2) is declared a match if $\sigma_{\text{composite}}(r_1, r_2) \geq \theta$ for a threshold θ chosen to balance precision and recall for the specific application.

2.5 Data Quality: Dimensions and Challenges

Data integration can perfectly resolve structural, semantic, syntactic, and granularity heterogeneity, correctly link every entity, and still produce a dataset that is analytically worthless—if the underlying data is of poor quality. The ***Veracity*** dimension of the 5Vs framework (Chapter 1) captures this

concern. Data quality problems are not exceptions; they are ubiquitous in production systems, and big data engineering requires systematic processes for detecting, quantifying, and remediating them.

The field has converged on six dimensions of data quality, originally formalised by Wang and Strong (1996) in their empirical study of information consumers' quality requirements.

Table 2.2: The six dimensions of data quality with engineering implications

Dimension	Definition	Detection and remediation
Accuracy	Values correctly represent the real-world entity or event	Statistical profiling; external reference datasets; domain range constraints
Completeness	All required values are present; no critical nulls	NULL ratio checks; conditional completeness rules (`if type="corporate" then tax_id must be non-null`)
Consistency	Values are not contradictory within or across sources	Cross-source join validation; referential integrity checks; temporal monotonicity checks
Timeliness	Data is current enough for its intended use	Staleness metrics; data freshness SLAs; arrival-time audits
Validity	Values conform to defined formats, ranges, and vocabularies	Regular expression validation; enumeration checks; numeric range constraints
Uniqueness	Records are not duplicated (within a source or across sources)	Duplicate detection; entity resolution (Section 2.4)

2.5.1 Missing Values

Missing values are among the most common data quality problems in large datasets. They arise from several distinct mechanisms, each with different analytical implications.

Missing Completely at Random (MCAR): the probability that a value is missing is independent of both the observed and missing values. A sensor that fails with a fixed, context-independent probability produces MCAR missingness. Data imputation strategies—such as mean or median imputation—are statistically valid under MCAR.

Missing at Random (MAR): the probability that a value is missing depends on observed values but not on the missing value itself. For example, older customers may be less likely to provide an email address, so the `email` field is missing more often for older cohorts. This is MAR because age (observed) predicts missingness, but the missing value itself (specific email) does not.

Missing Not at Random (MNAR): the most problematic case. The probability that a value is missing depends on the value itself. Survey data where respondents with very low incomes decline to answer the income question is MNAR—the missingness is informative about the missing value. Imputing MNAR data with mean imputation introduces systematic bias.

In big data engineering, the immediate engineering response to missing values is typically a combination of: (1) quantifying the NULL rate per column in the data quality dashboard, (2) identifying whether the missing values are upstream (the source system never had the value) or midstream (the ETL pipeline dropped them), and (3) applying domain-appropriate imputation, filtering, or flagging. Feeding a machine learning model records with missing feature values without appropriate handling is one of the most common sources of production ML failure.

2.5.2 Noise, Outliers, and Inconsistency

Noise refers to random errors in data values—measurement errors, transmission errors, or OCR mistakes. A temperature sensor that occasionally reads –999.0°C due to a firmware bug introduces noise; a mobile payment amount that loses a decimal point and appears as 10,000 times the correct value is noise. At scale, even a 0.01% noise rate in a 10-billion-row dataset

produces one million corrupted records—enough to systematically distort any aggregate computed over that column.

Outliers may be noise, or they may be genuine rare events. A single transaction of $50 million in a retail payment network is an outlier; it might represent fraud (noise), or it might represent a legitimate large commercial payment. In data engineering, the distinction between noise and genuine outliers must be resolved using domain knowledge and external validation, not purely statistical criteria.

Inconsistency arises when the same real-world fact is represented differently at different points in time or in different parts of the same system. A customer record that shows the customer's home city as "New York" in the CRM but "NYC" in the billing system is inconsistent. A product price that changes in the catalogue but is not updated in the recommendation engine creates a temporal inconsistency. Inconsistency is particularly dangerous in slowly changing dimension tables (a concept explored in the data warehousing literature) where historical accuracy must be maintained alongside current values.

2.6 Data Provenance and Lineage

Data provenance refers to the documented record of a data artifact's origin, transformation history, and movement through a system—the complete answer to the question: "Where did this value come from, and what was done to it?" ***Data lineage*** is the operationalised form of provenance: the graph of dependencies linking each output dataset to its input datasets and the transformations applied to produce it.

Provenance is not merely a technical nicety; it is increasingly a legal requirement. The European Union's General Data Protection Regulation (GDPR, 2018) requires organisations to be able to demonstrate the legal basis for processing personal data, trace where personal data flows within and across systems, and respond to subject access requests that require identifying all locations where an individual's data is held. The United States' Health Insurance Portability and Accountability Act (HIPAA) requires healthcare organisations to maintain audit trails documenting every access to

and movement of protected health information (PHI). Failure to demonstrate adequate provenance tracking has resulted in regulatory fines exceeding $10 million in individual GDPR enforcement actions.

Beyond regulatory compliance, provenance serves crucial operational purposes. When an analytical model produces an unexpected result, data lineage allows engineers to trace the result back to the exact source records and transformations that produced it—identifying whether the anomaly originates in a source system, an ETL transformation, or the analytical model itself. In production data pipelines, this capability can reduce the mean time to detect and resolve data quality incidents from days to hours.

Key Insight | Lineage as a First-Class Engineering Artifact

Modern data engineering treats lineage as a first-class artifact, not an afterthought. Tools such as Apache Atlas (metadata management), OpenLineage (open specification for lineage events), and dbt's built-in lineage graph capture transformations automatically as pipelines run. A comprehensive lineage graph answers: which tables feed this model, which source records produced this output row, which transformation version was active when this job ran, and who approved the pipeline change that introduced this transformation. Without this capability, debugging a corrupted KPI dashboard may require days of manual source tracing across dozens of systems.

2.7 ETL vs. ELT: Two Philosophies of Integration

The practical execution of data integration—moving, transforming, and loading data from source systems into analytical targets—is governed by two contrasting paradigms. The choice between them is one of the most consequential architectural decisions in data engineering, because it determines where and when data is transformed, how flexible the system is to evolving requirements, and what trade-offs are made between data freshness, quality, and engineering complexity.

2.7.1 ETL: Extract, Transform, Load

ETL is the traditional paradigm, dominant in data warehousing from the 1980s through the 2010s. It operates in three sequential phases:

Extract: Data is read from source systems—typically RDBMS tables, file exports, or API responses—and written to a staging area. The extraction is designed to be minimally invasive, typically using incremental extraction (change data capture, CDC) rather than full table scans, to avoid overloading source systems.

Transform: In the staging area, the extracted data is subjected to all required cleaning, mapping, enrichment, and aggregation operations before it is written to the target. The target (typically a data warehouse) enforces a strict schema; data that does not conform to the schema is rejected. This means that the transformation step must be comprehensive and correct before any data reaches the warehouse.

Load: The transformed, validated data is loaded into the target data warehouse in bulk. Only data that passes all quality and schema checks is written to the target.

The primary advantage of ETL is data quality at the target: by the time data reaches the warehouse, it has been fully validated and conforms precisely to the intended schema. The primary disadvantage is rigidity: adding a new source or changing a transformation requires modifying the ETL pipeline, which may require schema changes in the warehouse, regression testing of all downstream queries, and potentially a full historical reload.

2.7.2 ELT: Extract, Load, Transform

ELT inverts the order: raw data is extracted from sources and loaded immediately into a data lake (typically cloud object storage such as Amazon S3, Google Cloud Storage, or Azure Data Lake Storage) in its native format, without transformation. Transformation is deferred until analytical consumption time, when a compute engine such as Apache Spark or a transformation tool such as dbt applies the necessary logic on-demand.

ELT has become the dominant paradigm for modern data engineering for several reasons. First, cloud object storage is extraordinarily inexpensive (approximately $0.023 per GB per month for S3 Standard as of 2024), making

it economically feasible to store raw, untransformed data at any scale. Second, decoupling extraction from transformation enables *schema evolution*: when a source system changes its schema, the raw data in the lake is unaffected, and only the transformation logic needs to be updated. Third, deferred transformation supports multiple downstream use cases with different transformation requirements without requiring the ETL pipeline to anticipate every future analytical need.

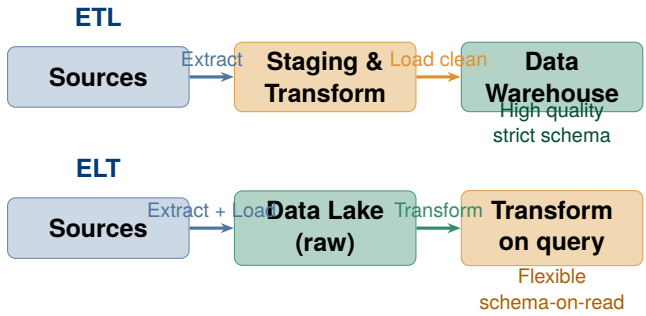

Figure 2.2: ETL vs. ELT: the order of operations determines where and when data quality constraints are enforced.

Common Misconception | ELT Does Not Mean Skip Transformation

A common misunderstanding is that ELT eliminates the need for data transformation. It does not—it defers transformation. The raw data in a data lake is typically not directly queryable or usable for analytics; it requires the same cleaning, normalisation, and enrichment that ETL performs, just applied at query time rather than ingestion time. The *dbt* (data build tool) framework has become the standard for managing this deferred transformation layer, implementing transformation logic as versioned, tested SQL models with lineage tracking. ELT without a robust transformation and testing layer produces a "data swamp"— a lake of raw data that nobody trusts or can efficiently query.

2.8 The Modern Data Integration Stack

Modern data integration pipelines combine tools that address different stages and dimensions of the integration problem. Understanding the ecosystem—which tool solves which problem—is essential for designing production systems.

A typical production ELT stack for a modern data-driven organisation operates as follows. Source systems (RDBMS, SaaS applications, IoT devices) emit data either via batch exports or real-time event streams. Batch sources are ingested using Sqoop (for RDBMS-to-HDFS transfer) or a managed connector platform (Airbyte, Fivetran) that handles incremental extraction and schema change detection. Real-time sources are ingested via Apache Kafka, which acts as a durable, ordered event bus with configurable retention—allowing consumers to replay the full event history at any time. Kafka is examined in depth in Chapter **??**.

Once raw data lands in the data lake (HDFS or cloud object storage), transformation is managed by dbt (for SQL-based transformations in a data warehouse engine) or Apache Spark (for large-scale Python/Scala transformations over raw files). The transformation layer produces ***data marts***—curated, domain-specific views of the integrated data designed for specific analytical workloads. A metadata and lineage platform such as Apache Atlas automatically captures the transformation history, maintaining the provenance graph required for regulatory compliance.

Schema evolution—the challenge of handling changes in source schemas without breaking downstream consumers—is managed by a ***schema registry***. Confluent Schema Registry (for Kafka-based pipelines) maintains versioned schemas for each topic, allowing producers and consumers to negotiate schema compatibility at runtime. When a source system adds a new field, the registry enforces *backward compatibility*: consumers using the old schema can still read new records by treating the new field as null; producers using the new schema can still write records that old consumers can parse.

2.9 Case Study: US Healthcare Interoperability and HL7 FHIR

Case Study | Healthcare Interoperability and HL7 FHIR — Integration at National Scale

Background. The United States healthcare system is one of the world's most data-rich and most data-fragmented. A typical US hospital uses between 20 and 50 different software systems: an electronic health record (EHR) platform, a laboratory information system, a radiology picture-archiving system, a pharmacy management system, a billing and insurance system, a clinical decision support system, and dozens of others. These systems were built by different vendors, at different times, using different data models, and have historically been unable to exchange patient data without significant manual effort.

The integration cost. A 2019 analysis by the Center for Health Information and Decision Systems (University of Maryland) estimated that integration failures cost US hospitals **$8.3 billion annually** in duplicated tests, delayed care, medication errors attributable to incomplete records, and manual reconciliation labour. A patient transferred between two hospitals using different EHR vendors may have their medication list, allergy history, and recent lab results effectively unavailable to the receiving physician—an information gap that directly affects clinical decisions.

The technical challenge as a heterogeneity problem. Healthcare data integration exhibits all four dimensions of heterogeneity simultaneously. *Structural*: patient identifiers vary by hospital (medical record number, patient account number, national identifier). *Semantic*: the same diagnosis code may mean different things under different coding systems (ICD-9 vs. ICD-10 vs. SNOMED CT). *Syntactic*: dates, dosages, and units of measurement are encoded differently across vendors. *Granularity*: laboratory reference ranges vary by testing equipment and laboratory protocol.

The HL7 FHIR standard. Health Level 7 (HL7) is the standards organisation that developed the Fast Healthcare Interoperability Resources (FHIR) standard to address these challenges. FHIR defines a RESTful API and a set of standardised data resources (Patient, Observation,

Medication, DiagnosticReport, etc.) that any EHR system can implement. By exposing patient data through a FHIR API, a hospital allows any authorised consumer—a mobile app, a referral system, a population health analytics platform—to query standardised patient records without knowing the internal database schema of the EHR.
FHIR resources are represented in JSON or XML. A standardised patient resource looks like:

```
{
  "resourceType": "Patient",
  "id":           "pat-001",
  "name": [{"family": "Smith", "given": ["John"]}],
  "birthDate":    "1978-04-22",
  "gender":       "male",
  "identifier": [
    {"system": "urn:oid:2.16.840.1.113883.4.1",
     "value": "123-45-6789"}
  ],
  "address": [
    {"line": ["123 Main Street"],
     "city": "Boston", "state": "MA",
     "postalCode": "02101"}
  ]
}
```

Listing 2.1: A partial HL7 FHIR Patient resource (JSON)

Integration architecture. A national health information exchange built on FHIR operates as follows. Each participating hospital implements a FHIR server that exposes its EHR data through the standard API. A health information exchange (HIE) platform federates queries across multiple FHIR servers, performing entity resolution (matching the same patient across hospitals using demographic attributes) and aggregating standardised data into a unified longitudinal patient record. The transformation from each hospital's internal schema to FHIR is handled by a vendor-specific FHIR adapter—essentially an ETL pipeline that runs inside or alongside the EHR system.

Data quality and entity resolution challenges. Even with FHIR standardisation, entity resolution across hospitals remains the hardest tech-

nical challenge. The US has no universal patient identifier (comparable to a national insurance number in many other countries); matching patients across hospitals relies on probabilistic matching on name, date of birth, address, and partial social security number—exactly the similarity-function approach described in Section 2.4. Typical production matching systems achieve 95–98% recall at 98–99% precision, meaning that 1–5% of true patient matches are missed and 1–2% of declared matches are incorrect—a significant error rate in clinical contexts.

Lessons for data engineering. The healthcare interoperability challenge illustrates three principles that apply across all large-scale data integration projects. First, technical standards alone (FHIR) are necessary but not sufficient; data quality, entity resolution, and governance must be addressed alongside standardisation. Second, the cost of integration failure is not abstract—it manifests in clinical outcomes, regulatory penalties, and operational waste. Third, integrating data from autonomous, heterogeneous systems at scale requires exactly the distributed tools—Kafka, Spark, schema registries, and lineage tracking—that constitute modern data engineering practice.

Chapter Summary

- **The data silo problem** is the primary barrier to enterprise analytics: data spread across independent, autonomous systems with incompatible schemas, identifiers, and formats cannot be jointly queried without integration.
- **Data integration** provides a unified query interface over heterogeneous, autonomous sources via three core operations: schema mapping (defining field correspondences), data exchange (materialising transformed data), and entity resolution (linking records representing the same real-world entity).
- **Four dimensions of heterogeneity** must all be addressed: structural (different schemas), semantic (different meanings), syntactic (different encodings), and scale/granularity (different units and resolutions). Struc-

tural and syntactic failures are loud; semantic and granularity failures are silent and dangerous.

- **Entity resolution** identifies matching records across sources with no common key. Blocking reduces the $O(n^2)$ comparison problem; similarity functions (Jaccard, edit distance) score candidate pairs; a composite threshold determines matches.
- **Data quality** has six dimensions: accuracy, completeness, consistency, timeliness, validity, and uniqueness. Missing values may be MCAR, MAR, or MNAR; the appropriate imputation strategy depends on the missingness mechanism.
- **Data provenance** records the origin and transformation history of all data artifacts. It is required by GDPR and HIPAA and is operationally critical for debugging data pipelines. OpenLineage, Apache Atlas, and dbt provide lineage tracking.
- **ETL** transforms data before loading (high quality at the target, rigid schema). **ELT** loads raw data first and transforms on demand (flexible, supports schema evolution, requires a separate transformation layer such as dbt).
- The **modern integration stack** combines Kafka (streaming ingestion), Sqoop/Airbyte (batch ingestion), Spark/dbt (transformation), and Atlas (lineage). Schema registries manage schema evolution across producers and consumers.

Exercises

Short Questions

SQ2.1. Define data integration. What three operations does it require, and what does each operation accomplish?

SQ2.2. Explain the difference between structural and semantic heterogeneity. Give one example of each from the domain of retail e-commerce.

SQ2.3. A source system stores dates as the string `"15/03/2024"` and a target

system expects dates as `DATE` values in ISO 8601 format (`"2024-03-15"`). Which dimension of heterogeneity does this represent? What operation resolves it?

SQ2.4. What is entity resolution? Why is the naive $O(n^2)$ approach infeasible at big data scale, and how does blocking address this?

SQ2.5. Define the six dimensions of data quality and give one concrete example for each from a financial services context.

SQ2.6. What is the difference between MCAR, MAR, and MNAR missingness? Why does the distinction matter when choosing an imputation strategy?

SQ2.7. What is data provenance? Name two regulatory frameworks that require it and explain what each framework demands.

SQ2.8. Compare ETL and ELT: (a) in which order are the transform and load steps applied, (b) where is the data quality enforcement point, and (c) which is better suited for schema evolution?

SQ2.9. True or false, with one-sentence justification: "The ELT paradigm eliminates the need for data transformation."

SQ2.10. What is a schema registry? What problem does it solve in a Kafka-based integration pipeline?

SQ2.11. A retail company finds that 15% of rows in its integrated customer table have `NULL` in the `email` column. List three possible reasons for this missingness and explain how you would distinguish between them.

SQ2.12. What is "schema drift"? Give a concrete example of how schema drift can cause a downstream machine learning model to fail silently.

SQ2.13. What is the GAV (global-as-view) approach to schema mapping? How does it differ from LAV (local-as-view), and what is the practical trade-off between them?

SQ2.14. A data engineer says: "Our ETL pipeline runs nightly; by morning we have yesterday's data." Identify the data quality dimension being compromised and explain what architecture change would address it.

Analytical Problems

AP2.1. Entity Resolution Complexity Analysis. A hospital system wants to resolve patient identities across two EHR databases: Hospital A has 2.8 million patient records; Hospital B has 1.4 million patient records. A blocking strategy based on birth-year and first three characters of the last name produces blocks of average size 350 pairs (350 A-records times the B-records in the same block).

(a) Calculate the number of candidate pairs under naive $O(n^2)$ comparison (all A against all B). At a rate of 5 million comparisons per second, how long would naive comparison take?

(b) Estimate the total number of candidate pairs after blocking, assuming each blocking key creates blocks of average size 350. (Hint: how many blocks are there if each A-record falls in exactly one block?)

(c) What is the speedup factor from blocking?

(d) A composite similarity score requires 12 floating-point operations per pair comparison. Estimate the total computation cost (in FLOPs) for the post-blocking comparisons, and determine whether it is feasible on a single server with 100 GFLOPS of sustained throughput.

AP2.2. ETL vs. ELT Storage and Latency Trade-off. A financial services company ingests 5 TB of raw trading data per day from 40 source systems. In the current ETL architecture, data is transformed and compressed before loading, resulting in a 4:1 compression ratio at the warehouse. The company is evaluating migrating to an ELT architecture with raw data stored in S3 at \$0.023/GB/month, and transformation via Spark on demand.

(a) Calculate the current warehouse storage cost after 12 months (assume warehouse storage at \$0.20/GB/month, and that the compression ratio applies to all stored data). Compare with the raw data storage cost in S3 over the same period.

(b) The ETL pipeline requires 4 hours of processing before data is available for queries. The ELT approach makes raw data available immediately but requires 45 minutes of Spark transformation before a specific analytical view is ready. For a use case that needs data refreshed every 6 hours, which approach provides better effective freshness?

(c) In the ELT system, 12 months of raw data are retained in S3. A new analytical requirement arrives, requiring a transformation that was not anticipated. Estimate the Spark compute cost to retroactively apply this transformation to 12 months of 5 TB/day data at $0.10 per Spark CPU-hour (assume 10 TB/hour processing throughput per 10 CPUs).

AP2.3. Data Quality Profiling. An e-commerce company's data engineering team profiles a 50-million-row product catalogue table and finds the following:

- 12% of rows have `NULL` in `product_weight`.
- 3.2% of rows have `price` = 0.
- 0.8% of rows have `price` > $10,000 (the 99.9th percentile is $950).
- 4% of rows appear to be duplicates (same product, same vendor, inserted on different dates).
- 7% of rows have `category_id` values that do not exist in the `categories` reference table.

For each issue: (a) identify the data quality dimension it violates, (b) classify it as noise, missing data, inconsistency, or validity violation, and (c) propose a specific remediation and the operational procedure to apply it without breaking downstream pipelines.

AP2.4. Similarity Scoring for Entity Resolution. Two records from different source systems are candidates for entity resolution. Using the composite similarity formula with weights $w_{\text{name}} = 0.5$, $w_{\text{dob}} = 0.3$, $w_{\text{city}} = 0.2$:

Attribute	Source A	Source B
Name	`Johnson Michael R.`	`Michael R Johnson`
Date of birth	`1985-07-14`	`07/14/1985`
City	`Los Angeles`	`LA`

(a) Parse both date representations and confirm they are the same date ($\sigma_{\text{dob}} = 1.0$). (b) Compute the Jaccard similarity on name tokens (split on space and punctuation); let $\sigma_{\text{name}} = J(T_A, T_B)$. (c) Assign $\sigma_{\text{city}} = 0.4$ ("Los Angeles" and "LA" are the same city but share no token; domain knowledge is required). (d) Compute $\sigma_{\text{composite}}$ and determine whether these records are a match at thresholds $\theta = 0.7$ and $\theta = 0.8$. (e) Discuss whether the

threshold choice here is an engineering or a business decision.

Design Problems

DP2.1. Integration Architecture for a Global Logistics Company. A global freight logistics company operates in 60 countries. Each country office uses a locally deployed ERP system (Oracle in North America, SAP in Europe, a custom-built system in Southeast Asia). The company needs a unified analytics platform that answers questions such as: "Which shipment routes had the highest delay rates last quarter?" and "What is the total revenue per customer across all regions?"

The three ERP systems use different customer identifiers, different currency representations (some systems store amounts in local currency only; others in USD), different status code vocabularies for shipment states, and different date formats. Real-time track-and-trace data (GPS position of shipments) arrives from 8,000 vehicles at 30-second intervals.

Design the integration architecture. Your design must specify:

(a) The integration paradigm (ETL or ELT) and justification.

(b) How each of the four heterogeneity dimensions is addressed.

(c) The entity resolution strategy for customers (who may appear in multiple regional ERP systems with different identifiers).

(d) How the real-time GPS stream is ingested and at what latency it becomes available for analytics.

(e) How data lineage and provenance are maintained across the pipeline.

(f) Which specific tools you would use at each stage and why.

DP2.2. Healthcare Data Quality Framework. A national health analytics agency receives patient-level data files from 200 hospitals every month. The files arrive as CSV exports from each hospital's EHR system. The agency uses this data to compute national statistics on chronic disease prevalence, hospital readmission rates, and treatment outcome metrics.

A preliminary audit reveals: (1) 18 different date formats across the 200 hospitals; (2) ICD-10 codes mixed with ICD-9 codes in the same diagnosis field; (3) patient ages ranging from –3 to 247 (clearly erroneous); (4) 22% of

records missing at least one of: gender, ethnicity, or primary diagnosis; and (5) duplicate patient records at rates between 0.5% and 8% depending on the hospital.

Design a data quality pipeline that the agency can apply to incoming files before they are used in national statistics. Your design must:

(a) Specify a quality gate for each of the five issues identified, including the rule, the action for records that fail (reject, flag, impute), and the threshold at which an entire hospital's submission is quarantined.

(b) Describe how MCAR vs. MAR vs. MNAR missingness would be diagnosed for the 22% missing records, and what imputation strategy is appropriate for each case.

(c) Explain how you would maintain provenance to satisfy HIPAA audit requirements while working with de-identified patient data.

(d) Propose a data quality SLA (e.g., "we require >98% completeness on primary diagnosis before publishing national statistics") and explain how it would be monitored in production.

Programming Exercises

PE2.1. Entity Resolution with Jaccard Similarity (Python). Implement the Jaccard similarity function for string tokenisation and apply it to resolve company names across two source datasets.

```
import re

source_a = [
    {"id": "A001", "name": "Google LLC",       "city": "
        Mountain View"},
    {"id": "A002", "name": "Apple Inc.",        "city": "
        Cupertino"},
    {"id": "A003", "name": "Microsoft Corp",    "city": "
        Redmond"},
    {"id": "A004", "name": "Meta Platforms Inc","city": "Menlo
        Park"},
    {"id": "A005", "name": "Amazon.com Inc",    "city": "
        Seattle"},
]
```

```
10
11 source_b = [
12     {"id": "B001", "name": "GOOGLE LLC",          "city": "
           Mountain View"},
13     {"id": "B002", "name": "Apple Incorporated",  "city": "
           Cupertino"},
14     {"id": "B003", "name": "Microsoft Corporation","city": "
           Redmond"},
15     {"id": "B004", "name": "Facebook (Meta)",      "city": "
           Menlo Park"},
16     {"id": "B005", "name": "Amazon Web Services",  "city": "
           Seattle"},
17 ]
18
19 def tokenise(text):
20     """Lowercase, remove punctuation, split into tokens."""
21     text = re.sub(r"[.,\-()']", " ", text.lower())
22     return set(text.split())
23
24 def jaccard(tokens_a, tokens_b):
25     """
26     Return Jaccard similarity between two token sets.
27     J(A, B) = |A intersection B| / |A union B|
28     Return 0.0 if the union is empty.
29     """
30     # TODO: implement
31     pass
32
33 def blocking_key(record):
34     """
35     Return a blocking key: first token of name + first 3 chars
           of city.
36     Records only compared within the same blocking key.
37     """
38     first_token = tokenise(record["name"])
39     city_prefix = record["city"][:3].lower()
40     # TODO: return key string
41     pass
42
43 def resolve_entities(source_a, source_b, threshold=0.5):
44     """
45     Return list of (a_id, b_id, score) tuples where score >=
           threshold.
46     Use blocking to reduce comparisons.
```

```
    """
    matches = []
    # TODO:
    # 1. Build index of source_b records by blocking key
    # 2. For each record in source_a, compute its blocking key
    # 3. Compare only against source_b records with the same key
    # 4. Compute Jaccard on name tokens; append match if >= threshold
    return matches

matches = resolve_entities(source_a, source_b, threshold=0.4)
print("Resolved entity pairs:")
for a_id, b_id, score in matches:
    a_rec = next(r for r in source_a if r["id"] == a_id)
    b_rec = next(r for r in source_b if r["id"] == b_id)
    print(f"  {a_id} ({a_rec['name']}) <-> {b_id} ({b_rec['name']})  "
          f"score={score:.3f}")
```

Listing 2.2: PE2.1: Company name entity resolution

PE2.2. Data Quality Audit (Python). Implement a data quality profiler that checks a list of records against defined quality rules and produces a summary report.

```
import datetime

records = [
    {"id": "R001", "name": "Alice Chen",
     "dob": "1990-05-14", "email": "alice@example.com", "amount": 1250.00},
    {"id": "R002", "name": "Bob Smith",
     "dob": "1985-13-01",  # invalid month
     "email": None, "amount": 0.0},
    {"id": "R003", "name": "",            # empty name
     "dob": "2035-01-01",  # future date
     "email": "carol@example.com", "amount": -50.0},
    {"id": "R004", "name": "Diana Kumar",
     "dob": "1978-03-22", "email": "diana@example.com", "amount": 980.50},
    {"id": "R005", "name": "Eve Johnson",
     "dob": "1978-03-22",  # duplicate dob+name (same as R004 roughly)
```

```
     "email": None, "amount": 99999.0},
]

def check_completeness(record, required_fields):
    """Return list of field names that are None or empty string."""
    missing = []
    for f in required_fields:
        v = record.get(f)
        if v is None or v == "":
            missing.append(f)
    return missing

def check_dob_validity(dob_str):
    """
    Return True if dob_str is a valid date in ISO format (YYYY-MM-DD)
    and not in the future.
    """
    try:
        dob = datetime.date.fromisoformat(dob_str)
        return dob <= datetime.date.today()
    except (ValueError, TypeError):
        return False

def check_amount_validity(amount):
    """Return True if amount is a positive number."""
    try:
        return float(amount) > 0
    except (ValueError, TypeError):
        return False

def profile_records(records):
    """
    Run all quality checks and print a summary report.
    Dimensions checked: Completeness, Validity, Accuracy (range).
    """
    required = ["name", "dob", "email", "amount"]
    issues = []

    for rec in records:
        rec_issues = {"id": rec["id"], "problems": []}

```

```
        # Completeness
        missing = check_completeness(rec, required)
        if missing:
            rec_issues["problems"].append(
                f"Missing fields: {missing}")

        # Validity: dob
        if rec.get("dob") and not check_dob_validity(rec["dob"
            ]):
            rec_issues["problems"].append(
                f"Invalid or future dob: {rec['dob']}")

        # Validity: amount
        if rec.get("amount") is not None:
            if not check_amount_validity(rec["amount"]):
                rec_issues["problems"].append(
                    f"Non-positive amount: {rec['amount']}")

        if rec_issues["problems"]:
            issues.append(rec_issues)

    # Summary
    total = len(records)
    clean = total - len(issues)
    print(f"Quality Report: {clean}/{total} records passed all
        checks\n")
    for issue in issues:
        print(f"  Record {issue['id']}:")
        for p in issue["problems"]:
            print(f"    - {p}")
    return issues

profile_records(records)
```

Listing 2.3: PE2.2: Data quality profiler

PE2.3. ETL Pipeline Simulation (Python). Simulate a three-stage ETL pipeline that extracts records from two heterogeneous sources, applies transformations to resolve structural and syntactic heterogeneity, and loads unified records into a target schema.

```
import datetime

# Source A: CRM system (European date format, price in EUR)
```

```
source_a = [
    {"contact_id": "C001", "fname": "Emma",  "lname": "Weber",
     "birth": "22.09.1988", "eur_value": 1400.0},
    {"contact_id": "C002", "fname": "Luca",  "lname": "Rossi",
     "birth": "05.03.1975", "eur_value": 2200.0},
]

# Source B: ERP system (US date format, price in USD, full name
    column)
source_b = [
    {"customer_no": "E045", "full_name": "Chen, Wei",
     "date_of_birth": "07/18/1992", "usd_value": 3100.0},
    {"customer_no": "E046", "full_name": "Park, Ji-Young",
     "date_of_birth": "12/04/1983", "usd_value": 870.0},
]

EUR_TO_USD = 1.09  # approximate exchange rate

# Target schema:
# { "id", "first_name", "last_name", "dob" (ISO), "value_usd" }

def transform_source_a(record):
    """
    Map CRM record to target schema.
    - contact_id -> id
    - fname/lname -> first_name/last_name
    - birth (DD.MM.YYYY) -> dob (YYYY-MM-DD)
    - eur_value * EUR_TO_USD -> value_usd
    """
    # TODO: implement transformation
    pass

def transform_source_b(record):
    """
    Map ERP record to target schema.
    - customer_no -> id
    - full_name ("Last, First") -> first_name, last_name
    - date_of_birth (MM/DD/YYYY) -> dob (YYYY-MM-DD)
    - usd_value -> value_usd (already USD)
    """
    # TODO: implement transformation
    pass

def run_etl(source_a, source_b):
```

```
    """Extract, transform both sources, and load into unified
        target."""
    target = []

    # Transform source A
    for rec in source_a:
        transformed = transform_source_a(rec)
        if transformed:
            target.append(transformed)

    # Transform source B
    for rec in source_b:
        transformed = transform_source_b(rec)
        if transformed:
            target.append(transformed)

    return target

unified = run_etl(source_a, source_b)
print("Unified target records:")
for r in unified:
    print(f"  {r}")
```

Listing 2.4: PE2.3: ETL pipeline simulation

Table 2.3: ETL vs. ELT: a systematic comparison

Attribute	**ETL**	**ELT**
Transformation timing	Before loading (staging area)	After loading (at query time)
Target schema	Predefined, strict (schema-on-write)	Flexible (schema-on-read)
Data quality at target	High (validated before load)	Variable (raw data preserved)
Historical replay	Expensive (re-run ETL jobs)	Easy (re-transform from raw)
Schema evolution	Difficult (pipeline rebuild)	Easy (update transformation only)
Raw data preservation	No (transformed data only)	Yes (raw always available)
Storage cost	Lower (transformed, compressed)	Higher (raw data at full volume)
Latency	Higher (transform-then-load)	Lower (load immediately)
Tooling	Informatica, Talend, SSIS	Spark, dbt, Airbyte, Fivetran
Best suited for	Regulated industries, strict SLAs	Data lakes, ML feature stores, exploratory analytics

Table 2.4: The modern data integration tool ecosystem

Tool	Direction	Pattern	Primary use case
Apache Sqoop	RDBMS ↔ HDFS	Batch	Bulk migration of relational tables into Hadoop
Apache Flume	Log sources → HDFS	Batch/micro-batch	Web server log aggregation
Apache Kafka	Any → Any	Stream	Real-time event bus; decoupled source-to-sink
Airbyte / Fivetran	SaaS APIs → Data lake	Batch/incremental	No-code connector platform for 200+ sources
Apache Spark	Data lake → Any	Batch/stream	Large-scale transformation and enrichment
dbt	Data lake/DW internal	Batch	SQL-based transformation, testing, lineage
Apache Atlas	Any	Metadata	Data governance, lineage graph, cataloguing

Part II

Distributed Storage

3

Distributed File Systems: GFS and HDFS

"We have designed the system for a usage pattern in which most files are mutated by appending new data rather than overwriting existing data. Random writes within a file are practically non-existent."

Ghemawat, Gobioff & Leung, SOSP 2003

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why conventional file systems (NFS, POSIX, SAN) cannot meet the storage requirements of petabyte-scale data engineering, and identify the specific assumptions they violate.
2. Describe the GFS architecture—master, chunkservers, clients, chunk size, and metadata design—and explain the key design decisions that made it suitable for Google's workload.
3. Draw and explain the HDFS architecture: NameNode, DataNodes, Secondary NameNode, and their interactions.
4. Trace the complete HDFS read and write paths step by step, including pipeline replication, checksum verification, and lease management.
5. Explain rack-aware replication: the default three-replica placement

policy and why it balances fault tolerance against write overhead.
6. Describe the NameNode high-availability (HA) architecture with Active/Standby nodes, ZooKeeper leader election, and shared Journal Nodes.
7. Identify HDFS's design limitations—the small file problem, no random writes, low-latency access—and contrast HDFS with cloud object stores (Amazon S3, Google Cloud Storage).

Chapter 2 established how data moves between systems and how heterogeneous sources are unified. Before any of that integration logic can execute, however, the raw data must live somewhere. In 2003, Google faced a storage problem that no existing file system could solve: billions of web pages, totalling petabytes of data, needed to be stored reliably across thousands of commodity servers, indexed sequentially at enormous throughput, and kept available even as individual servers failed—which, at that scale, happened multiple times per day.

The solution Google published was the ***Google File System*** (GFS) (Ghemawat, Gobioff, and Leung 2003). Three years later, the Apache Hadoop project reimplemented and open-sourced the same design as the ***Hadoop Distributed File System*** (HDFS). HDFS became the foundational storage layer for an entire ecosystem of distributed data processing tools— MapReduce, Hive, Pig, HBase, and eventually Spark—and it remains in active production use at organisations ranging from Yahoo! and Facebook to LinkedIn and the New York Times. Understanding HDFS is understanding the bedrock on which the rest of this book's processing stack rests.

3.1 The Storage Problem at Petabyte Scale

A ***file system*** is the software layer responsible for organising, naming, accessing, and protecting data stored on physical storage devices. The file system abstracts away the physical device—magnetic platters, solid-state cells, or optical discs—and presents the programmer with a uniform interface of named files in a hierarchical directory structure. For four decades, this

model served the computing industry well. It continues to serve it well for most workloads today.

The model breaks at petabyte scale for three fundamental reasons.

Single-machine capacity. A conventional file system is managed by a single host. Even the largest shared file systems— Network File System (NFS) and SAN-attached volumes—ultimately depend on a finite number of storage controllers. By 2003, Google's web crawl was producing hundreds of terabytes per month, a rate that had already surpassed what any single storage system could absorb. The only solution was to distribute data across a cluster of independent machines.

Throughput requirements. Web indexing requires reading an entire dataset sequentially to reprocess it—there is no practical way to index a subset of the web. A single high-performance SCSI disk in 2003 delivered approximately 50 MB/s of sequential read throughput. Reading one petabyte from a single disk would take over 230 days. Distributing the data across 1,000 disks and reading them in parallel reduces this to approximately five hours—a feasible engineering timeline.

Hardware failure as normal operation. A conventional file system—and the RDBMS systems that sit on top of it—treats hardware failure as an exceptional condition to be recovered from. At Google's scale in 2003, a cluster of a few thousand commodity PCs experienced approximately three node failures per day as normal operation: disk bearing failures, memory errors, motherboard failures, and network card failures, all occurring on a predictable statistical schedule (Ghemawat, Gobioff, and Leung 2003). Any storage system that treated such failures as exceptional events would be unavailable for recovery most of the time. The failure model must instead treat hardware failure as *the expected case* and design accordingly.

3.2 The Google File System

The Google File System, published by Ghemawat, Gobioff, and Leung at the ACM Symposium on Operating Systems Principles (SOSP) in October 2003, is one of the most consequential systems papers of the past two decades (Ghemawat, Gobioff, and Leung 2003). It describes a purpose-built

Table 3.1: Comparison of storage architectures for big data workloads

System	Max capacity	Fault model	Reason it fails at big data scale
POSIX local FS	Single disk / array	Single point of failure	No horizontal scaling; one server limit
NFS	NAS head + array	NAS head is SPOF	Centralized metadata and I/O bottleneck
SAN	Storage array	Expensive RAID	Proprietary hardware; prohibitive cost/PB
RDBMS blob	Server RAM + disk	WAL recovery	Tuned for random access; not sequential scan
GFS / HDFS	**Cluster-wide**	**Expects failure**	**Designed for this workload**

distributed file system designed not to satisfy general-purpose file system requirements—POSIX compliance, low-latency random access, small file support—but to satisfy the specific requirements of Google's production workloads.

3.2.1 Workload Assumptions and Design Decisions

GFS was designed around a careful empirical study of Google's actual workloads. The authors observed that virtually all files in Google's production environment were large—multiple gigabytes to hundreds of gigabytes—and that data access patterns were dominated by sequential reads and sequential appends, with random writes essentially absent. Files were written once, read many times, and never modified in place after initial creation. Concurrent reads from many clients were the norm; concurrent random writes to the same file were not.

These observations drove six key design decisions that distinguish GFS from conventional file systems:

1. **Large chunk size (64 MB).** Files are divided into fixed-size ***chunks*** of 64 MB. This is orders of magnitude larger than the 4-KB blocks used by

conventional file systems. Large chunks reduce the amount of metadata the master must maintain (one entry per chunk rather than one per block), reduce client-to-master communication (a client needs the location of fewer chunks to read a large file), and encourage direct TCP connections between clients and chunkservers for sustained sequential I/O.

2. **Centralised, in-memory metadata.** The GFS master holds all file system metadata—the file namespace, chunk-to-file mappings, chunk locations, access control lists—entirely in RAM. This enables sub-millisecond metadata lookups. The master has a single-machine capacity for metadata because large chunk sizes keep the number of chunks manageable: at 64 MB per chunk, one petabyte of data requires only ~16 million chunk entries, which fits comfortably in modern server RAM.
3. **No data caching.** GFS explicitly does not cache file data at either clients or chunkservers. The workload is dominated by streaming reads that iterate through files once; cached data would be evicted before it could be reused. The kernel's buffer cache handles any incidental temporal locality.
4. **Write-by-append semantics.** GFS optimises for atomic append operations rather than random writes. A client can append data to any file at the current end-of-file position; this corresponds to how log files, web crawl outputs, and search index segments are naturally produced.
5. **Fault tolerance through replication, not redundant hardware.** Each 64 MB chunk is stored on three chunkservers by default. If one fails, the master re-replicates the affected chunks on a surviving server. No RAID arrays or specialised storage hardware are required.
6. **Relaxed consistency.** GFS provides a consistency model weaker than POSIX. Concurrent writes to the same region from different clients may produce an interleaved, "defined but inconsistent" result. Google's applications were designed to tolerate this—for example, by writing unique chunk identifiers with each record and using deduplication at read time.

3.2.2 GFS Architecture

A GFS cluster consists of three types of components: one ***GFS master***, multiple ***chunkservers***, and multiple ***clients***.

The **GFS master** maintains all file system metadata in RAM. It does not participate in data I/O between clients and chunkservers; once a client has obtained the chunk location from the master, all data transfer occurs directly between the client and the chunkserver. This separation of the control plane (master) from the data plane (chunkservers and clients) is the central architectural decision that enables GFS to scale: the master handles only lightweight metadata operations, while the bulk data flows on separate direct connections.

The master communicates with chunkservers through two periodic mechanisms. Each chunkserver sends a ***heartbeat*** to the master every few seconds to confirm it is operational; the absence of heartbeats for a configurable period causes the master to mark the chunkserver as failed. Each chunkserver also sends a periodic ***chunk report*** listing all the chunk replicas it currently holds, which allows the master to reconstruct its view of chunk locations after a restart.

Chunkservers store chunks as plain files on their local Linux file systems. They do not cache chunks in memory; the kernel's buffer cache provides whatever temporal locality exists. Each chunkserver is responsible for maintaining checksum metadata for every chunk it stores, and it verifies checksums on every read to detect data corruption.

Clients implement the GFS API (which is not POSIX-compatible). A client contacts the master to resolve a file name to a list of chunk handles and their chunkserver locations, then communicates directly with the appropriate chunkservers for data I/O. Clients cache the chunk location information locally for a limited time, reducing master load.

Research Spotlight | Ghemawat, Gobioff & Leung (2003) — The Google File System

Citation: Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google File System. *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 29–43. (Ghemawat, Gobioff, and Leung 2003)

Key insight: The authors' central argument is that *workload-specific design* outperforms general-purpose design at extreme scale. By deliberately abandoning POSIX compliance, random write support, and fine-grained access control, GFS achieved throughput and fault tolerance impossible with any off-the-shelf file system.

Technical contributions:

- Demonstrated that a centralised metadata server (the master) is not a scalability bottleneck when (a) metadata is kept entirely in RAM, (b) chunk sizes are large enough that the total metadata volume is manageable, and (c) data I/O bypasses the master entirely.
- Introduced *atomic record append*—an operation that allows multiple clients to append to the same file concurrently without prior coordination, enabling producer-consumer pipelines over shared log files.
- Quantified failure rates in commodity clusters and provided the engineering argument that replication (software-based redundancy on commodity hardware) is more cost-effective than hardware RAID at petabyte scale.

Impact today: GFS is cited in over 14,000 academic papers— one of the most-cited systems papers ever published. It directly inspired HDFS (the open-source reimplementation), Amazon S3, Microsoft Azure Blob Storage, and Google's own successor system, Colossus. The paper is assigned reading at Stanford CS246, MIT 6.830, and CMU 15-721.

3.3 HDFS Architecture

The Hadoop Distributed File System is the open-source Apache implementation of the GFS design. It was initially developed at Yahoo! by Doug Cutting and Mike Cafarella—who had been building a web crawler called Nutch and needed a production-ready open-source replacement for GFS—and contributed to the Apache Software Foundation in 2006. HDFS translates the GFS architecture into Java, with some naming changes (GFS master → NameNode, chunkserver → DataNode, chunk → block) and some incremental design improvements.

3.3.1 The NameNode

The ***NameNode*** is HDFS's master process. It is the centralised metadata server responsible for maintaining the file system namespace and the block-to-DataNode mapping. Like the GFS master, it holds all metadata entirely in RAM; the typical NameNode memory footprint is approximately 150 bytes per file and 150 bytes per block, meaning that one million files average ~300 MB of heap—manageable on a modern server.

The NameNode's metadata is persisted to disk in two complementary structures. The ***FsImage*** is a complete, point-in-time snapshot of the file system namespace—a serialised checkpoint of all metadata. The ***EditLog*** is a write-ahead log of all namespace mutations since the last checkpoint: file creations, deletions, permission changes, and block allocations. On startup, the NameNode replays the EditLog against the last FsImage to reconstruct the current namespace state. Under normal operation, the EditLog grows continuously; it is periodically merged into the FsImage in a process called ***checkpointing***.

The NameNode does *not* store the physical locations of blocks in persistent metadata. Instead, block locations are rebuilt from scratch at startup through the DataNode ***block report***—each DataNode reports the full list of blocks it currently holds when it connects to the NameNode. This design avoids the risk of stale location data in the FsImage becoming inconsistent with the actual DataNode state after failures or maintenance.

Definition | NameNode Responsibilities

1. **Namespace management:** maintain the directory tree and file-to-block mapping.
2. **Block management:** track which DataNodes hold each block replica; initiate re-replication when a replica is lost.
3. **Access control:** enforce file permissions on all client requests.
4. **Lease management:** grant and revoke write leases that determine which client may currently write to a file.
5. **Load balancing:** coordinate the HDFS Balancer, which moves blocks between DataNodes to equalise disk utilisation.

3.3.2 DataNodes

DataNodes are the worker processes that store actual block data. Each DataNode manages its local storage—one or more hard disks or SSDs—and serves block read and write requests directly from clients. DataNodes are designed to run on commodity hardware; a typical DataNode configuration in 2024 uses 12–24 spinning hard disks of 8–18 TB each, providing 96–432 TB of raw capacity per node.

DataNodes communicate with the NameNode through two periodic messages. A ***heartbeat*** (sent every 3 seconds by default) confirms the DataNode is alive and operational. A ***block report*** (sent every 6 hours by default, or on demand at startup) lists all block replicas the DataNode currently holds. The NameNode uses both to maintain its live view of the cluster's block map. If a heartbeat is absent for more than 10.5 minutes (the default `dfs.heartbeat.interval` × `dfs.namenode.heartbeat.recheck-interval`), the NameNode marks the DataNode as dead and schedules re-replication of all blocks it held.

DataNodes also perform ***block scanning***: each DataNode independently verifies the checksums of all its stored blocks on a configurable schedule (default: every three weeks). A block that fails checksum verification is reported to the NameNode as corrupted, and the NameNode responds by scheduling re-replication from a healthy replica.

3.3.3 The Secondary NameNode

Common Misconception | The Secondary NameNode is Not a Hot Standby

Despite its misleading name, the **Secondary NameNode** is not a standby that takes over if the primary NameNode fails. It performs a single function: it periodically merges the EditLog into the FsImage, producing a new, updated FsImage that it sends back to the primary NameNode. This *checkpointing* service reduces the EditLog size and therefore the NameNode startup time after a crash. If the primary NameNode fails, the Secondary NameNode's checkpoint can be used to recover the namespace, but this requires a manual failover process and results in some minutes of downtime. True hot standby failover requires the High Availability architecture described in Section 3.6.

The checkpointing cycle works as follows. The Secondary NameNode fetches the current FsImage and EditLog from the primary, applies all EditLog transactions to the FsImage in memory, writes the resulting merged FsImage to disk, and sends it back to the primary. The primary then begins writing subsequent namespace mutations to a fresh, empty EditLog. The net result is that the EditLog never grows indefinitely; checkpointing occurs every hour by default, or when the EditLog exceeds one million transactions.

3.3.4 Block Storage

Blocks are the fundamental unit of storage in HDFS. The default block size is 128 MB (increased from the original 64 MB in Hadoop 2.x). When a client writes a file, HDFS divides it into 128 MB blocks and distributes those blocks across DataNodes; each block is stored three times by default (the ***replication factor***).

The large block size—32,768 times larger than a typical ext4 file system block—serves several purposes. It reduces the number of metadata entries the NameNode must maintain. It enables sustained sequential I/O at full disk throughput without per-block overhead. And it reduces the number of round trips between client and NameNode required to locate all blocks of a large file.

A notable edge case: the last block of a file may be smaller than 128 MB. HDFS does not pad it to 128 MB; the block occupies only as much disk space as the actual data. A 1-GB file consists of eight full 128 MB blocks, not nine.

The Complete HDFS Cluster Architecture

Figure 3.1 assembles all three components into a single view of the HDFS cluster architecture. The key point to internalise is the separation of the *control plane* (all metadata operations that involve the NameNode) from the *data plane* (all block transfers that flow directly between clients and DataNodes). The NameNode processes millions of metadata RPC calls per day but never touches a byte of block data; every byte of data is transferred directly between clients and DataNodes on independent TCP connections.

3.4 Replication and Fault Tolerance

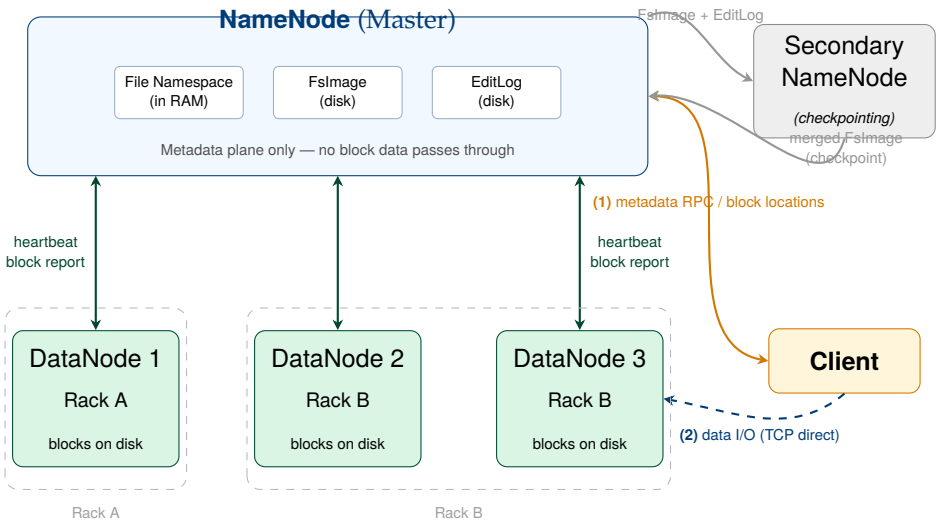

Figure 3.1: Complete HDFS cluster architecture. The NameNode (master) stores all file system metadata in RAM (file namespace, block-to-DataNode map) and persists mutations to the EditLog and FsImage. DataNodes (workers) store blocks on local disks and communicate with the NameNode via periodic heartbeats and block reports. Clients obtain block locations from the NameNode *(step 1 — control plane)*, then read/write block data directly to DataNodes *(step 2 — data plane)*, bypassing the NameNode entirely. The Secondary NameNode checkpoints the EditLog into the FsImage to bound restart time after a NameNode crash.

HDFS achieves fault tolerance through ***block replication***: storing each block on multiple DataNodes such that if any individual DataNode fails, the remaining replicas ensure data availability. The default replication factor of 3 means that the failure of any single DataNode does not result in any data loss or unavailability.

3.4.1 Rack-Aware Replication

Naive replication—placing all three replicas on randomly chosen DataNodes—would provide poor fault tolerance in a real data centre. Physical servers in a data centre are organised in ***racks***: sets of 20–40 servers sharing a top-of-rack switch. A rack-level failure (a switch failure or a power distribution unit trip) takes down every server in the rack simultaneously. If all three replicas

of a block happened to reside on nodes within the same rack, a rack failure would destroy all three replicas.

HDFS addresses this with ***rack-aware replication***. The default policy for replication factor 3 places:

- **Replica 1**: on the same node as the writing client (or a random node if the client is not running on a DataNode).
- **Replica 2**: on a DataNode in a *different* rack from Replica 1.
- **Replica 3**: on a *different* DataNode in the same rack as Replica 2.

This placement balances three competing objectives. Placing Replica 1 locally (or on a nearby node) minimises write latency for the initial block placement. Placing Replica 2 on a different rack ensures that a single rack failure cannot destroy all replicas. Placing Replica 3 in the same rack as Replica 2 (rather than a third rack) reduces cross-rack network traffic: reads can often be satisfied from within the same rack as the client, reducing the load on the inter-rack network bandwidth.

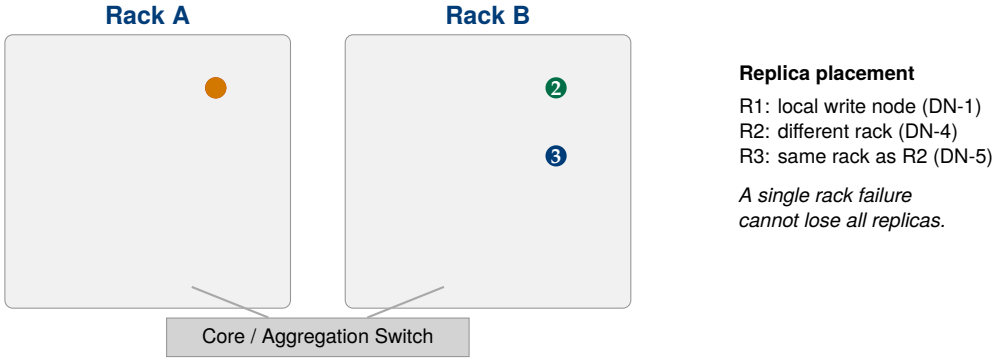

Figure 3.2: HDFS rack-aware replication with replication factor 3. Two replicas in Rack B; one in Rack A. A Rack A failure leaves Replicas 2 and 3 available.

3.4.2 Failure Detection and Re-replication

When the NameNode detects that a DataNode is dead (missed heartbeats for more than the configured threshold), it immediately identifies every block for which that DataNode held a replica. For each such block, the NameNode checks whether the remaining replicas satisfy the replication factor. If a block now has fewer than the required number of replicas, the NameNode

schedules ***re-replication***: it instructs one of the surviving DataNodes that holds a copy to transfer the block to a new DataNode, restoring the full replication factor.

Re-replication is prioritised by urgency. Blocks that have only one surviving replica (single-replica blocks) are re-replicated first, since they are at immediate risk of data loss if the last surviving DataNode also fails. Blocks with two surviving replicas are re-replicated next. The re-replication process consumes cluster network bandwidth; in a large cluster where many DataNodes are replaced simultaneously (e.g., during a scheduled maintenance window), the NameNode throttles re-replication to avoid saturating the network.

3.5 HDFS Read and Write Paths

Understanding how data actually flows in and out of HDFS—not merely the architecture of the components but the sequence of operations—is essential for diagnosing performance problems and for understanding why HDFS makes the consistency trade-offs it does.

3.5.1 The Read Path

Reading a file from HDFS involves the following sequence:

1. **Open.** The client calls `FileSystem.open()`. The HDFS client library contacts the NameNode via RPC and requests the block locations for the first few blocks of the target file.
2. **Block location response.** The NameNode returns an ordered list of DataNode addresses for each requested block, sorted by their network proximity to the client. The list prioritises DataNodes on the same node as the client, then the same rack, then other racks—the data locality principle from Chapter 1.
3. **Direct DataNode connection.** The client connects directly to the closest DataNode for the first block and streams the data. The client reads data from the DataNode through a TCP socket; no data passes through the NameNode.

4. **Checksum verification.** As each 512-byte ***checksum chunk*** is read, the client verifies its CRC-32C checksum against the stored checksum. A mismatch indicates data corruption; the client reports the bad block to the NameNode and retries on an alternative replica.
5. **Block boundary.** When one block is fully read, the client contacts the NameNode for the locations of subsequent blocks (if not already cached) and opens a new connection to the appropriate DataNode.
6. **Close.** When all blocks have been read, the input stream is closed.

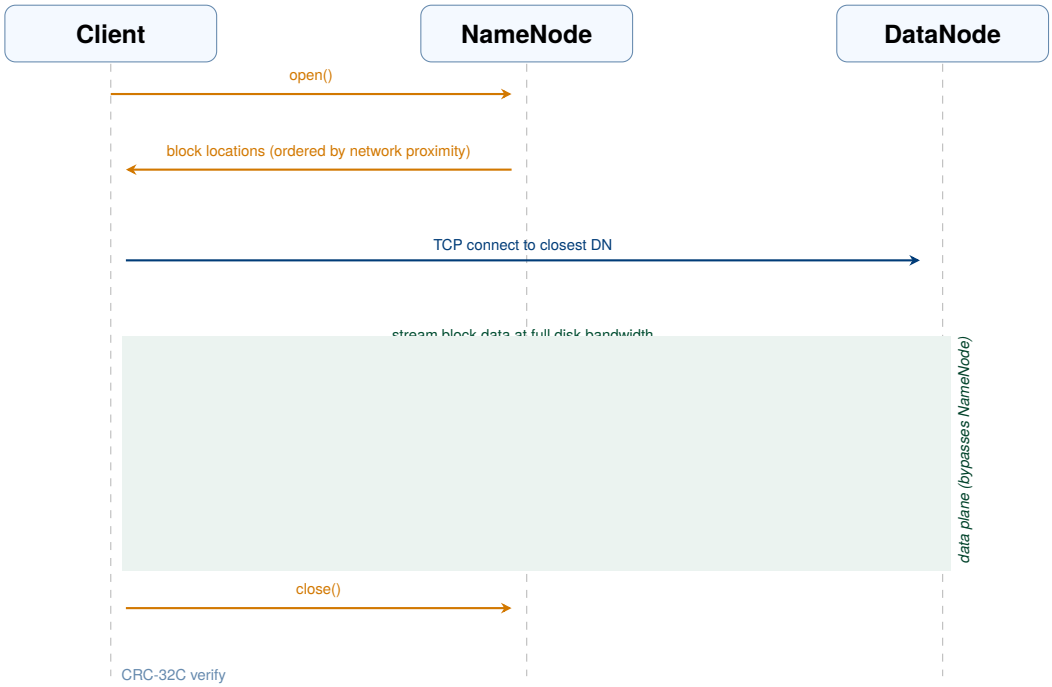

Figure 3.3: HDFS read-path sequence diagram. Steps 1–2 are control-plane operations (RPC to the NameNode); steps 3–6 are data-plane operations (direct TCP to DataNodes). The shaded region highlights that all block data flows between client and DataNode, never through the NameNode. Checksum verification (step 5) runs at the client for every 512-byte chunk; a mismatch triggers a retry on a different replica.

The key performance characteristic of the read path is that data flows directly between client and DataNode, bypassing the NameNode entirely. This means that the NameNode's throughput is not a bottleneck for read operations: it handles only the metadata lookup (typically measured in

microseconds), while data transfer runs at full network speed (typically 1–10 Gbit/s per connection).

3.5.2 The Write Path

Writing to HDFS is more complex than reading, because data must be reliably distributed to multiple DataNodes simultaneously while ensuring consistency in the presence of potential DataNode failures.

1. **Create.** The client calls `FileSystem.create()`. The client library contacts the NameNode to create a new file entry in the namespace. The NameNode verifies that the file does not already exist and that the client has write permission, then creates the file and grants the client a ***write lease***.
2. **Block allocation.** As the client accumulates data in a write buffer (default 64 KB per packet), it requests a new block from the NameNode. The NameNode selects DataNodes for the block—applying the rack-aware placement policy—and returns a list of DataNodes ordered by the pipeline sequence.
3. **Pipeline construction.** The client establishes a TCP connection to the first DataNode in the pipeline; that DataNode connects to the second; the second connects to the third. This forms a ***replication pipeline***: data flows through the pipeline rather than being sent from the client to each DataNode independently, reducing the client's outbound bandwidth requirement.
4. **Packet streaming.** The client sends data in ***packets*** of 64 KB down the pipeline. As the first DataNode receives a packet, it writes it to its local disk and simultaneously forwards it to the second DataNode. The second DataNode writes and forwards to the third. Each DataNode sends an ***acknowledgement*** back up the pipeline once it has durably written the packet.
5. **Block completion.** When all packets for a block have been acknowledged by all three DataNodes, the client notifies the NameNode that the block is complete. The NameNode records the block's DataNode locations in its metadata.
6. **Close.** When the client calls `close()`, all remaining buffered packets are flushed through the pipeline, all blocks are completed, and the

NameNode marks the file as complete, releasing the write lease.

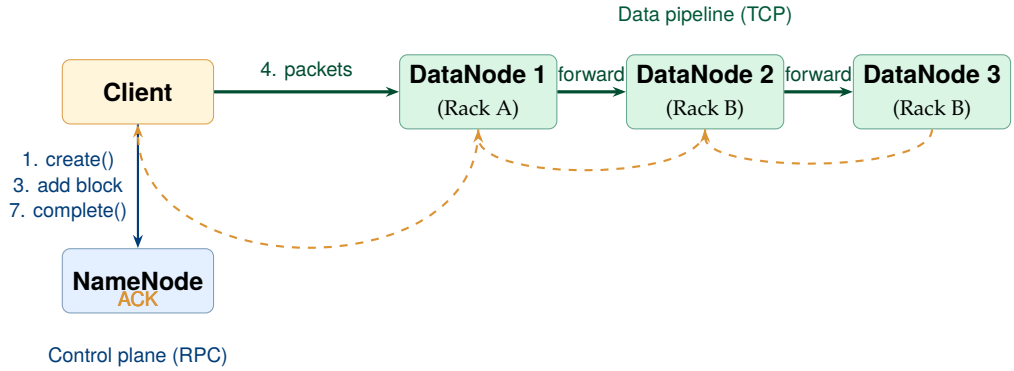

Figure 3.4: HDFS write path. The client communicates with the NameNode only for control messages; all block data flows through the three-DataNode replication pipeline. ACKs propagate back through the pipeline to confirm durable writes.

System Design Note | Pipeline Failure Handling

If a DataNode fails mid-pipeline during a write, HDFS recovers as follows. The pipeline is closed. Packets that were acknowledged by the failed DataNode but not yet confirmed by the client are marked for re-send. A new block replica is allocated on a healthy DataNode, and a new pipeline is constructed excluding the failed DataNode. The NameNode is notified of the incomplete replica on the failed node and schedules re-replication after the write completes. The client experiences a brief pause but the write succeeds as long as at least one DataNode survives. This mechanism allows HDFS to absorb DataNode failures *during* a write without requiring the client to restart the entire write operation.

3.6 HDFS High Availability

In the original HDFS design, the NameNode is a ***single point of failure*** (SPOF). If the NameNode process crashes or the NameNode machine becomes unavailable, the entire HDFS cluster is inaccessible: no client can

resolve file names to block locations, and no DataNode can be instructed to re-replicate data. At the scale of Yahoo!'s production Hadoop cluster—42,000 nodes in 2011—an unplanned NameNode outage meant that 42,000 DataNodes became inaccessible simultaneously. The resulting loss of analytical capacity during the recovery period (which could take 30–60 minutes for a large namespace) was a significant operational risk.

Hadoop 2.0 (released 2012) introduced ***HDFS High Availability*** (HA), which eliminates the NameNode SPOF by maintaining two NameNode instances simultaneously: one ***Active NameNode*** that handles all client and DataNode traffic, and one ***Standby NameNode*** that continuously mirrors the Active's metadata state and can take over within seconds if the Active fails.

3.6.1 The HA Architecture

The HA architecture requires three key components beyond the standard HDFS setup.

Journal Nodes are a quorum of lightweight processes (a minimum of three, to tolerate one failure) that implement a shared, fault-tolerant edit log. The Active NameNode writes every namespace mutation to the Journal Node quorum as it executes; the Standby NameNode reads from the Journal Node quorum continuously, replaying edits to keep its in-memory namespace state synchronised with the Active. Because the Journal Node quorum is shared, the Standby's namespace state is always at most a few hundred milliseconds behind the Active.

ZooKeeper provides distributed leader election. Each NameNode runs a ***ZooKeeper Failover Controller*** (ZKFC) process that monitors the NameNode's health and maintains a session with ZooKeeper. If the Active NameNode becomes unhealthy—as detected by the ZKFC— the Active's ZooKeeper session expires, and ZooKeeper triggers a leader election. The Standby's ZKFC wins the election and promotes the Standby to Active.

Fencing prevents the ***split-brain*** scenario—where both NameNodes believe themselves to be Active and independently accept writes, resulting in divergent metadata states. Before promoting the Standby, the ZKFC executes a fencing operation that either gracefully stops the former Active (via SSH) or, if the former Active is unreachable, forcibly terminates its ability

to communicate with DataNodes using SSH-based remote kill commands or STONITH ("Shoot The Other Node In The Head") hardware mechanisms.

Table 3.2: NameNode HA component summary

Component	Role
Active NameNode	Handles all client and DataNode traffic; writes edits to JN quorum
Standby NameNode	Continuously replays JN edits; ready to take over within seconds
Journal Nodes (3+)	Shared, quorum-based edit log; tolerates minority failures
ZooKeeper (3+)	Leader election and health monitoring for the two NameNodes
ZKFC	Per-NameNode daemon that manages ZooKeeper session and triggers failover

3.7 HDFS Design Characteristics and Limitations

HDFS is a purpose-built system optimised for specific workloads. Like GFS, its strengths and weaknesses are two sides of the same coin: the design decisions that make it excellent for batch processing of large files make it unsuitable for other access patterns.

3.7.1 What HDFS Is Optimised For

HDFS excels at **high-throughput, sequential access to large files**. A well-tuned HDFS cluster can deliver aggregate read throughput of several GB/s per node: with 100 DataNodes each delivering 200 MB/s from local disk, the cluster sustains 20 GB/s of aggregate read bandwidth. For workloads that need to scan entire datasets—MapReduce jobs, Hive queries that process all records in a table, Spark jobs that re-partition a large dataset—this throughput is the critical metric, and HDFS is optimised for it.

HDFS also provides **fault-tolerant write durability**. A write that returns a success acknowledgement to the client is guaranteed to have been committed to at least r DataNodes (where r is the replication factor). Hardware failure of any subset of up to $r - 1$ DataNodes simultaneously cannot cause data loss for any successfully written block.

3.7.2 The Small File Problem

HDFS's single most significant operational limitation is its poor handling of large numbers of small files. Each file, regardless of size, consumes approximately 150 bytes of NameNode heap space for its metadata. A single large file split into ten 128 MB blocks consumes approximately 1.65 KB of NameNode memory. Ten million small files— each taking one block— consume 1.5 GB of NameNode heap, regardless of the files' total data volume. At 100 million small files, the NameNode requires 15 GB of heap and metadata operations begin to slow noticeably.

This is a critical practical constraint. Web server log files, sensor telemetry archives, and social media ingestion pipelines naturally produce large numbers of small files if not carefully managed. The standard engineering response is to use ***SequenceFiles***— container files that aggregate many small records into a single large HDFS file, providing a sequential index for record-level access— or to pre-aggregate small files before ingestion using tools such as HDFS `archive` (HAR files) or Apache CombineFileInputFormat in the MapReduce layer.

3.7.3 No In-Place Random Writes

HDFS does not support random writes to existing files. Once a block has been written and acknowledged, its content cannot be modified in place. HDFS supports only sequential writes to new files and atomic appends to the end of existing files (the append semantics introduced in HDFS 0.21 and Hadoop 2.x). This is a deliberate architectural choice that greatly simplifies the consistency model and the replication protocol: if blocks are immutable after writing, the system never needs to coordinate writes across multiple replicas.

For workloads that require in-place updates—a transactional database, an incremental index—HDFS is the wrong tool. Apache HBase (examined

in Chapter 4) provides random read/write access on top of HDFS by using an LSM-tree (Log-Structured Merge-tree) structure that converts random writes into sequential HDFS writes.

3.7.4 No Low-Latency Random Access

HDFS is tuned for throughput, not latency. Opening an HDFS file requires a round trip to the NameNode; reading a specific block requires a TCP connection setup to the appropriate DataNode. Total latency for a random read is typically in the range of 50–200 ms— acceptable for batch processing but wholly unacceptable for interactive queries (which require sub-millisecond storage access) or for OLTP workloads. This is why HDFS is used as the storage substrate for batch and near-batch systems (MapReduce, Spark, Hive), not as a replacement for relational databases or key-value stores.

Table 3.3: HDFS strengths, limitations, and engineering mitigations

Characteristic	Impact	Mitigation / alternative
Optimised for sequential reads	Excellent batch throughput	Use for MapReduce, Spark, Hive
Large block size (128 MB)	Poor for small files	SequenceFiles, HAR archives, CombineFileInputFormat
No random writes	Cannot update in place	HBase (LSM-tree over HDFS)
High read latency (50–200 ms)	No interactive queries	Presto, Spark SQL with in-memory caching
NameNode SPOF (pre-HA)	Cluster-wide outage	HDFS HA with ZooKeeper
Namespace scalability	100M file ceiling	HDFS Federation (multiple NameNodes)

3.8 Beyond HDFS: Cloud Object Storage

Starting around 2012 and accelerating through 2015–2018, the data engineering industry began a significant migration away from on-premises HDFS clusters toward cloud-based ***object storage***: Amazon S3, Google Cloud Storage (GCS), and Azure Data Lake Storage (ADLS). By 2024, cloud object storage has largely supplanted HDFS as the primary storage layer for new data engineering deployments, though HDFS remains in use at organisations that operate their own data centres.

Cloud object storage differs from HDFS in several important ways. Objects in S3 or GCS are identified by a key (essentially a path string) and stored as a flat namespace with no directory hierarchy (though object key prefixes mimic directory structure). Unlike HDFS blocks, objects can be up to 5 TB each. Object storage is practically unlimited in capacity and scales automatically with usage—there is no NameNode to run out of heap, no replication factor to configure manually, and no DataNodes to provision and maintain.

From a pricing and operations standpoint, cloud object storage is dramatically simpler. Amazon S3 charges approximately $0.023/GB/month for standard storage; there are no servers to operate, no replication factor to configure, and no NameNode HA to maintain. A data engineering team using S3 eliminates the entire operational burden of the HDFS cluster in exchange for a predictable per-GB-per-month cost.

The principal trade-off is **decoupling storage from compute**. In a Hadoop cluster, DataNodes run both storage and compute (MapReduce or Spark tasks), exploiting data locality. Cloud object storage is physically separate from the compute cluster; every read requires a network hop. However, cloud data centre networks now deliver bandwidth of 25–100 Gbit/s between storage and compute instances, which is often higher than the inter-DataNode bandwidth in on-premises Hadoop clusters. Modern frameworks such as Apache Spark have been optimised to run effectively against object storage, and the loss of data locality has proven to be a minor performance penalty in most workloads compared to the operational simplicity of S3.

Key Insight | The Shift to Storage-Compute Separation

The move from HDFS to cloud object storage represents a fundamental architectural shift: from *tightly coupled* storage-compute (where data lives next to the CPU that processes it) to *loosely coupled* (where storage and compute scale independently). In a Hadoop cluster, you cannot add compute capacity without also adding storage, and vice versa. With S3 + cloud compute, you can scale a Spark cluster from 10 to 1,000 nodes and back to 10 without moving a single byte of data. This elasticity has proven to be the dominant engineering advantage, and it is the primary reason cloud object storage has displaced on-premises HDFS for new deployments.

3.9 Case Study: Yahoo!'s Hadoop Cluster (2006–2011)

Case Study | Yahoo! — The First Industrial-Scale Open-Source Distributed File System

Background. Yahoo! was the first organisation outside Google to operate a production Hadoop cluster at true industrial scale. The company had several workloads that required petabyte-scale batch processing: web ranking (recomputing the search index nightly from the full crawl), user behaviour analytics (processing billions of page views per day), and spam detection (scanning email metadata across hundreds of millions of user accounts). After evaluating alternatives, Yahoo! bet its entire search and analytics infrastructure on the open-source Hadoop ecosystem starting in 2006.

Scale and growth. Yahoo!'s Hadoop cluster grew from approximately 1,000 nodes in 2006 to a peak of **42,000 nodes** in 2011—a 42-fold scale-up in five years. At its peak, the cluster stored approximately **100 petabytes** of data in HDFS across two major data centres. The web index computation alone ran tens of thousands of MapReduce jobs per day. Yahoo!'s infrastructure team reported that the cluster experienced approximately **2–3 DataNode failures per day** on average—exactly the failure rate predicted by the GFS paper for commodity hardware at this scale.

Operational challenges. Managing a 42,000-node HDFS cluster required solving engineering problems that the original Hadoop design had not anticipated.

The *NameNode bottleneck* emerged as the cluster grew. By 2009, Yahoo!'s HDFS namespace contained over 50 billion blocks, consuming the full 64 GB of available NameNode heap. Startup time after a planned NameNode maintenance window exceeded 90 minutes—during which the entire cluster was inaccessible. Yahoo!'s engineering team developed several mitigations, including a distributed NameNode extension (*HDFS Federation*) that partitioned the namespace across multiple NameNodes, reducing the per-NameNode block count and memory pressure.

The *small file problem* was acute in production. Yahoo!'s log ingestion pipelines initially wrote one file per server per minute, producing hundreds of millions of small files per day. The engineering team developed automated compaction pipelines that aggregated hourly log files into daily SequenceFiles, reducing the namespace size by a factor of roughly 1,000 while preserving query access.

Contributions to open source. Yahoo! was by far the largest contributor to the Apache Hadoop project during this period. Key features that originated from Yahoo!'s production experience include: HDFS Append (enabling streaming log ingestion), Hadoop Security (Kerberos-based authentication, developed after Yahoo! experienced data governance incidents with an unsecured cluster), HDFS High Availability (eliminating the NameNode SPOF after a NameNode failure caused a 45-minute cluster-wide outage in 2010), and YARN (the resource manager that replaced the original MapReduce-only scheduler, enabling non-MapReduce workloads such as Spark and Tez to share the cluster).

Lessons for data engineering. Yahoo!'s experience establishes several principles that remain relevant today:

- HDFS is exceptionally robust for large-file, sequential-access workloads at any scale—but it requires careful operational management, particularly of the NameNode's memory and the accumulation of small files.
- Production distributed systems require active contributions back to

their upstream open-source projects: Yahoo! could not have operated at 42,000 nodes without the HDFS improvements it developed and contributed. This model of "upstream dependency, active contribution" has been adopted by Facebook, LinkedIn, Twitter (X), and most major technology companies.

- Failure planning is not optional. Yahoo!'s decision to invest in HDFS HA before a major outage occurred—rather than after—is the textbook example of proactive operational engineering.

Chapter Summary

- Conventional file systems (NFS, SAN, POSIX) fail at petabyte scale for three reasons: single-machine capacity limits, insufficient sequential throughput, and an inability to tolerate the continuous hardware failures that occur at cluster scale.
- **GFS** (2003) solved these problems by: using large 64 MB chunks, maintaining all metadata in a single master's RAM, separating control-plane (master) from data-plane (chunkservers), supporting append-only semantics, and relying on replication rather than RAID for fault tolerance.
- **HDFS** is the open-source GFS implementation. Its key components are the **NameNode** (namespace and block mapping in RAM, EditLog + FsImage persistence) and **DataNodes** (block storage, heartbeat, block report, checksum verification).
- **Rack-aware replication** (factor 3, default) places one replica locally, one on a different rack, one on a second node in the same rack as replica 2. A single rack failure cannot cause data loss.
- The **write pipeline** flows client → DN1 → DN2 → DN3 with acknowledgements propagating in reverse. The NameNode handles only control messages; data bypasses it entirely.
- **HDFS HA** uses Active/Standby NameNodes, shared Journal Nodes for the edit log, ZooKeeper for leader election, and fencing to prevent split-brain.
- HDFS limitations include the **small file problem** (NameNode heap

exhaustion), **no random writes**, and **high read latency**. Mitigations include SequenceFiles, HBase, and Presto.

- **Cloud object storage** (S3, GCS, ADLS) has largely replaced HDFS for new deployments by offering unlimited capacity, no operational burden, and elastic compute-storage separation—at the cost of data locality.

Exercises

Short Questions

SQ3.1. State three reasons why a conventional NFS-based file system cannot store and process a 500-petabyte dataset.

SQ3.2. What is a GFS chunk? What is the default size and why was this size chosen instead of the 4-KB blocks used by typical file systems?

SQ3.3. Explain the GFS design decision to keep all metadata in the master's RAM. What is the practical ceiling on the number of chunks this imposes, and at what dataset size is that ceiling reached for 64-MB chunks?

SQ3.4. What is the HDFS NameNode? List its five responsibilities and explain what would happen to a running HDFS cluster if the NameNode process were killed.

SQ3.5. What is the difference between the HDFS Secondary NameNode and a hot-standby NameNode? What problem does the Secondary NameNode actually solve?

SQ3.6. Describe the rack-aware replication policy for replication factor 3. If a rack fails, how many replicas of each block remain? Does this satisfy the fault-tolerance goal?

SQ3.7. Trace the HDFS write path step by step from `FileSystem.create()` to the final acknowledgement. At which step is the data actually sent to DataNodes, and how many network hops does it take before the client receives a write acknowledgement?

SQ3.8. What is a DataNode heartbeat and what is a block report? How

does the NameNode use each to manage the cluster?

SQ3.9. Explain the HDFS small file problem. If a 100-node cluster holds 200 million files, each occupying one 128- MB block, estimate the minimum NameNode heap required (assume 300 bytes per file–block pair).

SQ3.10. Why does HDFS not support random writes? What are the consistency advantages of the immutable-block design?

SQ3.11. What is HDFS High Availability? List the three components beyond the standard HDFS setup that HA requires and explain the role of each.

SQ3.12. What is a split-brain condition in an HA system? Why is fencing necessary, and what are two mechanisms by which HDFS HA can fence the former Active NameNode?

SQ3.13. Compare HDFS and Amazon S3 on four dimensions: (a) data locality, (b) operational burden, (c) capacity ceiling, (d) cost model.

SQ3.14. A data engineering team ingests one million web server log files per day, each 200 KB in size, directly into HDFS. After one year, the NameNode heap is full. Identify the root cause and propose two engineering solutions.

Analytical Problems

AP3.1. Replication and Storage Overhead Analysis. A genomics research institute stores 50 petabytes of raw sequencing data in HDFS with the default replication factor of 3 and a block size of 128 MB.

(a) Calculate the total HDFS storage capacity required (including replicas) in petabytes.

(b) Calculate the number of 128- MB blocks required to store 50 PB of data.

(c) If each DataNode has 200 TB of usable disk capacity, how many DataNodes are required to store the replicated dataset?

(d) Estimate the NameNode heap required to hold metadata for all blocks (assume 150 bytes per block). Is this within the 128 GB of RAM on a modern server?

(e) If the institute reduces the replication factor to 2, what is the storage saving in petabytes? What is the trade-off?

AP3.2. HDFS Write Throughput Analysis. A streaming ingestion pipeline writes data to HDFS at a sustained rate of 2 GB/s. Each DataNode has a 10 Gbit/s network interface (effective throughput 1.25 GB/s). The replication factor is 3.

(a) In the write pipeline (client → DN1 → DN2 → DN3), how much network bandwidth does each hop consume at 2 GB/s ingestion rate?

(b) Is a single DataNode's network interface the bottleneck? If the pipeline is the only network traffic, at what ingestion rate would the first DataNode's network interface saturate?

(c) How many simultaneous parallel write pipelines (each ingesting a different block) can a 10 Gbit/s interface support at 2 GB/s per pipeline? Why is it beneficial to parallelise writes?

(d) At 2 GB/s ingestion rate and 128 MB blocks, how many blocks per second does the system create? At what rate does the NameNode receive `addBlock` RPC calls?

AP3.3. NameNode HA Failover Timing. An HDFS HA cluster has the following configuration:

- ZooKeeper session timeout: 10 seconds
- ZKFC health check interval: 5 seconds
- Standby NameNode edit log replay lag: 200 milliseconds maximum
- SSH fencing timeout: 30 seconds

(a) What is the minimum time from Active NameNode failure to the Standby becoming Active (assuming fencing via SSH succeeds)?

(b) What is the maximum time in the worst case (ZooKeeper session timeout has just reset, fencing takes the full 30 seconds)?

(c) During the failover window, what happens to HDFS clients that are actively reading? What happens to clients that are in the middle of a write?

(d) Explain why fencing must be completed before the Standby is promoted. Describe the consequences of a successful promotion without fencing, given that the former Active's write lease is still valid.

AP3.4. Cloud vs. On-Premises Storage Economics. A media company

stores 2 PB of video content in on-premises HDFS with replication factor 3 (requiring 6 PB raw disk capacity). The data grows at 500 TB/year. DataNode servers cost $8,000 each with 72 TB usable disk per server; server lifetime is 4 years (amortised $2,000/server/year). Operations staff costs $150,000/year to manage the cluster. Amazon S3 charges $0.023/GB/month for standard storage.

(a) How many DataNode servers are needed today? What is the annualised hardware cost (amortised over 4 years)?

(b) What is the total annual on-premises cost (hardware + operations)?

(c) What is the annual S3 cost for 2 PB of raw data (no replication factor needed—S3 manages redundancy internally)?

(d) After 5 years of 500 TB/year growth, compare the cumulative 5-year costs of both approaches (assume S3 pricing remains constant; on-premises requires a hardware refresh at year 4).

(e) What non-monetary factors should also influence this decision?

Design Problems

DP3.1. HDFS Cluster Design for a Video Streaming Platform. A video streaming company archives 8 PB of source video files for transcoding and long-term retention. The files range from 500 MB to 40 GB each (typical feature film at broadcast quality). The company also processes 2 TB/day of viewer analytics events (one JSON record per play event, approximately 512 bytes each, producing ~4 billion records per day). The system must sustain 10 GB/s sustained write throughput during peak ingestion.

Design the HDFS cluster and ingestion pipeline. Your design must specify:

(a) The appropriate replication factor for video files vs. analytics events, and the justification for each choice.

(b) The HDFS block size configuration for video files and for analytics events (can differ per dataset).

(c) The DataNode hardware specification (disk capacity, network interface speed) and the number of DataNodes required.

(d) How the 4-billion-record-per-day analytics stream should be ingested without creating the small file problem (propose a specific ingestion architecture using tools from this or previous chapters).

(e) The HA configuration for the NameNode, including the minimum number of Journal Nodes and ZooKeeper nodes required for quorum.

(f) The monitoring metrics you would track in production and the alerting thresholds that would trigger an on-call escalation.

DP3.2. HDFS Migration to Cloud Object Storage. A financial services company operates a 500-node on-premises HDFS cluster storing 1.2 PB of trade data, customer records, and regulatory filings. The company is evaluating a migration to Amazon S3. The data has strict regulatory requirements: all data must be encrypted at rest and in transit, all access must be auditable (for SEC and FINRA compliance), and data must be retained for 7 years. Certain datasets must remain on-premises (customer PII, subject to contractual data residency obligations).

Design the migration strategy. Your design must address:

(a) A data classification scheme that determines which datasets can migrate to S3 and which must remain on-premises.

(b) The migration mechanism: how to transfer 1.2 PB to S3 with minimal disruption to running production pipelines.

(c) How encryption at rest and in transit is enforced for S3-stored data.

(d) How access auditing ("who accessed which object, when") is implemented in S3.

(e) The hybrid architecture that allows on-premises Spark jobs to read data from both the remaining HDFS cluster and S3 transparently.

(f) The rollback plan if the S3 migration reveals unacceptable latency or data access pattern incompatibilities.

Programming Exercises

PE3.1. HDFS Block Layout Simulator (Python). Simulate the block layout for a set of files in HDFS: compute how many blocks each file occupies, how

much space is wasted in the last block, and the total NameNode metadata overhead.

```
BLOCK_SIZE_MB = 128
BYTES_PER_MB = 1024 * 1024
METADATA_BYTES_PER_BLOCK = 150  # NameNode heap per block

files = [
    {"name": "web_crawl_2024_01.seq",       "size_gb": 312.0},
    {"name": "user_events_2024_01_01.json",  "size_mb": 0.8},
    {"name": "product_images.tar",           "size_gb": 1450.0},
    {"name": "transaction_log_hourly.orc",   "size_gb": 48.5},
    {"name": "genome_sample_001.fastq",      "size_gb": 220.0},
]

def analyse_file(name, size_bytes, block_size=BLOCK_SIZE_MB * BYTES_PER_MB,
                 replication=3):
    """
    Return a dict with:
      num_blocks       : number of HDFS blocks (ceil division)
      last_block_bytes : size of the final (partial) block
      wasted_bytes     : padding to fill the last block to block_size
                         (HDFS does NOT pad, so wasted_bytes = 0 always;
                          but report the theoretical waste if it did)
      metadata_bytes   : NameNode heap for this file's blocks
      replicated_bytes : total physical disk used (data * replication)
    """
    import math
    num_blocks = math.ceil(size_bytes / block_size)
    last_block_bytes = size_bytes % block_size or block_size
    # HDFS does not pad the last block
    wasted_bytes = 0
    metadata_bytes = num_blocks * METADATA_BYTES_PER_BLOCK
    replicated_bytes = size_bytes * replication
    return {
        "name": name,
        "size_bytes": size_bytes,
        "num_blocks": num_blocks,
```

```
        "last_block_bytes": last_block_bytes,
        "wasted_bytes": wasted_bytes,
        "metadata_bytes": metadata_bytes,
        "replicated_bytes": replicated_bytes,
    }

def summarise(files_spec):
    results = []
    for f in files_spec:
        if "size_gb" in f:
            size = int(f["size_gb"] * 1024 * BYTES_PER_MB)
        else:
            size = int(f["size_mb"] * BYTES_PER_MB)
        results.append(analyse_file(f["name"], size))

    total_blocks = sum(r["num_blocks"] for r in results)
    total_nn_mb  = sum(r["metadata_bytes"] for r in results) /
        BYTES_PER_MB
    total_rep_tb = sum(r["replicated_bytes"] for r in results)
        / (1024**4)

    print(f"{'File':<38} {'Blocks':>7} {'Last block':>12} {'NN
        heap':>10}")
    print("-" * 72)
    for r in results:
        lb_mb = r["last_block_bytes"] / BYTES_PER_MB
        nn_kb = r["metadata_bytes"] / 1024
        print(f"{r['name']:<38} {r['num_blocks']:>7,} "
              f"{lb_mb:>10.1f} MB {nn_kb:>8.1f} KB")
    print("-" * 72)
    print(f"{'TOTAL':<38} {total_blocks:>7,} "
          f"{'':>12} {total_nn_mb:>8.1f} MB")
    print(f"\nTotal replicated storage (3x): {total_rep_tb:.2f}
         TB")

summarise(files)
```

Listing 3.1: PE3.1: HDFS block layout simulator

PE3.2. Rack-Aware Replica Placement Simulator (Python). Simulate the HDFS rack-aware replica placement policy for a set of blocks being written to a cluster of DataNodes distributed across racks.

```
import random
```

```

# Cluster topology: rack -> list of DataNode IDs
cluster = {
    "rack-A": ["dn01", "dn02", "dn03", "dn04", "dn05"],
    "rack-B": ["dn06", "dn07", "dn08", "dn09", "dn10"],
    "rack-C": ["dn11", "dn12", "dn13", "dn14", "dn15"],
}

def get_rack(dn_id, cluster):
    for rack, nodes in cluster.items():
        if dn_id in nodes:
            return rack
    return None

def place_replicas(writer_dn, cluster, replication=3):
    """
    Apply HDFS rack-aware placement for replication factor 3:
      R1: same node as writer (or writer's rack if writer not
          in cluster)
      R2: a node on a DIFFERENT rack from R1
      R3: a DIFFERENT node on the SAME rack as R2

    Returns list of (dn_id, rack) tuples, one per replica.
    """
    writer_rack = get_rack(writer_dn, cluster)
    if writer_rack is None:
        # Writer is not a DataNode; pick any node for R1
        writer_rack = random.choice(list(cluster.keys()))
        r1 = random.choice(cluster[writer_rack])
    else:
        r1 = writer_dn

    # R2: different rack
    other_racks = [r for r in cluster if r != writer_rack]
    rack2 = random.choice(other_racks)
    r2 = random.choice(cluster[rack2])

    # R3: different node on same rack as R2
    rack2_others = [n for n in cluster[rack2] if n != r2]
    r3 = random.choice(rack2_others)

    return [(r1, writer_rack), (r2, rack2), (r3, rack2)]

def simulate_writes(num_blocks, writer_dn, cluster):
```

```
    placements = []
    for i in range(num_blocks):
        replicas = place_replicas(writer_dn, cluster)
        placements.append(replicas)

    # Count blocks per DataNode
    from collections import Counter
    node_counts = Counter()
    for rep_list in placements:
        for dn, rack in rep_list:
            node_counts[dn] += 1

    print(f"Block placement for {num_blocks} blocks "
          f"(writer: {writer_dn}, rack: {get_rack(writer_dn,
              cluster)})\n")
    print(f"{'Block':>6}  R1              R2              R3")
    print("-" * 55)
    for i, rep in enumerate(placements[:10]):
        cols = [f"{dn} ({rack})" for dn, rack in rep]
        print(f"{i+1:>6}  {cols[0]:<16}{cols[1]:<16}{cols[2]}")
    if num_blocks > 10:
        print(f"  ... ({num_blocks - 10} more blocks)")

    print(f"\nDataNode block distribution (top 5):")
    for dn, count in node_counts.most_common(5):
        rack = get_rack(dn, cluster)
        print(f"  {dn} ({rack}): {count} blocks")

simulate_writes(num_blocks=50, writer_dn="dn03", cluster=
    cluster)
```

Listing 3.2: PE3.2: Rack-aware placement simulator

PE3.3. Failure Simulation and Re-replication Scheduler (Python). Simulate the NameNode's re-replication logic when DataNodes fail: identify under-replicated blocks and schedule re-replication from surviving replicas.

```
# Initial block map: block_id -> list of DataNode IDs holding a
    replica
block_map = {
    "blk_001": ["dn01", "dn04", "dn07"],
    "blk_002": ["dn02", "dn05", "dn09"],
    "blk_003": ["dn01", "dn06", "dn11"],
    "blk_004": ["dn03", "dn07", "dn12"],
```

```
    "blk_005": ["dn01", "dn02", "dn13"],
    "blk_006": ["dn04", "dn08", "dn14"],
    "blk_007": ["dn02", "dn06", "dn10"],
}

REPLICATION_FACTOR = 3
HEALTHY_NODES = {"dn01", "dn02", "dn03", "dn04",
                 "dn05", "dn06", "dn07", "dn08",
                 "dn09", "dn10", "dn11", "dn12", "dn13", "dn14"
                     }

def simulate_failure(failed_nodes, block_map, healthy_nodes,
                     replication=REPLICATION_FACTOR):
    """
    Remove failed_nodes from block_map and healthy_nodes.
    Return a list of re-replication tasks:
      (block_id, source_dn, priority)
    where priority = 'CRITICAL' (1 surviving replica),
                     'HIGH'     (2 surviving replicas),
                     'NORMAL'   (0 deficit, should not appear)
    Tasks are sorted highest priority first.
    """
    live_nodes = healthy_nodes - set(failed_nodes)
    tasks = []

    for blk_id, replicas in block_map.items():
        surviving = [r for r in replicas if r in live_nodes]
        deficit = replication - len(surviving)
        if deficit <= 0:
            continue  # no re-replication needed
        priority = "CRITICAL" if len(surviving) == 1 else "HIGH
            "
        # Source: first surviving replica
        source = surviving[0] if surviving else None
        tasks.append((blk_id, source, priority, deficit,
            surviving))

    # Sort: CRITICAL first, then HIGH
    tasks.sort(key=lambda t: 0 if t[2] == "CRITICAL" else 1)
    return tasks, live_nodes

failed = {"dn01", "dn07"}
tasks, live = simulate_failure(failed, block_map, HEALTHY_NODES
    )
```

```
47
48 print(f"Failed nodes: {failed}")
49 print(f"Surviving nodes ({len(live)}): {sorted(live)}\n")
50 print(f"{'Block':<10} {'Source':<8} {'Priority':<10} "
51       f"{'Deficit':>8} {'Surviving replicas'}")
52 print("-" * 60)
53 for blk, src, pri, deficit, surviving in tasks:
54     print(f"{blk:<10} {str(src):<8} {pri:<10} "
55           f"{deficit:>8}  {surviving}")
56 print(f"\n{len(tasks)} blocks require re-replication.")
```

Listing 3.3: PE3.3: Re-replication scheduler

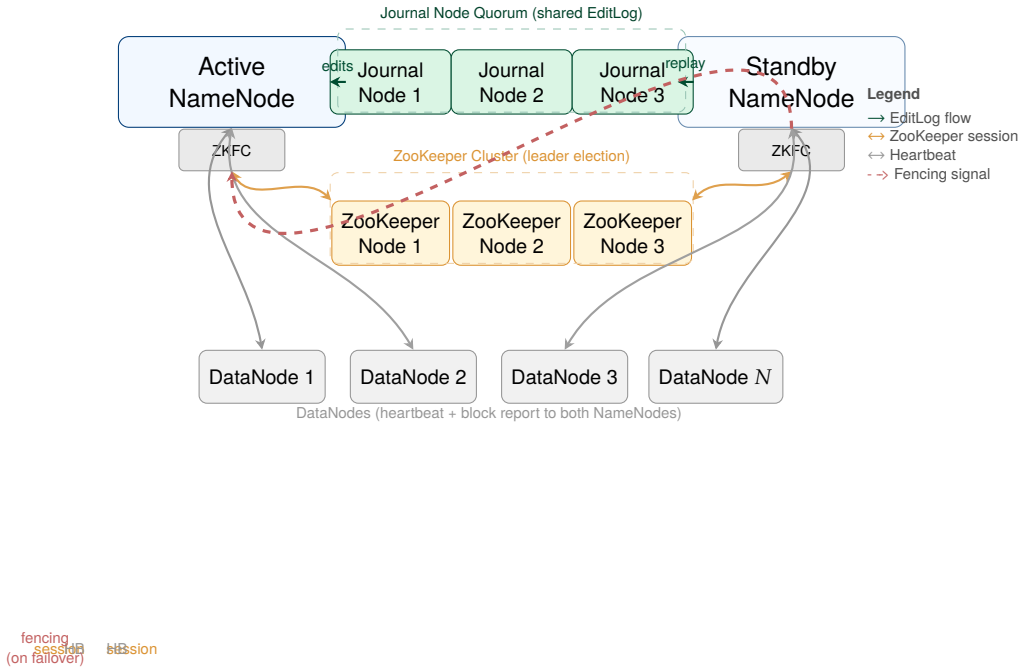

Figure 3.5: HDFS High Availability architecture. The Active NameNode writes every namespace mutation to the Journal Node quorum (shared EditLog); the Standby NameNode continuously replays those edits to maintain a synchronised copy of the namespace. Both NameNodes run a ZooKeeper Failover Controller (ZKFC) that monitors health and maintains a ZooKeeper session. On Active failure, the ZKFC fences the former Active (preventing split-brain), then promotes the Standby to Active in seconds. DataNodes send heartbeats and block reports to *both* NameNodes, so the Standby has a live block map ready for immediate service.

Part III

Batch Processing at Scale

4

The MapReduce Paradigm and the Hadoop Processing Stack

"Our abstraction is inspired by the map and reduce primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a map operation to each logical record in our input in order to compute a set of intermediate key/value pairs, and then applying a reduce operation to all the values that shared the same key."

Dean & Ghemawat, OSDI 2004

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why a new computation model was required for petabyte-scale data on distributed file systems, and identify the specific limitations of SQL and MPI that MapReduce addresses.
2. Write a MapReduce program by defining the Map and Reduce functions, and trace the complete execution through Map → Shuffle/Sort → Reduce using a concrete example.
3. Explain the roles of the Combiner (local optimisation) and the Partitioner (reducer assignment) and quantify the network savings a Combiner provides.

4. Describe the full MapReduce execution pipeline: input splits, task scheduling with data locality, speculative execution, and output materialisation.
5. Explain how MapReduce achieves fault tolerance through deterministic task re-execution.
6. Write Pig Latin scripts using LOAD, FILTER, FOREACH, GROUP, JOIN, and STORE, and explain how Pig Latin is compiled to MapReduce.
7. Write HiveQL queries using CREATE TABLE, partitioning, bucketing, and SELECT with GROUP BY, and explain the role of the Hive Metastore.
8. Describe the HBase data model (row key, column families, column qualifiers, timestamps) and the HBase architecture (HMaster, RegionServers, WAL, MemStore, HFile).
9. Select the appropriate tool from the Hadoop stack—raw MapReduce, Pig, Hive, or HBase—for a given data engineering problem.

Chapter 3 established how petabytes of data are stored reliably across a cluster of commodity servers. Storage, however, is only half the challenge. The data must be processed: indexed, aggregated, joined, filtered, and analysed. In 2003, Google's engineers faced precisely this problem. Their GFS cluster held hundreds of terabytes of web crawl data; building the search index required processing every document, aggregating term frequencies, and computing PageRank— computations that would take thousands of hours on a single machine.

The solution they published in 2004 is the subject of this chapter: ***MapReduce***, a programming model that allows a programmer to express a large-scale computation as two simple functions—*map* and *reduce*—and have the framework automatically parallelise the execution across hundreds or thousands of machines, handling data partitioning, scheduling, fault tolerance, and inter-machine communication transparently (Dean and Ghemawat 2004). Over the following decade, MapReduce became the computational backbone of an entire ecosystem of data processing tools: Apache Pig, Apache Hive, and Apache HBase, all of which this chapter also covers.

4.1 Why a New Computation Model?

The data processing challenges of petabyte-scale distributed storage do not yield to the tools that work well for smaller datasets. Two dominant paradigms—relational SQL and MPI-style parallel programming—both fail in this environment for distinct reasons.

SQL on a single server requires data to be loaded into a relational schema, supports random access via indexes, and executes on a single instance (or a tightly coupled SMP cluster). None of these properties hold for unstructured petabyte data stored on thousands of commodity DataNodes. Sequential full-table scans of a petabyte dataset at 200 MB/s on a single server would take over 57 days. SQL's assumption of a schema-on-write data model is incompatible with semi-structured and unstructured data (web pages, log files, binary media). SQL-on-Hadoop tools such as Hive (discussed in Section 4.7) resolve these issues by compiling SQL into MapReduce jobs.

MPI (Message Passing Interface) is the standard for scientific high-performance computing. MPI programmes explicitly manage inter-process communication, data partitioning, and synchronisation. This gives the programmer maximum flexibility but at enormous cost: the programmer must manually handle every aspect of distribution. More critically, MPI does not handle fault tolerance: if any MPI process fails, the entire job must typically be restarted from scratch. In a cluster of thousands of commodity machines where failures occur daily, a long-running MPI job that restarts on every node failure is effectively unusable.

MapReduce resolves both limitations. It enforces a structured programming model (Map and Reduce functions) that is restrictive enough for the framework to automate parallelisation, scheduling, and fault recovery—but expressive enough to implement a broad class of large-scale data transformations. The framework, not the programmer, decides how many tasks to spawn, which machine runs each task, how to partition intermediate data, and what to do when a machine fails.

Key Insight | The Expressiveness of Map + Reduce

The surprising result of Dean and Ghemawat's paper is that an enormous variety of large-scale computations—word frequency, inverted index construction, PageRank, distributed sort, join operations, machine learning feature extraction—can be expressed as a single Map phase followed by a single Reduce phase (or a sequence of MapReduce stages). The restrictive two-function interface is a *feature*, not a limitation: its uniformity is exactly what allows the framework to manage all distribution and fault tolerance automatically.

4.2 The MapReduce Programming Model

MapReduce adopts the functional programming concepts of *map* and *reduce* from Lisp and ML, applied to distributed key-value pairs at petabyte scale. The programmer provides two pure functions—with no shared mutable state and no side effects—and the framework handles everything else.

Definition | The MapReduce Functions

Given input data represented as a set of key-value pairs (k_1, v_1):

Map:

$$\text{map} : (k_1,\, v_1) \longrightarrow \text{list}\big((k_2,\, v_2)\big)$$

The Map function processes each input record independently and emits zero or more intermediate key-value pairs.

Reduce:

$$\text{reduce} : (k_2,\, \text{list}(v_2)) \longrightarrow \text{list}\big((k_3,\, v_3)\big)$$

The Reduce function receives all intermediate values associated with a single key k_2 and aggregates them into the final output.

Between the Map and Reduce phases, the framework executes the ***shuffle and sort***: it collects all intermediate (k_2, v_2) pairs emitted by all Map tasks, groups them by key, and sorts each group, delivering to each Reduce task an iterator over all values $\text{list}(v_2)$ for a single key k_2. The programmer never

writes shuffle-and-sort logic; it is the framework's responsibility.

4.2.1 The Canonical Example: Word Count

The canonical MapReduce example is word count: given a collection of text documents, compute the frequency of every distinct word across the entire collection. This example is simple enough to follow in detail, yet it illustrates every essential phase of MapReduce execution.

```
# ----- Map function -----
# Input:  (docid, document_text)
# Output: (word, 1) for every word in the document

def map(docid, text):
    for word in text.split():
        emit(word.lower(), 1)

# Example: map("doc1", "To be or not to be") emits:
#   ("to", 1), ("be", 1), ("or", 1), ("not", 1), ("to", 1), ("be", 1)

# ----- Shuffle and Sort (framework-managed) -----
# Groups all emitted pairs by key, sorts alphabetically:
#   ("be",  [1, 1, 1, ...])    <- all 1s from every document
#   ("not", [1, 1, ...])
#   ("or",  [1, ...])
#   ("to",  [1, 1, 1, ...])

# ----- Reduce function -----
# Input:  (word, iterator_of_counts)
# Output: (word, total_count)

def reduce(word, counts):
    total = sum(counts)
    emit(word, total)

# Example: reduce("be", [1,1,1,1,1]) emits: ("be", 5)
```

Listing 4.1: MapReduce word count: Map and Reduce functions

Each document in the collection is assigned to an independent Map task; if there are 10 million documents, 10 million Map tasks run in parallel (subject to cluster capacity). Each Map task processes only its own documents and

emits word counts for that subset. The framework then collects all $(word, 1)$ pairs from all Map tasks, groups them by word, and routes each group to a single Reduce task, which sums the counts. The output is the complete word frequency table for the entire collection.

4.2.2 Additional MapReduce Examples

Word count is illustrative, but the MapReduce model extends to a much wider class of computations. Two further examples demonstrate its generality.

Inverted index construction. A search engine's inverted index maps each term to the list of documents containing it. The Map function emits (term, docid) for each term in each document. After shuffle and sort, each Reduce task receives (term, [$\text{docid}_1, \text{docid}_2, \ldots$]) and concatenates the list into the posting list for that term.

Distributed sort. To sort a petabyte dataset, the Map function emits each record as (sort_key, record). The shuffle and sort phase sorts all intermediate keys globally (using a Partitioner that distributes key ranges uniformly across Reducers). Each Reducer receives records in sorted order for its key range; the concatenation of all Reducer outputs is globally sorted. This pattern— TeraSort—is the standard benchmark for distributed sorting, and Hadoop clusters have set world records for sorting 1 TB in minutes.

4.3 The Combiner and the Partitioner

Two optional but important components sit between the Map and Reduce phases: the Combiner and the Partitioner. Both are performance optimisations that the programmer can provide without changing the correctness of the computation.

4.3.1 The Combiner: Local Aggregation

In the word count example, each Map task might process a 128 MB input split containing hundreds of thousands of words. If the word "the" appears 50,000 times in that split, the Map task emits 50,000 `("the", 1)` pairs, all of which must be transferred over the network to the Reducer responsible for

"the". This is wasteful: the network cost is proportional to the number of word occurrences, not the number of distinct words.

The ***Combiner*** is a locally executed mini-Reducer that runs immediately after the Map phase *on the same node*. It receives the Map task's output and aggregates values for each key before they are shuffled across the network. For word count, the Combiner would reduce the 50,000 `("the", 1)` pairs to a single `("the", 50000)`, reducing the shuffle volume for this key by a factor of 50,000.

Combiners can be applied whenever the Reduce function is *commutative* and *associative*: if $f(a, f(b, c)) = f(f(a, b), c)$ and $f(a, b) = f(b, a)$, then partial application of f is always safe. Sum, count, min, and max all satisfy this property. Average does not—the Combiner cannot compute a local average because it does not know how many records the Reducer will receive from other Map tasks.

In practice, enabling a Combiner can reduce shuffle network traffic by 80–95% for aggregation workloads, dramatically reducing job completion time on bandwidth-constrained clusters.

4.3.2 The Partitioner: Directing Keys to Reducers

The ***Partitioner*** determines which Reducer receives each intermediate key. Given a key k and the number of Reduce tasks R, the default HashPartitioner computes:

$$\text{reducer\_id}(k) = h(k) \bmod R \tag{4.1}$$

where $h(k)$ is the hash of key k. This distributes keys uniformly across all Reduce tasks in expectation—the fundamental requirement for preventing ***reducer skew*** (one reducer receiving far more data than others and becoming the job's bottleneck).

For workloads where the hash distribution is insufficient—for example, when sorting requires that all keys in a range go to a specific Reducer—the programmer provides a custom Partitioner. The TeraSort example uses a ***TotalOrderPartitioner*** that divides the keyspace into R equal-sized ranges based on a sample of the input, ensuring that each Reducer receives a balanced partition of the sorted key space.

4.4 The Full MapReduce Execution Pipeline

Having defined the programming model, we can now trace the complete execution of a MapReduce job from input files on HDFS to output files on HDFS, through the framework's scheduling, data-locality optimisation, and fault recovery mechanisms.

4.4.1 Input Splits and the InputFormat

A MapReduce job begins with the client submitting a job to the cluster. The framework reads the input files from HDFS and divides them into ***input splits***—logical chunks of input data, typically aligned to HDFS block boundaries (128 MB by default). One Map task is instantiated per input split; the number of Map tasks is therefore approximately $\lceil$total input size/block size$\rceil$, matching the number of HDFS blocks that contain the input.

The ***InputFormat*** is the abstraction that translates raw HDFS bytes into (k_1, v_1) pairs that the Map function consumes. The standard `TextInputFormat` produces `(byte_offset, line)` pairs—one per line of text. `SequenceFileInputFormat` reads Hadoop SequenceFile format. `CombineFileInputFormat` merges many small files into a single split, mitigating the small file problem in the context of MapReduce job scheduling.

4.4.2 Task Scheduling and Data Locality

The cluster ***JobTracker*** (in Hadoop 1.x) or YARN ***ResourceManager*** (Hadoop 2.x and later, covered in Chapter **??**) accepts the submitted job and begins scheduling Map tasks. For each input split, the framework attempts to schedule the Map task on one of the three DataNodes that hold a replica of the corresponding HDFS block—applying the data locality principle introduced in Chapter 1. If a data-local slot is unavailable, the framework falls back to rack-local, then to off-rack scheduling.

Data locality is critical for MapReduce performance. A Map task reading its input from a local disk can sustain 100–300 MB/s of throughput; the same task reading from a remote rack over a 1 Gbit/s network connection is limited to approximately 125 MB/s, and in a congested cluster may receive

less. For a job with thousands of Map tasks all reading simultaneously, the difference between local and off-rack I/O can mean the difference between a one-hour job and a three-hour job.

Reduce tasks have no concept of data locality with respect to the original input—their input comes from the network (the shuffle phase). Reduce tasks are scheduled on any available node with sufficient memory and CPU capacity.

4.4.3 The Shuffle and Sort in Detail

The shuffle is the most network-intensive phase of MapReduce. As each Map task produces output, it does not write it directly to HDFS (which would incur expensive replication); instead, it writes intermediate data to a local ***spill buffer*** in RAM (default 100 MB). When the buffer is 80% full, a background thread ***spills*** the buffer content to disk after sorting by key within the spill and applying the Combiner (if present). Multiple spills from a single Map task are merged into a single sorted file before the task completes.

Each Reduce task then actively ***fetches*** its portion of the intermediate data from each Map task's output file via HTTP (a process called the ***copy phase***). As data arrives from different Map tasks, the Reduce side merges the multiple sorted streams (using a merge-sort) to produce a single sorted sequence of (k_2, v_2) pairs for each key. This merge-sort is the "sort" in shuffle-and-sort, and it is what guarantees that the Reduce function sees values for a single key in a well-defined, contiguous iterator.

4.5 Fault Tolerance in MapReduce

MapReduce's fault tolerance model is both simple and elegant: all Map and Reduce tasks are ***deterministic*** and ***idempotent***. A Map task that processes the same input split will always produce the same output; a Reduce task that processes the same sorted key-value sequence will always produce the same output. This determinism is the foundation of fault recovery.

The ***JobTracker*** (or YARN ResourceManager) monitors the progress of every running task through periodic ***heartbeats***. If a task has not reported

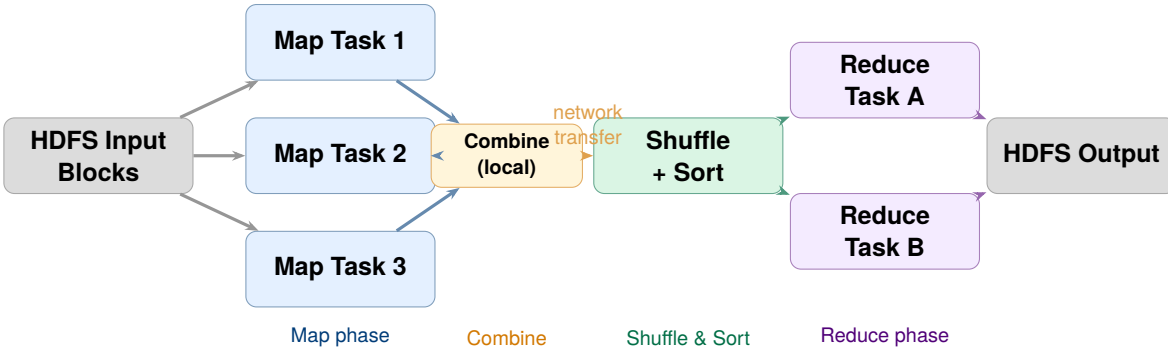

Figure 4.1: The full MapReduce execution pipeline. Map tasks run in parallel, each processing one input split. The optional Combiner reduces shuffle traffic. The framework-managed shuffle groups and sorts all intermediate data by key before delivering it to Reduce tasks.

progress within a configurable timeout (default 10 minutes), it is considered failed. The framework re-schedules the failed task on a different node, which re-reads the input split from HDFS and re-executes the Map function from scratch. Because HDFS provides three replicas of every input block, the task can always find its input on a surviving DataNode.

For Reduce tasks, re-execution is more expensive because the Reduce task may have already fetched a significant portion of its shuffle data. When a Reduce task fails, the framework restarts it from the beginning of the copy phase; all already-fetched intermediate data is discarded and re-fetched from Map task outputs.

Speculative execution addresses a different failure mode: the slow task (***straggler***). In a large cluster, statistical variation in hardware performance means that some tasks will always run significantly slower than average, even without a hard failure. A single slow Map task on a degraded disk or a hot machine can delay the entire job, because the Reduce phase cannot begin until all Map tasks complete. MapReduce mitigates this by detecting tasks that are running slower than expected and launching ***backup tasks***—duplicate executions of the same task on different nodes. Whichever task—original or backup—completes first writes its output; the other is killed. This has no correctness impact because both produce identical outputs (determinism), but can reduce job completion time by 40% in practice.

Table 4.1: MapReduce fault tolerance mechanisms

Failure type	Detection	Recovery mechanism
Map task failure	Heartbeat timeout	Re-schedule on another node; re-read HDFS block
Reduce task failure	Heartbeat timeout	Re-schedule; re-fetch all shuffle data
TaskTracker failure	Heartbeat timeout	Re-schedule all tasks that were running on that node
Slow task (straggler)	Progress rate below mean	Launch speculative backup task; keep first to complete
JobTracker failure	Not auto-recovered (Hadoop 1.x)	Manual restart; job history lost unless YARN HA used

4.6 Apache Pig: Dataflow Scripting

Raw MapReduce is powerful but verbose. A simple join of two datasets requires writing multiple Java classes: an InputFormat for each source, a custom Mapper that tags records with their source, a custom Partitioner, and a Reducer that handles the merge logic—hundreds of lines of boilerplate code. Apache Pig addresses this by providing a high-level dataflow scripting language called ***Pig Latin***, which compiles to one or more MapReduce jobs automatically.

Definition | Apache Pig

Apache Pig is a dataflow platform for analysing large datasets on Hadoop. Pig Latin programs describe a **directed acyclic graph (DAG) of data transformations**: each statement loads, filters, transforms, groups, joins, or stores a relation. The Pig compiler translates the DAG

into a sequence of MapReduce jobs, optimising the plan by combining compatible operations into single jobs where possible.

A Pig Latin script operates on ***relations***—bag-like collections of tuples, similar to relational tables but without requiring a predefined schema. The key operators are:

- `LOAD`: read data from HDFS into a named relation.
- `FILTER`: select tuples matching a predicate.
- `FOREACH ... GENERATE`: project or transform fields (analogous to SQL's `SELECT`).
- `GROUP`: group tuples by a key (generates a relation of (key, bag) pairs).
- `JOIN`: equi-join two relations on a key.
- `ORDER BY`: sort a relation by a key.
- `STORE`: write a relation to HDFS.

The following Pig Latin script analyses a web server access log to find the most-requested URLs that returned HTTP 404 errors:

```
-- Load the access log (tab-delimited)
logs = LOAD 'hdfs:///data/access_log'
       USING PigStorage('\t')
       AS (host:chararray, ts:chararray, method:chararray,
           url:chararray, status:int, bytes:long);

-- Keep only 404 responses
errors = FILTER logs BY status == 404;

-- Group by URL
grouped = GROUP errors BY url;

-- Count 404s per URL
counts = FOREACH grouped GENERATE
             group        AS url,
             COUNT(errors) AS num_errors;

-- Sort descending
ranked = ORDER counts BY num_errors DESC;

-- Keep top 20
```

```
top20 = LIMIT ranked 20;

-- Write output to HDFS
STORE top20 INTO 'hdfs:///output/top404' USING PigStorage(',');
```

Listing 4.2: Pig Latin: 404 error frequency analysis

The Pig compiler translates this seven-statement script into typically two MapReduce jobs: one for the FILTER + GROUP + FOREACH (a shuffle-intensive aggregate), and one for the ORDER BY + LIMIT (a second sort phase). The programmer writes dataflow logic; the framework decides how many MapReduce stages are needed and how to parallelise each one.

Pig is particularly well-suited for ad hoc exploratory analysis on HDFS data, for ETL pipelines that require complex multi-step transformations, and for data scientists who are comfortable with a scripting language but do not want to write Java MapReduce code. Its primary limitation—shared with raw MapReduce—is high latency: even simple queries require at least one MapReduce job, with startup overhead of 30–60 seconds before computation begins. For interactive analytical queries, Hive with a modern execution engine (Tez or Spark) or Presto (Chapter **??**) is more appropriate.

4.7 Apache Hive: SQL-on-Hadoop

Apache Hive, developed at Facebook and open-sourced in 2008, takes a different approach to Hadoop usability: instead of a new scripting language, it provides a SQL-like query language called ***HiveQL*** (or simply "Hive SQL") that compiles to MapReduce (and, since Hive 0.13, to Apache Tez or Apache Spark). Hive allows data analysts comfortable with SQL to query petabyte-scale datasets on HDFS without learning MapReduce or Pig Latin.

4.7.1 The Hive Metastore

For SQL queries to work over HDFS files, Hive needs a way to map SQL table names and column names to HDFS paths and file formats. This mapping is managed by the ***Hive Metastore***: a relational database (typically MySQL or PostgreSQL) that stores table definitions, schema information, partition metadata, and storage descriptors.

When a user creates a Hive table:

```
CREATE TABLE web_events (
    user_id     BIGINT,
    event_type  STRING,
    url         STRING,
    session_id  STRING,
    duration_ms INT
)
PARTITIONED BY (dt STRING)
STORED AS ORC
LOCATION 'hdfs:///data/web_events';
```

Listing 4.3: HiveQL: creating a partitioned ORC table

Hive records in the Metastore that the table `web_events` maps to the HDFS directory `/data/web_events`, is stored in ORC format, and is partitioned by the column `dt` (date string). Data for 2024-01-15 lives in the subdirectory `/data/web_events/dt=2024-01-15/`. The Metastore is the source of truth that the HiveQL compiler consults to generate the correct HDFS path for each query.

4.7.2 Partitioning and Bucketing

Partitioning is one of the most important performance techniques in Hive. A partitioned table stores its data in separate HDFS subdirectories, one per partition value. A query that filters on the partition column can skip reading all irrelevant partitions entirely— a technique called ***partition pruning***. For a table with 365 date partitions and 1 TB per partition (365 TB total), a query filtering on a single day reads only 1 TB rather than 365 TB:

```
-- Only reads /data/web_events/dt=2024-01-15/
-- Hive's query planner prunes all other 364 partitions
SELECT  event_type,
        COUNT(*)        AS event_count,
        AVG(duration_ms) AS avg_duration_ms
FROM    web_events
WHERE   dt = '2024-01-15'         -- partition filter
  AND   event_type IN ('purchase', 'add_to_cart')
GROUP BY event_type
ORDER BY event_count DESC;
```

Listing 4.4: HiveQL: partition pruning in a query

Bucketing provides a second level of data organisation within a partition. Rows are assigned to a fixed number of ***buckets*** (files) based on the hash of a designated column. Bucketing improves the efficiency of equi-joins (two tables bucketed on the join key and with the same number of buckets can be joined without a full shuffle) and enables efficient sampling (read only one bucket for a $1/N$ sample).

4.7.3 HiveQL to MapReduce Compilation

When a user submits a HiveQL query, the Hive driver executes the following steps:

1. **Parse**: convert the SQL string into an abstract syntax tree (AST).
2. **Semantic analysis**: resolve table and column names against the Metastore; validate types and permissions.
3. **Logical plan**: convert the AST into a logical query plan (a DAG of relational operators: scan, filter, project, aggregate, join).
4. **Optimisation**: apply rule-based optimisations (predicate push-down, column pruning, partition pruning) to reduce data read volume.
5. **Physical plan**: translate the logical plan into a sequence of MapReduce (or Tez, or Spark) jobs.
6. **Execution**: submit the jobs to the cluster and monitor their progress.

A `SELECT ... GROUP BY` query maps straightforwardly to a single MapReduce job: the Map function applies the predicate and projects the required columns; the Reduce function aggregates by the GROUP BY key. A multi-table `JOIN` may generate two or more MapReduce jobs depending on the join strategy (common-join, map-join, or bucket-map-join).

Research Spotlight | Thusoo et al. (2009) — Hive: A Warehousing Solution over a Map-R

Citation: Thusoo, A., et al. (2009). Hive – A Warehousing Solution over a Map-Reduce Framework. *Proceedings of the VLDB Endowment*, 2(2):1626–1629. (Thusoo et al. 2009)

Key insight: The paper's central argument is that SQL is the right interface for large-scale data analysis—not because SQL is technically superior to MapReduce, but because SQL is what hundreds of millions of analysts already know. Hive's contribution is a translation layer, not

a new programming model.

Technical contributions:

- Defined the Hive data model: tables, partitions, buckets, and complex data types (arrays, maps, structs) as SQL extensions compatible with semi-structured data.
- Introduced the *Metastore* architecture for mapping SQL schemas to HDFS directory structures, enabling schema-on-read while providing schema-on-write semantics for the SQL layer.
- Demonstrated that a HiveQL-to-MapReduce compiler could express all SQL-92 analytics workloads (aggregation, joins, subqueries, user-defined functions) on petabyte datasets.

Impact today: Hive is the most-used large-scale SQL engine in the history of open-source data engineering. Its Metastore is now an independent component used by Spark SQL, Presto/Trino, and Amazon Athena as their shared table catalog. The HiveQL dialect has become the *de facto* standard for analytical SQL over big data platforms, influencing the syntax of SparkSQL, BigQuery, and Snowflake.

4.8 Apache HBase: Column-Family NoSQL on Hadoop

MapReduce, Pig, and Hive are all batch processing tools: they read from HDFS, process data, and write back to HDFS, with latencies measured in minutes to hours. There is a class of workload for which this latency is unacceptable: applications that need to read or write individual records by key in milliseconds—user profile lookups, messaging systems, real-time recommendation updates, and fraud scoring.

Conventional HDFS provides no such capability: its read path requires a NameNode round trip and a DataNode TCP connection per block, delivering 50–200 ms latency per operation. Apache HBase, modelled directly on Google's Bigtable (Chang et al. 2006), provides *random read/write access* at sub-10 ms latency on data stored on HDFS—bridging the gap between the Hadoop ecosystem and the performance requirements of operational applications.

4.8.1 The HBase Data Model

HBase stores data in a model that differs fundamentally from both the relational model and the key-value model.

Definition | The HBase Data Model

An HBase table is a sorted map of:

$$(\text{row\_key},\ \text{column\_family} : \text{qualifier},\ \text{timestamp}) \longrightarrow \text{value}$$

- **Row key:** an arbitrary byte array, lexicographically sorted. Row key design is the central HBase schema design decision; the entire table is physically sorted by row key, enabling efficient range scans.
- **Column family:** a named group of columns, defined at table creation time. All columns in a family are stored together on disk.
- **Column qualifier:** a dynamic column name within a family, created on the fly without altering the table schema. A single row can have zero or thousands of qualifiers within a family.
- **Timestamp:** each cell stores multiple versions, indexed by timestamp. The most recent version is returned by default; older versions are accessible by specifying a timestamp.

Consider a social network's user activity table:

```
# Row structure: (row_key, column_family:qualifier, timestamp)
    -> value

# User u-10042's profile
("u-10042", "profile:name",     T3) -> "Ada Lovelace"
("u-10042", "profile:country",  T3) -> "GB"
("u-10042", "profile:join_date",T1) -> "2019-03-14"

# User u-10042's activity (sparse: not every qualifier exists
    for every user)
("u-10042", "activity:last_login",  T5) -> "2024-01-15T14:22:31
    Z"
("u-10042", "activity:post_count",  T4) -> "847"
("u-10042", "activity:follower_ct", T4) -> "12941"

# Multiple versions: profile:name was updated
```

```
("u-10042", "profile:name", T1) -> "Ada Byron"
("u-10042", "profile:name", T3) -> "Ada Lovelace"  <- default (
    latest)
```

Listing 4.5: HBase data model: user activity table

The row key design `u-10042` is intentional: all rows for a user's profile and activity are co-located in the same row, enabling an atomic, single-row fetch of the complete user record without a join. This ***denormalisation by design*** is characteristic of column-family databases and reflects the NoSQL principle that schema design should optimise for the read access pattern, not for normalisation.

4.8.2 HBase Architecture

HBase's architecture closely mirrors the GFS/HDFS master-worker pattern, adapted for random access.

The ***HMaster*** is the cluster coordinator. It handles table creation, deletion, and schema changes; assigns ***regions*** (contiguous ranges of row keys) to ***RegionServers***; and monitors RegionServer health via ZooKeeper. The HMaster is not in the read/write data path—all client I/O goes directly to RegionServers after the initial routing lookup.

RegionServers are the worker processes that handle all read and write operations. Each RegionServer manages between 10 and 1,000 regions. When a client writes a record, the RegionServer:

1. Appends the write to the ***Write-Ahead Log*** (WAL)—an append-only HDFS file that ensures durability even if the RegionServer crashes before flushing to disk.
2. Writes the record into the ***MemStore***: an in-memory sorted map (implemented as a Red-Black tree). All reads first check the MemStore.
3. Returns success to the client.

When the MemStore grows beyond a threshold (default 128 MB), it is ***flushed*** to HDFS as an immutable ***HFile*** (a sorted, block-compressed file in the HDFS namespace). Reads are served from a combination of the MemStore (for recent writes) and HFiles (for older data), with a Bloom filter on each HFile to avoid unnecessary disk reads for absent keys.

Over time, many small HFiles accumulate for a region. ***Compaction***

periodically merges HFiles into larger, consolidated files, eliminating deleted records and expired versions. This is the LSM-tree (Log-Structured Merge-tree) storage model: by converting all writes into sequential appends (WAL and HFile flushes), HBase achieves write throughput competitive with hash-based databases while maintaining sorted order for efficient range scans.

Research Spotlight | Chang et al. (2006) — Bigtable: A Distributed Storage System for S

Citation: Chang, F., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. *Proceedings of the 7th USENIX OSDI Conference*, pp. 205–218. (Chang et al. 2006)

Key insight: Bigtable demonstrates that a single, flexible data model—the sorted, multi-dimensional map—can efficiently support workloads as diverse as web indexing (sparse, write-heavy), Google Earth (large values, read-heavy), and Google Finance (time-series with many versions). The same model that serves all of Google's internal applications is now accessible to any developer through HBase and Google Cloud Bigtable.

Technical contributions:

- Introduced the *column family* abstraction—grouping related columns for co-location on disk—which enables efficient access to subsets of columns without reading entire rows.
- Described the *Tablet* architecture (equivalent to HBase's Region): range-partitioned, sorted data units managed by a Tablet Server, with the master handling only assignment and load balancing.
- Demonstrated the LSM-tree storage model in a production distributed system, establishing the architectural pattern that HBase, Cassandra, RocksDB, and LevelDB all follow.

Impact today: Bigtable is cited in over 13,000 papers. HBase (the open-source implementation) serves as the storage backend for Apache Phoenix (SQL on HBase), Facebook's internal messaging infrastructure (1 billion messages per day at peak), and numerous real-time recommendation systems. Google Cloud Bigtable (the managed service version) is used by companies including Spotify, Dow Jones, and T-Mobile.

4.9 Ecosystem Tool Selection

With four tools in the Hadoop processing stack—raw MapReduce, Pig, Hive, and HBase—the practical question for a data engineer is which tool to use for a given problem. The selection matrix below provides a decision framework based on access pattern, latency requirement, user skill set, and data structure.

Table 4.2: Hadoop processing stack tool selection matrix

Tool	Best use case	Latency	Interface	Not suited for
Raw MapReduce	Custom iterative algorithms; tight performance control	Hours	Java	Interactive queries; rapid development
Apache Pig	Multi-step ETL; ad hoc data exploration; complex transforms	Hours	Pig Latin (script)	Interactive queries; SQL-familiar analysts
Apache Hive	Reporting & BI; data warehouse queries; SQL analysts	Minutes (batch)	HiveQL (SQL)	Sub-second queries; random reads
Apache HBase	Random read/write; operational data; real-time lookups	<10 ms	Java API / REST	Full table scans; complex analytics

A practical guideline: use **Hive** as the default for SQL-capable analysts running reporting and BI workloads over HDFS data; use **Pig** when a pipeline requires complex multi-step ETL that would be awkward to express

in SQL; use **raw MapReduce** when performance tuning or an algorithm that does not map cleanly to Map/Reduce (such as iterative graph algorithms) is required; use **HBase** when the application requires sub-millisecond random access to individual records by key. In practice, Hive over Apache Tez (which avoids materialising intermediate results to HDFS) or Hive over Spark (Chapter 6) replaces much of the original Hive-on-MapReduce workload, delivering 10–100× lower latency.

4.10 Case Study: Facebook Data Infrastructure (2008–2012)

Case Study | Facebook — Hive and HBase at the Petabyte Frontier

Background. By 2008, Facebook had accumulated more than 2.5 petabytes of user data and was generating approximately 15 terabytes of new compressed data per day. The company needed to provide its data analytics team—several hundred analysts—with the ability to query this data interactively in SQL, without requiring them to write MapReduce jobs. Simultaneously, Facebook Messenger was processing over one billion messages per day and required sub-millisecond random access to individual message threads for retrieval.

The Hive deployment. Facebook's data infrastructure team developed Hive internally and open-sourced it to Apache in 2008. By 2010, Facebook's Hadoop cluster had grown to over 2,400 nodes with 30 PB of storage, running over 7,500 Hive queries per day submitted by analysts across the company. The Hive Metastore tracked more than 5,000 registered tables, the largest of which—Facebook's "events" table—exceeded 1 PB of compressed ORC data.

A key engineering challenge was query latency. Early Hive-on-MapReduce queries over large tables took 20–60 minutes due to MapReduce job startup overhead and the requirement to materialise all intermediate results to HDFS. Facebook's team introduced several optimisations that were later contributed to open source: predicate push-down (filter early), partition pruning (read only relevant date partitions), and index-

based skip-scan. These reduced the median analyst query time from 35 minutes to under 5 minutes.

The HBase deployment. Facebook used HBase as the storage backend for Facebook Messenger and Inbox. Each user's message thread was stored as an HBase row, with individual messages as column qualifier-value pairs. The row key design was critical: Facebook chose a reversed-timestamp prefix to ensure that the most recent messages for a thread were lexicographically adjacent on disk, enabling efficient range scans for loading a conversation inbox.

At peak, Facebook's HBase cluster served over **one billion messages per day** across a cluster of several hundred RegionServers. The P99 read latency for fetching a user's 20 most recent messages was approximately 8 milliseconds—competitive with purpose-built NoSQL databases and far below the latency achievable with HDFS-native batch access.

The architectural lesson. Facebook's use of both Hive and HBase simultaneously—on the same underlying HDFS cluster—illustrates a key principle of data engineering: different workloads require different tools, but those tools can share the same storage layer. Hive queries and HBase operations both read from and write to HDFS DataNodes; the NameNode manages the unified namespace. This shared storage model avoids data duplication and simplifies the operational burden of managing two separate storage systems.

By 2012, Facebook's Hadoop cluster had grown to over 100 petabytes, making it the largest known Hadoop deployment in the world at that time. The infrastructure team published a series of papers and blog posts documenting their engineering decisions, which directly influenced the design of subsequent generations of distributed analytics tools—particularly the development of Apache Spark, which addressed the primary limitation of the Hive-on-MapReduce stack: latency.

Chapter Summary

- **MapReduce** resolves two limitations of prior parallel computing paradigms: unlike SQL, it requires no predefined schema and handles unstructured data on HDFS; unlike MPI, it handles fault tolerance automatically through deterministic task re-execution.
- The MapReduce model requires the programmer to provide two functions: **map** $(k_1, v_1) \rightarrow \text{list}(k_2, v_2)$ and **reduce** $(k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$. The framework handles parallelisation, shuffle-and-sort, and fault recovery.
- The **Combiner** (local mini-reducer) reduces shuffle network traffic for commutative-associative operations. The **Partitioner** controls key-to-reducer assignment via hash or custom range logic.
- **Fault tolerance** is achieved through deterministic task re-execution on failure; **speculative execution** launches backup tasks for slow stragglers.
- **Apache Pig** compiles a dataflow scripting language (Pig Latin) to MapReduce, enabling multi-step ETL without Java programming.
- **Apache Hive** compiles HiveQL (SQL dialect) to MapReduce. The **Metastore** maps SQL tables to HDFS paths. **Partitioning** enables partition pruning; **bucketing** accelerates equi-joins and sampling.
- **Apache HBase** provides random read/write access at <10 ms latency on HDFS, using the LSM-tree model: WAL for durability, MemStore for recent writes, HFiles for historical data, compaction to merge HFiles.
- Tool selection: Hive for SQL analytics; Pig for scripted ETL; raw MapReduce for custom algorithms; HBase for random-access operational data.

Exercises

Short Questions

SQ4.1. Why does SQL on a single server fail for petabyte-scale data on HDFS? Name three specific assumptions SQL makes that do not hold in the HDFS environment.

SQ4.2. Write the type signatures of the Map and Reduce functions in the MapReduce model. What is the framework's responsibility between the Map and Reduce phases?

SQ4.3. Trace the word count MapReduce computation for the input string "the cat sat on the mat the cat". List every (k, v) pair emitted by the Map function, the grouped shuffle output, and the final (k, v) output of the Reduce function.

SQ4.4. What is the Combiner? For what class of Reduce functions can a Combiner be applied safely? Give an example of a Reduce function for which a Combiner would give incorrect results.

SQ4.5. What is speculative execution in MapReduce? Under what circumstances does the framework launch a backup task, and why does it not affect the correctness of the computation?

SQ4.6. A MapReduce job has 1,000 Map tasks and 50 Reduce tasks. Describe what happens when one Map task fails partway through execution. Describe what happens when one Reduce task fails after it has already fetched 80% of its shuffle data.

SQ4.7. What is the Hive Metastore? What information does it store, and what would happen to a HiveQL query if the Metastore database were unavailable?

SQ4.8. Explain partition pruning in Hive. A table has 365 daily partitions each containing 500 GB of ORC data (total 182 TB). A query filters on a single day. How much data does Hive read, and what is the speedup factor compared to a full-table scan?

SQ4.9. What is the HBase data model? Describe the four coordinates that uniquely identify a cell in an HBase table.

SQ4.10. Explain the LSM-tree write path in HBase: what happens to a client write at each of the WAL, MemStore, and HFile stages?

SQ4.11. What is HBase compaction? Explain the difference between minor and major compaction, and why compaction is necessary for read performance.

SQ4.12. Compare Apache Pig and Apache Hive on three dimensions: target user, language paradigm, and primary use case.

SQ4.13. A data engineer needs to find the top 10 most active users in a 1-petabyte social media activity dataset (schema: user_id, event_type, ts). Which Hadoop tool would you recommend, and why?

SQ4.14. True or false, with justification: "HBase stores data in HDFS, so its read latency is determined by HDFS block access latency (50–200 ms)." *(Hint: consider the MemStore and the block cache.)*

Analytical Problems

AP4.1. MapReduce Shuffle Volume Analysis. A MapReduce word count job processes a 1 TB corpus of English text. The average word length is 5 characters, and the average word frequency is 100 occurrences per 128-MB input split. There are 100,000 distinct words in the corpus. The cluster has 500 Map tasks.

(a) Estimate the total number of $(word, 1)$ pairs emitted by all Map tasks (assume 128 MB per split, and that the text has an average word density of 150 words per KB).

(b) Estimate the total shuffle data volume in GB without a Combiner (each pair requires approximately $|word| + 8$ bytes for the key-value pair overhead).

(c) With a Combiner enabled, each Map task emits at most one pair per distinct word. Estimate the new shuffle volume and the reduction factor.

(d) If the cluster's inter-node network bandwidth is 10 Gbit/s per node, and the shuffle phase transfers all data through the network simultaneously, estimate the minimum time for the shuffle to complete (a) without and (b) with the Combiner.

AP4.2. Hive Partitioning Strategy. An e-commerce company has a `transactions` table (500 TB total, 5 years of data) with columns: `transaction_id`, `customer_id`, `product_id`, `amount`, `country`, `ts` (timestamp).

(a) The company's most frequent query pattern is: "What was the total revenue per product in Germany in Q1 2024?" Propose a partitioning

scheme. Which column(s) should be the partition key(s), and how much data does the optimised query read?

(b) A second query pattern is: "For a given customer, list all their transactions across all time." Evaluate whether your partitioning scheme from (a) also optimises this query. If not, propose how to handle both query patterns (hint: consider secondary indexes or a separate HBase table).

(c) If the data is additionally bucketed on `customer_id` into 256 buckets, and a join query joins this table with a 30-GB `customers` dimension table on `customer_id`, explain what bucket-map-join does and estimate the speedup over a common join.

AP4.3. HBase Row Key Design Analysis. A healthcare analytics system stores patient vital signs readings in HBase. Each reading has: `patient_id` (8-char string), `metric` (heart_rate, SpO2, temperature, blood_pressure), and `timestamp` (epoch milliseconds).

The following three row key designs are proposed:

- **Design A:** `patient_id + timestamp` (e.g., `P-000042|1705315200000`)
- **Design B:** `timestamp + patient_id` (e.g., `1705315200000|P-000042`)
- **Design C:** `patient_id + reverse_timestamp` (e.g., `P-000042|9994684799999`)

(a) For Design B, describe the write hotspot problem: if all writes have approximately the same timestamp, what happens to the distribution of writes across RegionServers?

(b) For Design A, explain how a range scan for "all vitals for patient P-000042 in the last 24 hours" would be executed.

(c) For Design C (reverse timestamp), explain what a scan for "most recent 10 readings for patient P-000042" looks like. Why does this design outperform Design A for this access pattern?

(d) Propose a salting strategy for Design A that eliminates the write hotspot for high-frequency patients.

AP4.4. MapReduce vs. Pig vs. Hive Performance Comparison. A financial services firm must compute the total trading volume per stock symbol per day for one year of trade records (1.5 TB 150 DataNodes, 10 Gbit/s network). Estimate completion time for each tool:

(a) **Raw MapReduce** (Java): Map emits `(symbol+date, amount)`; Reduce sums. Assume 80% data locality, 200 MB/s local read. 50 Reduce tasks. Estimate total Map I/O time, shuffle time (assume 20% of input volume is shuffled), and Reduce time.

(b) **Apache Pig**: same logical plan as MapReduce, compiled to one MapReduce job with 30 seconds additional startup overhead per job. Estimate total elapsed time.

(c) **Apache Hive on MapReduce**: same as Pig with 60 seconds additional startup overhead (Metastore lookup + plan compilation). But if the table is partitioned by `date`, and the query selects a 30-day period, how much data is read, and what is the new estimated time?

(d) Which tool minimises calendar time? Which minimises engineering effort? Which minimises query latency for an analyst who runs this query daily?

Design Problems

DP4.1. MapReduce Pipeline for a Search Engine Inverted Index. Design a MapReduce pipeline that builds a compressed inverted index from a 500-terabyte web crawl stored in HDFS. Each crawl record is a JSON document containing the URL, crawl timestamp, and raw HTML. The inverted index maps each term to a posting list of `(url, tf-idf_score)` pairs, sorted by score descending.

Your design must specify:

(a) The number of MapReduce stages required and what each stage computes (hint: tf-idf requires knowing the total document frequency of each term across the entire corpus, which is not available in a single pass).

(b) The Map and Reduce function type signatures and logic for each stage.

(c) The Partitioner strategy to ensure all records for the same term go to the same Reducer in the appropriate stage.

(d) How the Combiner can be applied in Stage 1 and what the network savings are for a typical term with 10,000 occurrences per 128-MB input

split.

(e) How the pipeline handles the case where a single extremely common term (e.g., "the") causes a Reducer skew: the single Reducer responsible for "the" processes 100× more data than the average Reducer.

DP4.2. HBase Schema Design for a Real-Time Recommendation System. An online streaming music service needs to store and serve user listening history and song metadata for a real-time recommendation engine. Requirements:

- 50 million users; each user has an average of 2,000 play events, growing at 10 events/day.
- Each play event: `user_id`, `song_id`, `play_ts`, `duration_played_sec`, `skipped` (boolean).
- The recommendation engine reads the 200 most recent plays for a user in <5 ms.
- A nightly batch job (Hive or Spark) must scan all plays in the last 7 days across all users to recompute collaborative filtering features.

Design the HBase schema. Your design must specify:

(a) The table name(s), column families, and column qualifiers.

(b) The row key format, with explicit justification for how it supports the 5-ms recommendation read requirement.

(c) How the "most recent 200 plays" query is executed (scan direction, stop condition, use of reverse timestamp).

(d) How to configure cell versioning to automatically expire play events older than 90 days without a separate compaction job.

(e) How the nightly batch Hive/Spark job reads from HBase (what API it uses and what the scan looks like).

(f) The estimated RegionServer count if each region is limited to 10 GB, and the strategy for pre-splitting regions at table creation to avoid write hotspots.

Programming Exercises

PE4.1. MapReduce Word Count in Python (MRJob-style). Implement a word count MapReduce job in pure Python without any framework, simulating the Map, Shuffle/Sort, and Reduce phases. Extend it with a Combiner and measure the shuffle volume reduction.

```
import re
from collections import defaultdict

documents = [
    ("doc1", "To be or not to be that is the question"),
    ("doc2", "All the world is a stage and all the men and
        women"),
    ("doc3", "To be is to do to do is to be to be is to do"),
    ("doc4", "Be not afraid of greatness some are born great"),
]

# ---------- MAP PHASE ----------
def mapper(doc_id, text):
    """Emit (word, 1) for every word in text."""
    for word in re.findall(r'[a-z]+', text.lower()):
        yield (word, 1)

# ---------- COMBINER (optional local aggregation) ----------
def combiner(map_output):
    """
    Locally aggregate (word, 1) pairs from a single mapper.
    Input:  iterable of (word, 1) tuples
    Output: iterable of (word, local_count) tuples
    """
    local_counts = defaultdict(int)
    for word, count in map_output:
        local_counts[word] += count
    return list(local_counts.items())

# ---------- SHUFFLE AND SORT ----------
def shuffle_and_sort(all_map_output):
    """
    Group all (word, value) pairs by word.
    Returns dict: word -> list of values
    """
    grouped = defaultdict(list)
    for word, val in all_map_output:
        grouped[word].append(val)
    return dict(sorted(grouped.items()))  # sorted by key
```

```
39
40 # ---------- REDUCE PHASE ----------
41 def reducer(word, values):
42     """Sum all counts for a word."""
43     return (word, sum(values))
44
45 # ---------- JOB RUNNER ----------
46 def run_mapreduce(documents, use_combiner=True):
47     all_map_output = []
48     per_mapper_counts = []
49
50     for doc_id, text in documents:
51         map_out = list(mapper(doc_id, text))
52         per_mapper_counts.append(len(map_out))
53
54         if use_combiner:
55             map_out = combiner(map_out)
56
57         all_map_output.extend(map_out)
58
59     shuffle_out = shuffle_and_sort(all_map_output)
60
61     results = {}
62     for word, values in shuffle_out.items():
63         _, count = reducer(word, values)
64         results[word] = count
65
66     shuffle_volume = sum(len(str(w)) + len(str(v))
67                          for w, v in all_map_output)
68     return results, shuffle_volume, per_mapper_counts
69
70 # Run without and with Combiner
71 res_no_comb,  vol_no,  cnts_no  = run_mapreduce(documents,
       use_combiner=False)
72 res_with_comb, vol_yes, cnts_yes = run_mapreduce(documents,
       use_combiner=True)
73
74 print("Top 10 words:")
75 for w, c in sorted(res_with_comb.items(), key=lambda x: -x[1])
       [:10]:
76     print(f"  {w:<15} {c}")
77
78 print(f"\nShuffle volume WITHOUT combiner: {vol_no:,} bytes")
79 print(f"Shuffle volume WITH    combiner: {vol_yes:,} bytes")
```

```
print(f"Reduction factor: {vol_no / vol_yes:.1f}x")
```

Listing 4.6: PE4.1: MapReduce word count with Combiner

PE4.2. HiveQL Query Writing and Partitioning Analysis (Python simulation). Simulate the effect of Hive partitioning on query I/O by implementing a partition-pruning model that shows how much data is skipped for various query predicates.

```
import random
from datetime import date, timedelta

# Simulate a partitioned Hive table: (date, country) -> size_gb
random.seed(42)
countries = ["US", "GB", "DE", "FR", "JP", "IN", "BR", "AU"]
start = date(2023, 1, 1)

# Build partition metadata (what the Hive Metastore stores)
partitions = {}
for i in range(365):
    dt = (start + timedelta(days=i)).isoformat()
    for country in countries:
        size_gb = random.uniform(0.5, 2.5)
        partitions[(dt, country)] = round(size_gb, 3)

total_tb = sum(partitions.values()) / 1024
print(f"Total table size: {total_tb:.2f} TB across "
      f"{len(partitions):,} partitions\n")

def query_io(predicate_dt=None, predicate_country=None):
    """
    Simulate partition pruning for a query with optional
        filters.
    predicate_dt      : exact date string or None (full scan)
    predicate_country : country code or None (full scan)
    Returns: (partitions_read, gb_read)
    """
    read = {k: v for k, v in partitions.items()
            if (predicate_dt      is None or k[0] ==
                predicate_dt)
            and (predicate_country is None or k[1] ==
                predicate_country)}
    return len(read), sum(read.values())

```

```
queries = [
    ("Full table scan",
     dict(predicate_dt=None, predicate_country=None)),
    ("Single date (2024-01-15)",
     dict(predicate_dt="2024-01-15", predicate_country=None)),
    ("Single country (US), all dates",
     dict(predicate_dt=None, predicate_country="US")),
    ("Single date + country (2024-01-15, DE)",
     dict(predicate_dt="2024-01-15", predicate_country="DE")),
]

total_parts = len(partitions)
total_gb    = sum(partitions.values())

print(f"{'Query':<45} {'Parts read':>12} {'GB read':>10} {'Pruned':>8}")
print("-" * 78)
for desc, kwargs in queries:
    n_parts, gb = query_io(**kwargs)
    pruned_pct = (1 - n_parts / total_parts) * 100
    print(f"{desc:<45} {n_parts:>12,} {gb:>10.1f} {pruned_pct:>7.1f}%")
```

Listing 4.7: PE4.2: Hive partition pruning simulator

PE4.3. HBase MemStore and Compaction Simulator (Python). Simulate HBase's write path: WAL append, MemStore insertion, flush to HFile, and minor compaction.

```
import time

MEMSTORE_THRESHOLD = 10  # flush after 10 entries (demo scale)
COMPACTION_THRESHOLD = 3  # compact after 3 HFiles

class HBaseRegionSimulator:
    def __init__(self, region_name):
        self.region_name   = region_name
        self.wal           = []          # Write-Ahead Log (list of ops)
        self.memstore      = {}          # row_key -> {col -> value}
        self.hfiles        = []          # list of sorted dicts
        self.write_count   = 0

```

```
    def put(self, row_key, column, value):
        ts = int(time.time() * 1000)
        # 1. Append to WAL
        self.wal.append({"op": "PUT", "row": row_key,
                         "col": column, "val": value, "ts": ts
                             })
        # 2. Write to MemStore
        if row_key not in self.memstore:
            self.memstore[row_key] = {}
        self.memstore[row_key][column] = (value, ts)
        self.write_count += 1

        # 3. Flush if threshold reached
        if len(self.memstore) >= MEMSTORE_THRESHOLD:
            self._flush()

    def _flush(self):
        if not self.memstore:
            return
        # Sort by row key (lexicographic) and materialise as
            HFile
        hfile = dict(sorted(self.memstore.items()))
        self.hfiles.append(hfile)
        print(f"  [FLUSH] MemStore -> HFile #{len(self.hfiles)}
             "
              f"({len(hfile)} rows)")
        self.memstore.clear()

        # Compact if too many HFiles
        if len(self.hfiles) >= COMPACTION_THRESHOLD:
            self._minor_compact()

    def _minor_compact(self):
        print(f"  [COMPACT] Merging {len(self.hfiles)} HFiles
            ...")
        merged = {}
        for hfile in self.hfiles:
            for row, cols in hfile.items():
                if row not in merged:
                    merged[row] = {}
                merged[row].update(cols)
        self.hfiles = [dict(sorted(merged.items()))]
        print(f"  [COMPACT] Done. 1 HFile with {len(merged)}
            rows.")
```

```
53
54      def get(self, row_key):
55          # Check MemStore first
56          if row_key in self.memstore:
57              return {"source": "memstore", "data": self.memstore
                    [row_key]}
58          # Scan HFiles (most recent first)
59          for hfile in reversed(self.hfiles):
60              if row_key in hfile:
61                  return {"source": "hfile", "data": hfile[
                        row_key]}
62          return None
63
64      def status(self):
65          print(f"\nRegion '{self.region_name}' status:")
66          print(f"  WAL entries : {len(self.wal)}")
67          print(f"  MemStore    : {len(self.memstore)} rows")
68          print(f"  HFiles      : {len(self.hfiles)}")
69
70  # Simulate 25 writes to a user-activity HBase region
71  region = HBaseRegionSimulator("user-activity-0001")
72
73  users = [f"u-{i:05d}" for i in range(1, 8)]
74  import random; random.seed(0)
75  for _ in range(25):
76      uid = random.choice(users)
77      col = random.choice(["activity:logins", "activity:purchases
            ",
78                           "profile:name", "profile:country"])
79      val = str(random.randint(1, 1000))
80      region.put(uid, col, val)
81
82  region._flush()  # flush remaining MemStore
83  region.status()
84
85  # Test a GET
86  print("\nGET u-00001:", region.get("u-00001"))
```

Listing 4.8: PE4.3: HBase write path and compaction simulator

5

Resource Management, File Formats, and I/O Optimisation

"Storing data in HDFS is cheap. Storing it in the wrong format is expensive — you pay the cost every single time you query it."

Julien Le Dem, Apache Parquet co-creator, 2014

Learning Objectives

After completing this chapter, you will be able to:

1. Identify the specific bottlenecks of the Hadoop 1.x JobTracker architecture and explain why they imposed an approximate 4,000-node scalability ceiling.
2. Describe YARN's three-component architecture—ResourceManager, NodeManager, and ApplicationMaster—and trace the complete job execution flow from client submission to container completion.
3. Explain what a YARN Container is, why it replaced the fixed MapReduce *slot* model, and how it enables framework-agnostic resource sharing.
4. Compare YARN's three pluggable schedulers—FIFO, Capacity, and Fair—and select the appropriate scheduler for a given multi-tenant production scenario.

5. Describe YARN's three-tier fault tolerance model (NodeManager, ApplicationMaster, and ResourceManager failures) and identify which tier each failure type is handled at.
6. Explain the fundamental difference between row-oriented and columnar storage, derive the I/O reduction formula for a k-column query on a c-column table, and apply it to a concrete performance calculation.
7. Describe the internal structure of a Parquet file—row groups, column chunks, and pages—and explain how the footer enables predicate pushdown to skip row groups without reading them.
8. Compare Avro, Parquet, and ORC on orientation, schema model, splittability, compression, and primary use case; and select the appropriate format for a given data engineering workload.
9. Explain the splittability property for compression codecs and explain why Gzip-compressed CSV is an anti-pattern in HDFS.
10. Describe Dominant Resource Fairness (DRF) and explain why single-resource fairness metrics fail for heterogeneous workloads.

Chapters 3 and 4 established the two pillars of the original Hadoop stack: HDFS stores data, and MapReduce processes it. By 2012, however, two engineering problems were threatening the viability of that stack at production scale.

The first problem was *scheduling*. A Hadoop 1.x cluster could only run MapReduce jobs; when Apache Spark (2010) and Apache Tez (2013) emerged as significantly faster alternatives, they had no path to cluster resources because the MapReduce JobTracker was hardcoded to manage only MapReduce tasks. Organisations that wanted to use Spark had to maintain a completely separate cluster, with all the associated duplication of hardware, operations effort, and data movement overhead.

The second problem was *storage format*. As analytical workloads evolved from full-table scans to targeted column selections—driven by the growth of SQL analytics (Hive) and machine learning feature engineering—the row-oriented file formats that Hadoop had inherited (CSV, JSON, sequence files) became serious performance bottlenecks. Selecting 3 columns from a

100-column CSV table on HDFS required reading and discarding 97% of the data on every query.

YARN (Vavilapalli et al. 2013) and columnar file formats (Parquet, ORC) were the industry's responses to these two problems. This chapter covers both in depth, then examines the compression and I/O optimisation techniques that complete the performance picture.

5.1 The Hadoop 1.x Scheduling Bottleneck

In Hadoop 1.x, cluster resource management and MapReduce job execution were fused into a single process: the ***JobTracker***. The JobTracker performed four distinct functions simultaneously:

1. Accepted job submissions from clients.
2. Allocated fixed ***slots***—Map slots and Reduce slots—to tasks running on TaskTrackers.
3. Monitored every running task and detected failures.
4. Maintained the job history for completed jobs.

This fusion created three interlocking problems that collectively prevented Hadoop 1.x from scaling beyond approximately 4,000 nodes.

Framework lock-in. The slot allocation model was hardcoded for MapReduce. A Map slot could run exactly one Map task; a Reduce slot could run exactly one Reduce task. There was no abstraction for the resource requirements of a non-MapReduce task—no way for a Spark executor to request a container of 8 GB RAM and 4 CPU cores, for example. Any framework that needed to run on the cluster had to pretend to be MapReduce.

Rigid slot granularity. Map slots and Reduce slots were configured with fixed memory and CPU limits cluster-wide. A job with small Map tasks and large Reduce tasks could not allocate more memory to Reducers at the expense of Mappers; both were constrained by the same slot size. A node with 10 Map slots and 0 available Reduce slots could not repurpose its unused Map slots to serve waiting Reduce tasks, even if it had spare RAM and CPU capacity.

Single-point scalability. The JobTracker's memory footprint grew linearly with the number of running tasks: each task required a metadata

entry in the JobTracker's heap. At 4,000 nodes with 10 tasks per node, the JobTracker managed 40,000 task records simultaneously. Beyond this scale, JVM garbage collection pressure made the JobTracker unresponsive (Vavilapalli et al. 2013).

5.2 YARN: Yet Another Resource Negotiator

Apache Hadoop YARN, published by Vavilapalli et al. in 2013, resolved all three bottlenecks through a single architectural decision: separate cluster resource management from application-specific execution logic (Vavilapalli et al. 2013). YARN replaces the monolithic JobTracker with three distinct components, each with a clearly bounded responsibility.

5.2.1 The ResourceManager

The ***ResourceManager*** (RM) is the cluster-level resource scheduler. It runs on a dedicated master node and contains two sub-components.

The ***Scheduler*** is responsible for allocating ***containers***— the YARN resource abstraction—to running applications. The Scheduler is *pluggable*: operators can choose from three built-in policies (described in Section 5.3) or implement a custom policy. Critically, the Scheduler has no knowledge of the application logic; it only knows the container requirements (`{vcores: N, memory: M MB}`) and the cluster's current capacity. This separation means the same Scheduler can manage MapReduce tasks, Spark executors, Samza stream processors, and any other future framework without modification.

The ***ApplicationsManager*** accepts job submissions from clients and coordinates the launch of each application's first container, which will host the ApplicationMaster.

5.2.2 The NodeManager

A ***NodeManager*** (NM) runs on every worker node in the cluster. Its responsibilities are narrowly scoped to the local machine:

- Launch containers requested by ApplicationMasters, using Linux cgroups for CPU isolation and enforced memory limits.

- Monitor the resource usage (CPU, RAM, disk I/O, network) of every container on the local node and report to the ResourceManager via periodic heartbeats.
- Kill any container that exceeds its allocated memory limit to protect other containers on the same node.
- Aggregate container logs and upload them to HDFS for post-job debugging.

The NodeManager has no global view of the cluster; it knows only about its own node. This is a deliberate design choice: the scalability of the YARN control plane depends on the RM and each NM having well-bounded state, with the RM collecting aggregate information rather than processing per-task details.

5.2.3 The ApplicationMaster

The ***ApplicationMaster*** (AM) is the component that makes YARN framework-agnostic. One ApplicationMaster runs per submitted job, inside its own container allocated by the ResourceManager. The AM contains all application-specific logic that the old JobTracker had embedded:

- **Resource negotiation:** the AM communicates with the RM to request containers for its tasks. A Spark AM requests containers with specific CPU and memory configurations for Spark Executors; a MapReduce AM requests separate Map-phase and Reduce-phase containers.
- **Task placement:** the AM instructs NodeManagers to launch specific tasks inside allocated containers. The AM can apply its own data-locality preferences when choosing which nodes to request containers from.
- **Progress monitoring:** the AM tracks the progress of all running tasks and re-requests containers for failed tasks.
- **Framework-specific fault tolerance:** the MapReduce AM implements speculative execution; the Spark AM manages executor loss and task re-submission.

The AM model means YARN is, in the authors' words, "a cluster operating system" (Vavilapalli et al. 2013): the RM and NMs are the kernel that manages hardware resources; the AM is a user-space process that

implements the policy of a specific framework. Just as Linux supports arbitrary user-space processes, YARN supports arbitrary AMs.

5.2.4 The Container: YARN's Resource Unit

Definition | YARN Container

A **YARN Container** is an allocation of a specific quantity of cluster resources on a specific node:

$$\text{Container} = \{\ \text{node} : m,\ \ \text{vCores} : n,\ \ \text{memory} : M\ \text{MB}\ \}$$

The Container replaces Hadoop 1.x's fixed Map/Reduce slot with a flexible, application-specified resource envelope. The NodeManager on node m enforces the allocated limits; any container exceeding its memory allocation is immediately killed by the NM.

The Container abstraction is the key enabler of framework diversity. Because a Container is specified in terms of generic resources (vCores and RAM)—not in terms of task types—the same ResourceManager can grant containers to a MapReduce AM requesting small (2 vCPU, 4 GB) Map-task containers, a Spark AM requesting large (8 vCPU, 32 GB) executor containers, and a Samza AM requesting tiny (1 vCPU, 2 GB) stream-task containers, all simultaneously on the same cluster.

5.2.5 YARN Job Execution Flow

The YARN job execution follows six steps from client submission to completion:

1. **Submit.** The client submits the job to the ResourceManager's ApplicationsManager, providing the application JAR (or binary), configuration, and resource requirements.
2. **Launch AM container.** The RM allocates one container on an available NodeManager for the ApplicationMaster. The NM launches the AM process inside that container.
3. **AM registers and requests resources.** The AM registers itself with the RM and submits resource requests: a list of containers with their required

vCores, RAM, and preferred nodes (for data locality).

4. **RM grants containers.** The Scheduler allocates containers based on the scheduling policy and current cluster availability. Granted containers are communicated to the AM.
5. **AM launches tasks.** For each granted container, the AM contacts the appropriate NodeManager via RPC and instructs it to launch a specific task process inside the container.
6. **Task execution, monitoring, and completion.** Tasks run inside containers, report progress to the AM. On task completion, the AM releases the container back to the RM. When all tasks complete, the AM unregisters from the RM, and the RM releases the AM's own container.

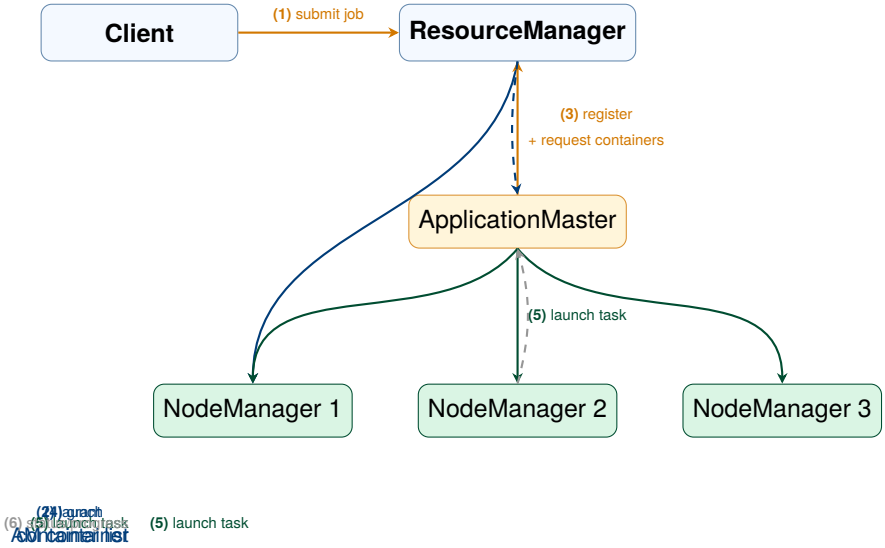

Figure 5.1: YARN job execution flow. The ResourceManager (RM) manages global resources and launches the ApplicationMaster (AM) in its own container. The AM then negotiates additional containers with the RM and instructs NodeManagers to launch application tasks, monitoring their progress independently of the RM.

5.3 YARN Schedulers: Sharing the Cluster

With multiple teams submitting jobs simultaneously to a shared cluster, the Scheduler's policy determines which jobs get resources first and how much capacity each team is guaranteed. YARN provides three built-in scheduler implementations.

5.3.1 The FIFO Scheduler

The ***FIFO (First In, First Out) Scheduler*** maintains a single job queue and assigns resources in submission order. The first job submitted receives all available cluster capacity until it completes; subsequent jobs wait. FIFO is simple to understand and configure, but it is unsuitable for multi-tenant clusters. A single long-running batch job (e.g., a multi-hour ETL pipeline) can starve time-sensitive interactive queries for hours. FIFO is appropriate only for single-user development clusters or automated pipeline clusters that run one job at a time.

5.3.2 The Capacity Scheduler

The ***Capacity Scheduler***, the default in Hadoop distributions from Hortonworks and Cloudera, divides the cluster's total capacity into named ***queues***. Each queue has a guaranteed minimum capacity and a maximum capacity:

- The **minimum capacity** ensures that jobs in this queue can always obtain at least $X\%$ of cluster resources, even when other queues are busy.
- The **maximum capacity** prevents a single queue from consuming the entire cluster, even when the cluster is otherwise idle.
- **Elastic borrowing**: when a queue has unused capacity, other queues can borrow it. The capacity is reclaimed—with a configurable grace period for preemption—when the owning queue submits new jobs.

A typical multi-team Hadoop cluster might configure:

Queue	Min capacity	Max capacity
`production`	60%	80%
`analytics`	30%	60%
`adhoc`	10%	40%

The production queue always has at least 60% of cluster capacity available for revenue-generating pipelines, even if the analytics and adhoc queues are idle.

5.3.3 The Fair Scheduler

The ***Fair Scheduler***, the default in Apache Hadoop distributions and historically used by Facebook and Twitter, allocates resources so that all running jobs receive an equal share of available capacity over time. When only one job is running, it receives 100% of the cluster. When a second job is submitted, it begins receiving resources immediately; over a short window, both jobs converge on 50% each. When the second job completes, the first job again receives 100%.

The key property of the Fair Scheduler is its treatment of newly submitted short jobs. Under the Capacity Scheduler, a short interactive query submitted to a busy production queue must wait behind already-running batch jobs. Under the Fair Scheduler, the new query preempts some resources from running jobs immediately, completing quickly before returning those resources to the batch jobs. This makes the Fair Scheduler particularly effective for clusters that serve a mix of batch pipelines and interactive analytical queries.

5.3.4 Dominant Resource Fairness

Both the Capacity and Fair Schedulers must decide how to measure "fairness" in a cluster with multiple resource dimensions (vCores and RAM). A job requiring many CPUs but little memory and a job requiring few CPUs but much memory have different resource profiles; allocating equal CPU to both may leave one job RAM-starved while the other has excess RAM.

Dominant Resource Fairness (DRF), introduced by Ghodsi et al. at NSDI 2011 (Ghodsi et al. 2011), resolves this by defining each job's fairness share in terms of its *dominant resource*—the resource type for which the job's relative allocation is highest. If a job consumes 20% of the cluster's CPU but only 5% of its RAM, its dominant resource is CPU and its dominant share is 20%. The DRF scheduler equalises dominant shares across all running jobs, ensuring that no job dominates the cluster in the resource dimension that

matters most to it. YARN's Fair Scheduler implements DRF as its fairness metric for multi-resource scheduling.

5.3.5 YARN Fault Tolerance

YARN's fault tolerance is layered across three tiers, matching the three components of its architecture.

NodeManager failure. If a NodeManager stops sending heartbeats to the ResourceManager (default timeout: 10 minutes), the RM marks it as dead. All containers on that NodeManager are immediately marked as failed, and the RM notifies the relevant ApplicationMasters. Each AM decides how to respond according to its framework's fault tolerance policy: a MapReduce AM re-requests replacement containers for failed Map or Reduce tasks; a Spark AM marks the executor as lost and reschedules its tasks on surviving executors.

ApplicationMaster failure. If the AM process crashes, the RM detects the failure via a missed heartbeat and restarts the AM in a new container (up to a configurable maximum number of attempts, default 2). The restarted AM can recover job state from HDFS checkpoints if the framework supports it. MapReduce AMs checkpoint completed task outputs to HDFS, allowing a restarted AM to resume from the last completed stage. Early versions of Spark did not checkpoint AM state, so AM failure required restarting the entire job.

ResourceManager failure. In Hadoop 1.x, RM failure was catastrophic: the entire cluster became inaccessible. YARN 2.4 introduced ***ResourceManager High Availability***, mirroring the HDFS HA design: an Active RM and a Standby RM, with ZooKeeper providing leader election and automatic failover in under 60 seconds. The Active RM's state (running applications, container allocations) is periodically checkpointed to ZooKeeper so the Standby can reconstruct it on promotion.

Research Spotlight | Vavilapalli et al. (2013) — Apache Hadoop YARN: Yet Another Resource

Citation: Vavilapalli, V. K., et al. (2013). Apache Hadoop YARN: Yet Another Resource Negotiator. *Proceedings of the 4th ACM Symposium on Cloud Computing (SoCC 2013)*, pp. 5:1–5:16. (Vavilapalli et al. 2013)

Key insight: The paper's central thesis is that the fusion of resource

management and MapReduce execution logic in Hadoop 1.x's JobTracker was the primary scalability and extensibility bottleneck. Separating these concerns—through the RM/NM/AM decomposition—makes the cluster a general-purpose computing platform rather than a MapReduce appliance.

Technical contributions:

- Defined the Container abstraction (`{node, vCores, memory}`) as the universal resource unit, replacing the rigid Map/Reduce slot model and enabling flexible, framework-specific resource allocation.
- Introduced the ApplicationMaster pattern: a per-application process that encapsulates framework-specific scheduling, fault tolerance, and monitoring logic, making YARN framework-agnostic by design.
- Demonstrated scalability to 6,000 nodes with 100,000 simultaneous tasks, exceeding Hadoop 1.x's 4,000-node limit by 50%, with ResourceManager scheduling latency below 1 second at 1,000 heartbeats/second.

Impact today: YARN is cited over 3,000 times and became the default cluster OS for the entire Hadoop ecosystem. Its three-tier architecture directly influenced Kubernetes's Pod/Node/Master model; the Container abstraction is the direct ancestor of the Kubernetes Pod. Apache Spark, Apache Tez, Apache Samza, Apache Flink, and TensorFlow on Hadoop all ran as YARN applications. The ApplicationMaster pattern established a design precedent that every subsequent cluster computing framework has replicated.

5.4 Row-Oriented vs. Columnar Storage

The second major performance dimension this chapter addresses is the physical organisation of data within files. The choice between row-oriented and columnar (column-oriented) storage can change the I/O cost of an analytical query by one to two orders of magnitude, making it one of the highest-leverage decisions in a data engineering system.

5.4.1 Row-Oriented Storage

In a ***row-oriented*** (or *N-ary storage model*, NSM) file format, all fields of a single record are stored contiguously on disk:

```
user_id=1, name="Alice", age=24, score=91, country="US", ...
user_id=2, name="Bob", age=31, score=78, country="DE", ...
user_id=3, name="Carol", age=27, score=88, country="JP", ...
```

Row-oriented storage is optimal for workloads that need to read or write complete records: transactional databases (OLTP), log ingestion pipelines, and streaming append operations. Writing a new record requires a single sequential write. Reading a specific record by key requires reading one contiguous block of bytes.

Row-oriented storage is *suboptimal* for analytical workloads that select a small number of columns from a wide table. A query `SELECT score FROM users` must read every field of every row— `user_id`, `name`, `age`, `country`, and all other columns—even though only the `score` field is needed. For a table with c columns, a query selecting k columns reads c/k times more data than necessary.

5.4.2 Columnar Storage: The I/O Reduction Formula

In a ***columnar*** (or *decomposition storage model*, DSM) file format, all values of a single column are stored contiguously on disk:

```
user_id: [1, 2, 3, ...]   name: [Alice, Bob, Carol, ...]
score: [91, 78, 88, ...]
```

For the same `SELECT score FROM users` query, a columnar store reads *only the score column chunk*—a contiguous block containing `[91, 78, 88, ...]`, and nothing else. The `user_id`, `name`, `age`, and other column chunks are not read at all.

Definition | Columnar I/O Reduction

Let a table have c columns, n rows, and $\bar{s}$ bytes per cell on average. A

query selecting k columns:

$$\text{Row store I/O} = n \cdot c \cdot \bar{s} \tag{5.1}$$

$$\text{Column store I/O} = n \cdot k \cdot \bar{s} \tag{5.2}$$

$$\text{I/O reduction} = \frac{k}{c} \tag{5.3}$$

For $k = 3$, $c = 100$: the columnar store reads only 3% of the data that a row store would read—a 33× reduction.

Additional compression efficiency: because all values in a column chunk have the same data type, they compress far better than a mixed row. A column of `country` codes ("US", "DE", "JP", ...) with high repetition compresses 10–50× better than an entire row containing a mix of integers, strings, and floats.

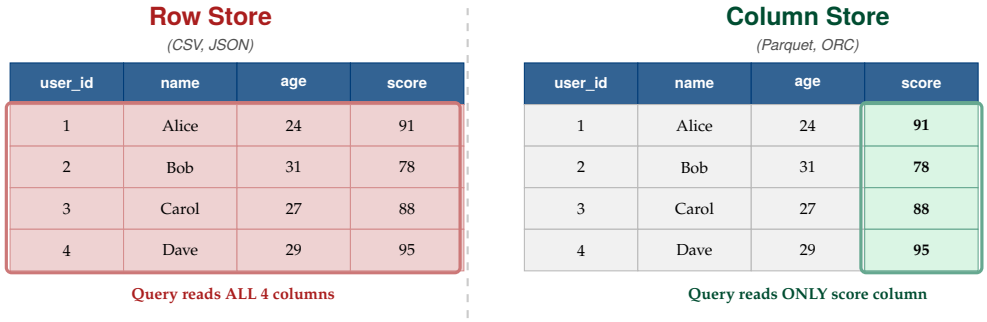

Figure 5.2: Row-oriented vs. columnar storage for a `SELECT score FROM users` query. The row store reads all four columns for every row (red box). The column store reads only the `score` column chunk (green box), skipping `user_id`, `name`, and `age` entirely. For 100 columns with a 3-column query, the columnar store provides a 33× I/O reduction.

5.5 Big Data File Formats in Depth

Three file formats dominate the modern Hadoop and cloud data lake ecosystem: Apache Avro, Apache Parquet, and Apache ORC. Each represents a different design point in the space of storage orientation, schema management, compression efficiency, and processing engine compatibility.

5.5.1 Apache Avro: Schema-Evolving Row Storage

Apache Avro is a row-oriented serialisation framework developed at Yahoo! in 2009 and now an Apache top-level project. Avro stores each record as a single contiguous binary blob, with the schema described as a JSON document embedded in the file header. It is the preferred format for data ingestion pipelines—particularly Kafka-to-HDFS streams—for two reasons.

Schema evolution. As event schemas change over time (new fields added, field names changed, types widened), Avro allows *reader schemas* and *writer schemas* to differ according to well-defined compatibility rules. A consumer reading data written with an older schema can specify a new reader schema; Avro's binary codec automatically:

- Fills missing fields with their declared default values.
- Ignores fields present in the data but absent from the reader schema.
- Promotes numeric types (e.g., `int` to `long`).

This forward and backward compatibility makes Avro the standard for streaming ingestion (Kafka) to long-term storage (HDFS), where the schema evolves continuously as the application changes.

Splittability. An Avro file is divided into ***blocks*** (not to be confused with HDFS blocks) separated by sync markers. A Hadoop InputFormat can seek to a sync marker and start reading from any block without decompressing from the beginning, making Avro files natively splittable.

5.5.2 Apache Parquet: Columnar Analytics Storage

Apache Parquet, developed jointly by Twitter and Cloudera in 2013, is the de facto standard for analytical data storage in the Hadoop ecosystem and on cloud data lakes. Its internal organisation is designed to minimise I/O for column-selective analytical queries.

File structure. A Parquet file is divided into ***row groups*** (default ~128 MB, aligning with HDFS block size). Each row group is subdivided into ***column chunks***—one per column. Each column chunk is further divided into ***pages*** of approximately 1 MB each. Data within a page is optionally dictionary-encoded (replacing repeated values with short integer codes) and then compressed with a configurable codec (Snappy by default).

Predicate pushdown. The Parquet ***file footer*** stores column-level

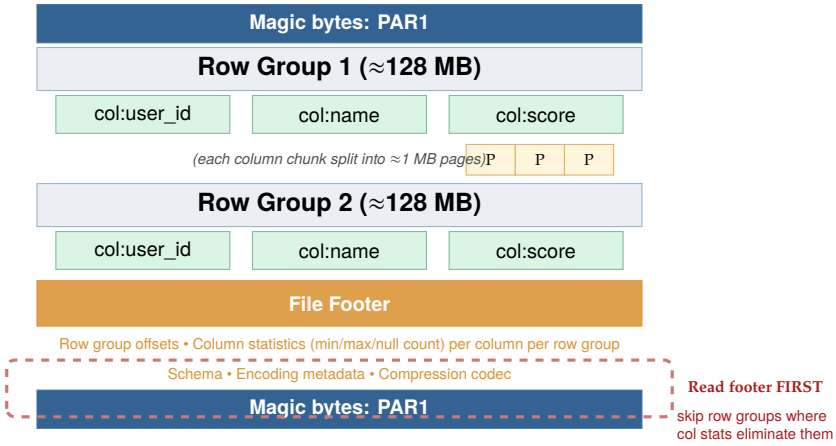

Figure 5.3: Parquet file internal structure. The footer contains per-column, per-row-group statistics (min, max, null count) that enable predicate pushdown: a query with `WHERE score > 90` reads the footer first, then skips all row groups where `max(score)` $\leq$ `90`, reading zero bytes from those row groups.

statistics—minimum value, maximum value, and null count—for every column chunk in every row group. A query engine (Spark SQL, Hive on Tez, Presto) reads the footer first. If the footer shows that the `score` column in Row Group 1 has min = 60, max = 85, and the query filters `WHERE score > 90`, the engine can skip the entire Row Group 1 without reading a single byte of its data. This *predicate pushdown* to the storage layer can reduce the physical I/O of a selective query from terabytes to gigabytes.

Dictionary encoding. Before compression, Parquet applies ***dictionary encoding*** to columns with low cardinality. A column containing country codes with 200 distinct values out of 100 million rows stores the 200 distinct values in a compact dictionary, then stores each row's value as a small integer index into the dictionary. This can reduce the size of a low-cardinality column by 80–95% before compression is even applied.

5.5.3 Apache ORC: Hive-Optimised Columnar Storage

Apache ORC (Optimised Row Columnar) was developed at Hortonworks in 2013 specifically for Apache Hive and is the recommended format for the Hive ecosystem. ORC and Parquet are architecturally similar—both are columnar, self-describing, and support predicate pushdown—but differ in

several engineering choices.

ORC uses ***stripes*** (analogous to Parquet's row groups, default 250 MB) and stores a ***file-level index*** in the footer plus a ***stripe-level index*** within each stripe. The stripe-level index provides statistics at 10,000-row granularity within the stripe, enabling finer-grained skipping than Parquet's row-group granularity.

ORC's default compression codec is Zlib (gzip), which achieves higher compression ratios than Parquet's default Snappy at the cost of slower decompression speed. For Hive workloads where query latency is measured in minutes rather than seconds, the storage savings from Zlib typically outweigh the decompression overhead. ORC also supports Hive's ACID (atomicity, consistency, isolation, durability) transactions—a feature Parquet does not natively support—enabling `UPDATE` and `DELETE` operations on Hive tables.

Table 5.1: Comparison of the five major Hadoop file formats

Format	Orient.	Schema	Split.	Compress.	Primary use case
Text/CSV	Row	External	With codec	None (default)	Small data; portability
JSON	Row	Self (verbose)	No (raw)	External	APIs; log ingestion
Avro	Row	Self (JSON)	Yes	Snappy/Deflate	Kafka ingest; schema evolution
Parquet	Columnar	Self	Yes	Snappy/Gzip	Spark, Presto, ML; analytics
ORC	Columnar	Self	Yes	Zlib/Snappy	Hive warehouse; ACID; archival

5.6 Compression Codecs and Splittability

Compression reduces the physical size of data on disk and in transit, lowering both storage costs and I/O time. In the Hadoop ecosystem, choosing the right compression codec requires balancing three dimensions: compression ratio, decompression speed, and—most critically for distributed processing—*splittability*.

5.6.1 The Splittability Property

Definition | Splittability

A file format is **splittable** if a Hadoop InputFormat can start reading from any point in the file without decompressing from the beginning. Splittability determines whether HDFS can assign multiple independent Map tasks to different portions of the same file, enabling full parallelism.

The practical consequence is dramatic. A 10 GB unsplittable Gzip CSV file on HDFS is stored in 80 consecutive 128 MB blocks. HDFS distributes these blocks across the cluster normally, but the MapReduce or Spark job can assign only a *single* task to this file: the task must read all 80 blocks sequentially from start to finish, since decompression must begin at byte 0 and proceed in order. No parallelism is possible.

A 10 GB Parquet+Snappy file with 128 MB row groups is similarly stored in 80 blocks. But the Parquet InputFormat can start reading at any row group boundary independently. The job assigns 80 parallel tasks, each reading one row group from a local DataNode—achieving 80× parallelism and completing the job in 1/80th the wall-clock time of the single-task case.

5.6.2 Compression Codec Comparison

Common Misconception | The Gzip Anti-Pattern

The most common performance anti-pattern in Hadoop data pipelines is storing large datasets as Gzip-compressed CSV or JSON files. Gzip's block format does not contain synchronisation markers that allow random seeks; decompression must begin at byte 0. A 1 TB Gzip

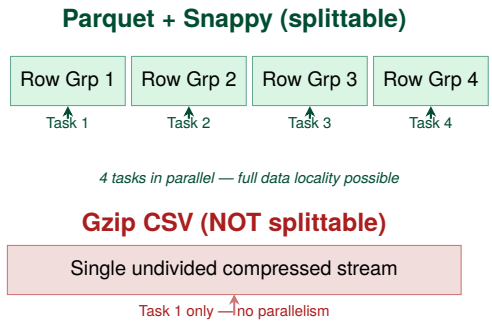

Figure 5.4: Splittability comparison. Parquet's row-group boundaries allow N tasks to process N row groups in parallel. A Gzip-compressed CSV file has no internal boundaries the InputFormat can seek to, so the entire file is processed by a single task.

CSV file processed by a 1,000-node Spark cluster is forced to run on *a single task*, using one core on one node, while the other 999 nodes sit idle. The correct alternative is **Parquet + Snappy**: the Parquet format provides splittable boundaries (row groups) even though Snappy itself is not splittable, and Snappy's very fast decompression minimises CPU overhead.

5.7 I/O Optimisation Techniques

Beyond format and codec selection, three additional query-time optimisation techniques reduce the I/O cost of analytical workloads on columnar storage systems.

5.7.1 Predicate Pushdown

Predicate pushdown moves filter conditions from the query engine into the storage layer. In Parquet, the file footer stores column-level statistics (min, max, null count) per row group. When a query contains a filter predicate (e.g., `WHERE revenue > 1000000`), the query engine evaluates the predicate against the footer statistics for each row group *before reading the row group's data*. Any row group for which the predicate is definitely false

Table 5.2: Hadoop compression codecs: performance and splittability

Codec	Speed	Ratio	Splittable?	Recommended use
Snappy	Fastest	Medium	No (but Parquet/Avro row groups are)	Parquet, Avro in Spark/Hive (default)
Gzip	Slow	High	**No**	Avoid for large HDFS files; use Parquet+Snappy instead
LZO	Fast	Medium	Yes (with index)	Legacy Hadoop; requires external index file
Bzip2	Slowest	Highest	Yes (natively)	Large plain-text files where I/O cost dominates
Zstd	Fast	High	No (block-level)	ORC archival; Parquet cold storage

(i.e., `max(revenue)` $\leq$ 1,000,000) is skipped entirely. Only row groups where the predicate *might* be true are read.

In practice, for a time-series table partitioned by date and sorted by user ID within each partition, predicate pushdown on a user-ID filter can skip 99% of the data in large Parquet files, reducing a 1-hour query to under 2 minutes.

5.7.2 Column Pruning

Column pruning (also called *projection pushdown*) ensures that only the columns required by the query are read from disk. In a row-oriented format, this is impossible: all columns of a row are stored together, so reading one requires reading all. In a columnar format, each column is stored

independently; the query engine passes the list of required columns to the file reader, which opens and reads only those column chunks. A query selecting 5 of 200 columns from a Parquet file reads exactly 5/200 = 2.5% of the storage bytes.

Apache Spark applies column pruning automatically for Parquet files through the Catalyst query optimiser: the optimiser analyses the query's column references before execution and passes only the referenced columns to the Parquet reader. No manual configuration is required.

5.7.3 Partition Pruning

Partition pruning, introduced in the context of Hive in Chapter 4, operates at the file system (HDFS directory) level rather than within a file. A table partitioned by `dt` (date) stores each day's data in a separate HDFS subdirectory (`/data/table/dt=2024-01-15/`). A query filtering on a specific date reads only that subdirectory's files, skipping all other partitions entirely. Partition pruning, predicate pushdown, and column pruning are complementary and can be applied simultaneously:

1. **Partition pruning** eliminates entire HDFS directories based on the partition key filter.
2. **Predicate pushdown** eliminates row groups within the remaining Parquet files based on column statistics in the footer.
3. **Column pruning** eliminates irrelevant column chunks within the remaining row groups.

The combined effect can reduce the I/O of a targeted analytical query from terabytes to megabytes—multiple orders of magnitude—purely through optimisations at the storage layer, without any changes to the query itself.

5.8 Case Study: LinkedIn's Multi-Framework YARN Cluster (2013–2016)

Case Study | LinkedIn — Consolidating Five Frameworks on One 5,000-Node YARN Clus

Background. By 2013, LinkedIn was processing approximately 500 TB of data per day for its professional networking platform—analysing member profiles, connections, job postings, messages, and advertising signals to power features used by over 300 million members. The company ran five distinct data processing frameworks: MapReduce (for Hive ETL), Apache Spark (for the People You May Know recommendation algorithm, PYMK), Apache Samza (for real-time Kafka stream processing), Apache Pig (for ad targeting feature engineering), and Apache Tez (for interactive Hive-on-Tez queries). Before YARN, these frameworks ran on *separate siloed clusters*—three distinct Hadoop clusters, each configured and operated independently.

The silo problem. The siloed cluster architecture created two systemic inefficiencies. First, each cluster was under-utilised: the MapReduce cluster averaged 45% utilisation (heavily used during nightly batch runs, idle during the day), while the Spark cluster was heavily utilised for the PYMK job's 4-hour compute window and nearly idle the rest of the time. Combining both clusters would have provided the same total capacity at significantly higher average utilisation. Second, cross-cluster data movement added 30–90 minutes of latency to pipelines that needed data from multiple clusters: an ETL job on the MapReduce cluster had to export results to HDFS and wait for the Spark cluster to ingest them.

The unified YARN architecture. In 2013, LinkedIn migrated to a unified 5,000-node YARN cluster using the Capacity Scheduler with six queues:

Queue	Capacity	Primary workload
`production`	40%	Hive ETL; Pig ad features
`spark`	25%	PYMK; Spark ML models
`samza`	15%	Kafka stream processing
`adhoc`	10%	Analyst Hive queries
`tez`	7%	Interactive Hive-on-Tez
`reserve`	3%	Overflow / burst capacity

The PYMK pipeline. LinkedIn's People You May Know ("Friend recommendations") feature illustrates how multiple frameworks share the YARN cluster in a single end-to-end pipeline:

1. **Ingest** (Samza, `samza` queue): a Kafka consumer reads member-profile-view events at approximately 2 million events per second, materialising them to HDFS in Avro format.
2. **Feature store** (Hive, `production` queue): a nightly Hive ETL job joins first-, second-, and third-degree connection data for all 300 million members into a wide feature table stored in ORC+Zlib format (partitioned by member-ID bucket).
3. **Graph model** (Spark GraphX, `spark` queue): a Spark job runs label propagation over the social graph to score approximately 10 billion candidate recommendation pairs per day. The PYMK Spark job used 600 executors each with 8 GB RAM and 4 vCPUs—4,800 GB of RAM and 2,400 vCPUs drawn from the `spark` queue's 25% allocation of the 5,000-node cluster.
4. **Serve**: top-k recommendations per member are written to Voldemort (LinkedIn's key-value store) for real-time serving at sub-10-millisecond latency.

Quantified outcomes. The migration from three siloed clusters to a unified YARN cluster produced measurable improvements across every operational dimension:

- **Utilisation**: average cluster utilisation rose from ~45% (across three separate clusters) to ~78% (on the unified cluster)—a 73% increase in effective compute capacity from the same hardware.
- **PYMK latency**: the PYMK Spark job recompute time dropped from 18 hours (on a separate, undersized Hadoop 1.x MapReduce cluster) to 4 hours (on YARN with Spark GraphX), enabling daily rather than every-other-day recommendation refreshes.
- **Operational cost**: consolidation eliminated one full team's worth of cluster operations overhead and reduced inter-cluster data synchronisation latency by 90%.
- **Peak containers**: at peak, the unified cluster ran over 400,000 simulta-

neous containers across all six queues.

The architectural lesson. LinkedIn's deployment established a principle that has since become an industry standard: *the cluster is the computer, and the resource manager is its operating system*. Just as an operating system allows multiple processes with different CPU and memory profiles to share hardware fairly, YARN allows multiple data processing frameworks with different resource profiles to share a cluster fairly. The Capacity Scheduler's queue hierarchy provides the "process priority" mechanism; the Container is the "process memory allocation"; the NodeManager's cgroup-based isolation is the "memory protection". This analogy— cluster OS / cluster kernel—was explicitly used by the YARN paper's authors and has guided the design of Kubernetes, Mesos, and every subsequent cluster resource manager.

Chapter Summary

- Hadoop 1.x's **JobTracker** bottleneck had three causes: framework lock-in (MapReduce only), rigid fixed-size slots, and single-process scalability limits (≈4,000 nodes).
- **YARN** separates resource management into three components: **ResourceManager** (global scheduler), **NodeManager** (per-node agent), and **ApplicationMaster** (per-job framework logic). A **Container** (`{node, vCores, memory}`) replaces the fixed slot.
- YARN's three schedulers: **FIFO** (single queue), **Capacity** (guaranteed multi-queue allocation), and **Fair** (equal sharing over time). **DRF** extends fairness to multiple resource dimensions.
- **Row-oriented** formats (CSV, JSON, Avro) store all fields of a record contiguously. **Columnar** formats (Parquet, ORC) store all values of a column contiguously. For a k-column query on a c-column table, columnar I/O is reduced by factor k/c.
- **Avro**: row-oriented, schema evolution (reader/writer schemas), splittable, Kafka-native. **Parquet**: columnar, row groups, footer statistics for predicate pushdown, Spark/Presto native. **ORC**: columnar, stripe-level

index, Zlib compression, Hive ACID support.

- A file is **splittable** if multiple tasks can read different ranges independently. Gzip CSV is not splittable (one task only). Parquet/Avro blocks are splittable even if the codec (Snappy) is not.
- The three I/O optimisation layers apply in order: **partition pruning** (skip directories), **predicate pushdown** (skip row groups via footer statistics), **column pruning** (read only required column chunks).

Exercises

Short Questions

SQ5.1. What is the YARN ResourceManager's Scheduler component? What information does the Scheduler use to make allocation decisions, and what information does it deliberately *not* know?

SQ5.2. Explain the role of the ApplicationMaster (AM) in YARN. Why does one AM per job—rather than a centralised AM for all jobs—improve the scalability of the ResourceManager?

SQ5.3. What is a YARN Container? Write the resource specification for a container that would host a Spark executor requiring 8 CPU cores and 24 GB of RAM on node `dn-07.cluster.internal`.

SQ5.4. Explain the difference between YARN's Capacity Scheduler and Fair Scheduler. Give a production scenario where the Capacity Scheduler is the better choice, and a scenario where the Fair Scheduler is better.

SQ5.5. What is Dominant Resource Fairness (DRF)? Give an example where allocating equal CPU across two jobs would result in an unfair outcome, and explain how DRF resolves it.

SQ5.6. A YARN NodeManager stops sending heartbeats to the ResourceManager. Trace the sequence of events that occurs: which component detects the failure, what happens to the containers on that NodeManager, and which component decides what to do next?

SQ5.7. What is the difference between the *write schema* and the *read schema* in Apache Avro? Give a concrete example where they differ and explain how Avro handles the mismatch.

SQ5.8. Explain predicate pushdown in Apache Parquet. What information is stored in the Parquet file footer, and how does a query engine use it to avoid reading data?

SQ5.9. What is the splittability property for a Hadoop file format? A 10 GB file on a 100-node cluster is stored as Gzip-compressed CSV. How many Map tasks does MapReduce assign to this file, and why?

SQ5.10. Compare Parquet and ORC on three dimensions: primary processing engine, default compression codec, and ACID transaction support.

SQ5.11. A data engineer proposes storing a 500 TB analytics warehouse as raw JSON files (one JSON object per line) in HDFS. Identify three specific performance problems with this proposal and propose a better alternative.

SQ5.12. Explain column pruning (projection pushdown). For a 200-column Parquet table, a query references 6 columns. What fraction of the physical storage does the Parquet reader access?

SQ5.13. What does it mean for a columnar format to use *dictionary encoding*? For a column containing the values "US", "GB", "DE", "FR" repeated across 500 million rows, explain how dictionary encoding reduces the column chunk size.

SQ5.14. YARN's ResourceManager HA uses ZooKeeper for failover. What role does ZooKeeper play, and what happens to running applications during the 30–60 second RM failover window?

Analytical Problems

AP5.1. YARN Capacity Planning. An e-commerce company operates a 2,000-node YARN cluster. Each node has 64 GB RAM and 16 vCPUs. The Capacity Scheduler is configured with three queues: `batch` (50%), `streaming` (30%), `adhoc` (20%). Container sizes: batch MapReduce mapper: {4 GB, 2 vCPU}; streaming Samza task: {2 GB, 1 vCPU}; adhoc Spark executor: {8 GB, 4 vCPU}.

(a) Calculate the total cluster RAM and vCPU capacity.

(b) For each queue, calculate the maximum number of concurrent containers, and identify whether the bottleneck resource is RAM or vCPU.

(c) A batch job needs 5,000 mapper containers simultaneously. How many batch nodes can serve this simultaneously? If demand exceeds the `batch` queue's allocation, describe what happens under (i) Capacity Scheduler with preemption and (ii) Capacity Scheduler without preemption.

(d) The streaming team submits a Samza job with 3,000 tasks, each requiring {2 GB, 1 vCPU}. Calculate the dominant resource for the streaming queue using DRF.

AP5.2. File Format I/O Comparison. A retail company stores 5 years of transaction records in HDFS. The table has 80 columns, 10 billion rows, and an average cell size of 12 bytes. A weekly analytics query selects 4 columns, filters on a date range covering 7 days out of 365, and has a predicate on one column that eliminates 90% of row groups.

(a) Calculate the total uncompressed table size in terabytes.

(b) If stored as CSV, estimate the I/O for the weekly query (no partition pruning, no predicate pushdown).

(c) If stored as Parquet + Snappy, partitioned by date, with predicate pushdown: apply partition pruning (7/365 days), then predicate pushdown (90% of remaining row groups skipped), then column pruning (4/80 columns). Calculate the remaining I/O.

(d) Express the I/O reduction as a ratio. If each I/O unit (TB read) costs the cluster 5 minutes of compute time on a 100-node cluster, estimate the query time saving.

AP5.3. Splittability and Parallelism Analysis. A genomics pipeline ingests 2 TB of DNA sequencing data per day. Three format options are proposed:

- **Option A**: single 2 TB Gzip-compressed FASTQ file
- **Option B**: 16,000 Gzip-compressed FASTQ files of 128 MB each
- **Option C**: Parquet files with 128 MB row groups (2 TB total across ≈16,000 row groups)

(a) For each option, calculate the number of Map tasks HDFS assigns, and

explain whether data-local execution is achievable.

(b) Option B produces 16,000 files. What is the NameNode memory overhead (assume 300 bytes per file + block pair, and each file fits in one 128 MB block)?

(c) Option C uses a single logical Parquet file of 2 TB divided into 16,000 row groups. Assuming perfect data locality, how many Map tasks run in parallel if the cluster has 500 nodes with 40 task slots each?

(d) Rank the three options on: (i) MapReduce parallelism, (ii) NameNode memory pressure, (iii) engineering simplicity. Recommend one option for the production pipeline.

AP5.4. YARN Failure Cascade Analysis. A 500-node YARN cluster is running a Spark job with 400 executor containers distributed across 400 nodes. Each executor holds 50 GB of cached RDD data (4 partitions of 12.5 GB each). A power distribution unit trip simultaneously takes down 50 nodes.

(a) How many executor containers are immediately lost?

(b) Which YARN components detect the node failure, and in what order?

(c) The Spark ApplicationMaster requests 50 replacement executors. If the remaining 450 nodes each have 4 available executor slots, how quickly can the RM satisfy the request (assume 500 ms scheduling cycle)?

(d) The replacement executors must recompute the 50 lost executors' cached RDD partitions (200 partitions, 12.5 GB each, at 100 MB/s recompute throughput). Estimate the total recompute time.

(e) What data engineering design change would eliminate the need for full RDD recomputation after executor failure?

Design Problems

DP5.1. Data Lake Format Strategy for a Financial Services Platform. A global investment bank ingests 15 sources of market data into a central data lake: real-time tick data (1 billion trades/day, each trade: {symbol, price, volume, timestamp, exchange, side}), reference data (static security

metadata: 2 million instruments, 120 columns), and regulatory reporting data (daily settlement reports, 80 columns, 7-year retention requirement).

Design the format and compression strategy for each data tier. Your design must specify:

(a) The HDFS file format and compression codec for each of the three source types, with explicit justification for each choice.

(b) The partitioning scheme for tick data (which columns to partition on, at what granularity, and why).

(c) The YARN queue configuration for three consumer teams: algorithmic trading analytics (latency-sensitive, needs <5-minute query response), risk analytics (batch, 2-hour window nightly), and compliance reporting (weekly, long-running, low priority).

(d) How you would handle schema evolution if the tick data schema adds three new fields for a new exchange that emits extended market data.

(e) The estimated storage reduction from converting raw CSV tick data (at 35 bytes per event on average) to Parquet+Snappy (assuming 6:1 compression ratio on this workload) for one year of data.

DP5.2. YARN Migration from Three Siloed Clusters. A media company runs three separate Hadoop clusters: Cluster A (1,000 nodes, MapReduce for nightly content ETL), Cluster B (500 nodes, Spark for recommendation models), Cluster C (300 nodes, Kafka/Samza for real-time user event processing). Average utilisation: Cluster A 40%, Cluster B 35%, Cluster C 55%. The company plans to consolidate into a single YARN cluster.

Design the consolidation. Your design must address:

(a) Total node count for the unified cluster, given a target average utilisation of 70% (justify your sizing).

(b) Capacity Scheduler queue configuration: queue names, capacity percentages, and maximum capacities for each team's workload.

(c) The migration sequence: which workload migrates first, how parallel operation during the transition is managed, and how you validate that the migrated workload performs correctly.

(d) How preemption should be configured between the nightly ETL jobs (MapReduce, 8-hour window) and the recommendation Spark jobs (4-hour window, started 2 hours before ETL completes).

(e) The rollback plan if the unified cluster shows unexpected performance degradation for the Samza real-time workload.

Programming Exercises

PE5.1. YARN Capacity Planning Calculator (Python). Implement a YARN capacity planning tool that models a multi-queue cluster, computes maximum concurrent containers per queue, identifies the bottleneck resource (RAM vs. vCPU), and detects over-subscription.

```
from dataclasses import dataclass

@dataclass
class NodeSpec:
    count:   int
    ram_gb:  int
    vcpus:   int

@dataclass
class QueueSpec:
    name:          str
    capacity_pct: float    # fraction of cluster [0, 1]
    container_ram_gb: int
    container_vcpus:  int

def plan_cluster(node: NodeSpec, queues: list[QueueSpec]):
    total_ram_gb = node.count * node.ram_gb
    total_vcpus  = node.count * node.vcpus

    print(f"Cluster: {node.count} nodes x {node.ram_gb} GB / {
        node.vcpus} vCPU")
    print(f"Total:   {total_ram_gb:,} GB RAM | {total_vcpus:,}
        vCPUs")
    print()
    print(f"{'Queue':<14} {'Cap':>5} {'RAM alloc':>10} {'CPU
        alloc':>10} "
          f"{'Max conts':>10} {'Bottleneck':>12}")
    print("-" * 65)

    for q in queues:
        q_ram_gb = total_ram_gb  * q.capacity_pct
```

```
        q_vcpus   = total_vcpus   * q.capacity_pct
        by_ram    = int(q_ram_gb  / q.container_ram_gb)
        by_cpu    = int(q_vcpus   / q.container_vcpus)
        max_c     = min(by_ram, by_cpu)
        bottle    = "RAM" if by_ram < by_cpu else "vCPU"
        print(f"{q.name:<14} {q.capacity_pct*100:>4.0f}% "
              f"{q_ram_gb:>9,.0f} GB {q_vcpus:>9,.0f}    "
              f"{max_c:>10,} {bottle:>12}")

    # Detect container spec errors: container bigger than node
    print()
    for q in queues:
        if q.container_ram_gb > node.ram_gb:
            print(f"[ERROR] Queue {q.name}: container RAM {q.
                container_ram_gb} GB "
                  f"> node RAM {node.ram_gb} GB")
        if q.container_vcpus > node.vcpus:
            print(f"[ERROR] Queue {q.name}: container vCPUs {q.
                container_vcpus} "
                  f"> node vCPUs {node.vcpus}")

# LinkedIn 5,000-node cluster model
node = NodeSpec(count=5_000, ram_gb=64, vcpus=16)

queues = [
    QueueSpec("production", 0.40, container_ram_gb=4,
        container_vcpus=2),
    QueueSpec("spark",      0.25, container_ram_gb=8,
        container_vcpus=4),
    QueueSpec("samza",      0.15, container_ram_gb=2,
        container_vcpus=1),
    QueueSpec("adhoc",      0.10, container_ram_gb=4,
        container_vcpus=2),
    QueueSpec("tez",        0.07, container_ram_gb=4,
        container_vcpus=2),
    QueueSpec("reserve",    0.03, container_ram_gb=4,
        container_vcpus=2),
]

plan_cluster(node, queues)
```

Listing 5.1: PE5.1: YARN capacity planning calculator

PE5.2. Parquet Layout and Predicate Pushdown Simulator (Python). Simu-

late the Parquet file structure—row groups, column chunks, footer statistics—and implement predicate pushdown to skip row groups for a simple filter predicate.

```
import random
import math

random.seed(42)

ROW_GROUP_ROWS = 1_000        # rows per row group (small for demo)
TOTAL_ROWS     = 10_000       # total table rows
COLUMNS        = ["user_id", "name", "country", "age", "score", "revenue"]
BYTES_PER_CELL = 8            # average bytes per cell

def generate_row_group(rg_id, start_row, rows):
    """Simulate one Parquet row group with column stats."""
    data = {
        "user_id": list(range(start_row, start_row + rows)),
        "score":   [random.randint(40, 100) for _ in range(rows)],
        "revenue": [random.randint(0, 1_000_000) for _ in range(rows)],
        "country": [random.choice(["US","GB","DE","JP","FR"]) for _ in range(rows)],
        "age":     [random.randint(18, 80) for _ in range(rows)],
        "name":    [f"User_{i}" for i in range(start_row, start_row + rows)],
    }
    # Build footer stats per column
    stats = {}
    for col, vals in data.items():
        if isinstance(vals[0], int):
            stats[col] = {"min": min(vals), "max": max(vals),
                          "null_count": 0}
        else:
            stats[col] = {"min": min(vals), "max": max(vals),
                          "null_count": 0}
    return {"rg_id": rg_id, "row_count": rows, "data": data, "stats": stats}

# Build Parquet file (footer + row groups)
```

```
num_rg  = math.ceil(TOTAL_ROWS / ROW_GROUP_ROWS)
rg_list = [generate_row_group(i, i * ROW_GROUP_ROWS,
                              min(ROW_GROUP_ROWS,
                                  TOTAL_ROWS - i *
                                      ROW_GROUP_ROWS))
           for i in range(num_rg)]

footer = {"num_row_groups": num_rg,
          "schema": COLUMNS,
          "row_group_stats": [rg["stats"] for rg in rg_list]}

def parquet_scan(row_groups, footer, predicate_col,
    predicate_op, threshold,
                 select_cols):
    """
    Simulate Parquet scan with:
      - predicate pushdown (skip row groups via footer stats)
      - column pruning (read only select_cols)
    """
    total_rg = len(row_groups)
    rg_read  = 0
    rows_returned = 0

    for i, rg in enumerate(row_groups):
        stats = footer["row_group_stats"][i]
        col_stat = stats.get(predicate_col, {})

        # Predicate pushdown: can we skip this row group?
        if predicate_op == ">" and col_stat.get("max", float("
            inf")) <= threshold:
            continue    # definitely no rows satisfy predicate
                -> skip
        if predicate_op == "<" and col_stat.get("min", float("-
            inf")) >= threshold:
            continue

        rg_read += 1
        # Column pruning: only access select_cols
        for row_i in range(rg["row_count"]):
            row_val = rg["data"][predicate_col][row_i]
            if predicate_op == ">" and row_val > threshold:
                rows_returned += 1
            elif predicate_op == "<" and row_val < threshold:
                rows_returned += 1
```

```
72
73      # I/O calculation
74      full_scan_bytes  = total_rg * ROW_GROUP_ROWS * len(COLUMNS)
            * BYTES_PER_CELL
75      actual_col_bytes = rg_read  * ROW_GROUP_ROWS * len(
            select_cols) * BYTES_PER_CELL
76      io_reduction_pct = (1 - actual_col_bytes / full_scan_bytes)
            * 100
77
78      print(f"Query: SELECT {select_cols} WHERE {predicate_col} {
            predicate_op} {threshold}")
79      print(f"  Row groups total : {total_rg}")
80      print(f"  Row groups read  : {rg_read}  (skipped {total_rg
            - rg_read} via pushdown)")
81      print(f"  Rows returned    : {rows_returned:,}")
82      print(f"  Full scan bytes  : {full_scan_bytes:,}")
83      print(f"  Actual I/O bytes : {actual_col_bytes:,}")
84      print(f"  I/O reduction    : {io_reduction_pct:.1f}%")
85
86  # Query: SELECT user_id, score WHERE score > 90
87  parquet_scan(rg_list, footer,
88               predicate_col="score", predicate_op=">", threshold
                     =90,
89               select_cols=["user_id", "score"])
90  print()
91  # Query: SELECT user_id, revenue WHERE revenue > 800000
92  parquet_scan(rg_list, footer,
93               predicate_col="revenue", predicate_op=">",
                     threshold=800_000,
94               select_cols=["user_id", "revenue"])
```

Listing 5.2: PE5.2: Parquet predicate pushdown simulator

PE5.3. DRF Fairness Simulation (Python). Simulate Dominant Resource Fairness for a cluster with two resource dimensions (CPU and RAM) and three competing jobs, showing how DRF equalises dominant shares.

```
1   def drf_allocate(cluster_cpu, cluster_ram_gb, jobs, max_rounds
        =20):
2       """
3       Simulate DRF allocation.
4       Each job specifies: name, cpu_per_task, ram_per_task_gb,
            max_tasks
5       DRF grants one task at a time to the job with the lowest
```

```
        dominant share.
    """
    allocated = {j["name"]: {"tasks": 0, "cpu": 0.0, "ram":
        0.0}
                 for j in jobs}
    used_cpu = 0.0
    used_ram = 0.0

    print(f"Cluster: {cluster_cpu} CPUs | {cluster_ram_gb} GB
        RAM")
    print(f"\n{'Round':>5}  {'Job':>12}  {'Dominant':>10}  "
          f"{'Dom share':>10}  {'Tasks':>7}  {'CPU':>7}  {'RAM
              ':>7}")
    print("-" * 68)

    for rnd in range(1, max_rounds + 1):
        # Compute dominant share for each job
        dom_shares = {}
        for j in jobs:
            a = allocated[j["name"]]
            if a["tasks"] >= j["max_tasks"]:
                dom_shares[j["name"]] = float("inf")
                continue
            cpu_share = a["cpu"] / cluster_cpu if cluster_cpu >
                 0 else 0
            ram_share = a["ram"] / cluster_ram_gb if
                cluster_ram_gb > 0 else 0
            dom_res   = "cpu" if cpu_share >= ram_share else "
                ram"
            dom_shares[j["name"]] = (max(cpu_share, ram_share),
                 dom_res)

        # Select job with minimum dominant share
        eligible = [j for j in jobs
                    if isinstance(dom_shares[j["name"]], tuple)
                    and used_cpu + j["cpu_per_task"] <=
                        cluster_cpu
                    and used_ram + j["ram_per_task_gb"] <=
                        cluster_ram_gb]
        if not eligible:
            break

        winner = min(eligible, key=lambda j: dom_shares[j["name
            "]][0])
```

```
        w_name = winner["name"]
        dom_val, dom_res = dom_shares[w_name]

        allocated[w_name]["tasks"] += 1
        allocated[w_name]["cpu"]   += winner["cpu_per_task"]
        allocated[w_name]["ram"]   += winner["ram_per_task_gb"]
        used_cpu += winner["cpu_per_task"]
        used_ram += winner["ram_per_task_gb"]

        print(f"{rnd:>5}  {w_name:>12}  {dom_res:>10}  "
              f"{dom_val*100:>9.1f}%  "
              f"{allocated[w_name]['tasks']:>7}  "
              f"{allocated[w_name]['cpu']:>7.1f}  "
              f"{allocated[w_name]['ram']:>7.1f}")

    print("\nFinal allocations:")
    for j in jobs:
        a = allocated[j["name"]]
        cpu_share = a["cpu"] / cluster_cpu * 100
        ram_share = a["ram"] / cluster_ram_gb * 100
        print(f"  {j['name']:<14}: {a['tasks']} tasks | "
              f"CPU: {a['cpu']:.1f} ({cpu_share:.1f}%) | "
              f"RAM: {a['ram']:.1f} GB ({ram_share:.1f}%)")

# 3 jobs with different resource profiles
jobs = [
    {"name": "MapReduce",  "cpu_per_task": 2,  "ram_per_task_gb": 4,   "max_tasks": 8},
    {"name": "Spark",      "cpu_per_task": 4,  "ram_per_task_gb": 16,  "max_tasks": 6},
    {"name": "Samza",      "cpu_per_task": 1,  "ram_per_task_gb": 2,   "max_tasks": 12},
]

drf_allocate(cluster_cpu=32, cluster_ram_gb=128, jobs=jobs, max_rounds=20)
```

Listing 5.3: PE5.3: Dominant Resource Fairness (DRF) simulator

Part IV

In-Memory Processing

6

Apache Spark: In-Memory Distributed Computing

"The most important question we faced was: what should be the fundamental programming model? We chose to expose distributed memory as a first-class abstraction, and that single decision determined everything else about Spark's design."

Matei Zaharia, co-creator of Apache Spark, 2016

Learning Objectives

After completing this chapter, you will be able to:

1. Identify the specific I/O bottleneck of MapReduce for iterative workloads and derive the disk-I/O cost formula that motivated the design of Spark.
2. Define a Resilient Distributed Dataset (RDD), explain its five defining properties, and describe how lineage-based fault tolerance differs from replication-based fault tolerance.
3. Trace the life cycle of a Spark job from `sc.textFile()` through the DAG Scheduler, Task Scheduler, and Executor, identifying the role of each component.
4. Distinguish narrow transformations from wide transformations,

explain why wide transformations introduce shuffle boundaries, and identify which RDD operations fall into each category.

5. Explain the performance difference between `groupByKey` and `reduceByKey`, and apply the correct operator to avoid unnecessary data movement.
6. Describe the four phases of the Catalyst query optimizer (analysis, logical optimisation, physical planning, and code generation) and explain how each phase reduces query execution cost.
7. Select the appropriate RDD persistence level from `MEMORY_ONLY`, `MEMORY_AND_DISK`, `MEMORY_ONLY_SER`, and `DISK_ONLY` for a given workload, cost, and fault-tolerance requirement.
8. Describe the Spark MLlib Pipeline API—Transformer, Estimator, Pipeline, and PipelineModel—and construct a three-stage feature engineering and classification pipeline.
9. Explain Spark's unified memory management model, distinguish execution memory from storage memory, and calculate the available memory for each region given a heap size and configuration.
10. Apply the salting technique to resolve data skew in a wide transformation, and explain why skew is undetectable by the Catalyst optimizer.

By 2010, Hadoop's MapReduce framework had conquered the storage and batch-processing challenges that motivated its creation. Thousands of organisations processed petabytes per day with clusters of commodity servers, achieving the economics that Google had demonstrated internally with its MapReduce implementation (Dean and Ghemawat 2004). Yet practitioners who attempted to use MapReduce for machine learning training, graph analysis, and iterative data mining encountered a fundamental performance barrier: MapReduce was designed for data that is read once, processed once, and written once. Any algorithm that reads the same data repeatedly—which describes virtually every machine learning training algorithm—paid an enormous, repeated disk I/O penalty that made computation on even modest datasets prohibitively slow.

This chapter introduces Apache Spark, the distributed computing frame-

work that resolved the iterative-workload bottleneck by elevating distributed memory to a first-class programming abstraction. Spark's central concept, the ***Resilient Distributed Dataset*** (RDD), allows the results of an expensive computation to be stored in the combined RAM of a cluster and reused across multiple algorithm iterations without a single disk read. For workloads such as logistic regression training, this in-memory approach achieved speedups of 20× to 100× over MapReduce in the original benchmarks (Zaharia, Chowdhury, Das, et al. 2012).

6.1 The MapReduce Iteration Problem

To understand why Spark was necessary, we must quantify precisely what MapReduce does poorly: iterative computation over the same dataset.

Consider training a logistic regression model on a 100 GB training set using gradient descent. The training algorithm proceeds in rounds, called *iterations* or *epochs*. In each iteration, the algorithm: (1) reads the entire training set; (2) computes the gradient of the loss function with respect to the current model parameters; (3) updates the parameters; and (4) checks a convergence criterion. A typical training run requires 50 to 200 iterations before the model converges.

The MapReduce implementation maps one iteration to one MapReduce job. Each job:

1. Reads the 100 GB training set from HDFS (3× replicated = 300 GB of disk reads).
2. Runs Map tasks to compute per-record gradient contributions.
3. Runs a Reduce task to aggregate the gradients.
4. Writes the updated model parameters to HDFS.
5. The next iteration reads HDFS again from the beginning.

Definition | MapReduce Iteration I/O Cost

For an iterative algorithm requiring I iterations over a dataset of size D

bytes, stored on HDFS with replication factor r:

$$\text{Total disk I/O (MapReduce)} = I \times D \times r \tag{6.1}$$

For logistic regression with $I = 100$ iterations, $D = 100\,\text{GB}$, $r = 3$:

$$\text{Total disk I/O} = 100 \times 100\,\text{GB} \times 3 = 30{,}000\,\text{GB} = 30\,\text{TB}$$

Processing 30 TB of disk I/O to train a model on 100 GB of unique data is a 300× I/O amplification. On a cluster that sustains 1 GB/s aggregate disk read throughput per node, with 100 nodes, this represents 5 minutes of wall-clock time *per iteration*—8.3 hours total for 100 iterations.

The root cause is straightforward: MapReduce has no mechanism for retaining intermediate data in memory between jobs. Every job boundary is a forced write to and read from HDFS. The framework was designed for single-pass batch processing, not iterative refinement. The cluster's RAM—which by 2010 already totalled hundreds of gigabytes across even a modest 100-node cluster—is completely wasted between iterations: it is cleared, and the entire dataset is re-read from disk for the next round.

Interactive analytical workloads suffer a related problem. A data analyst running ten queries over the same dataset against a Hive-on-MapReduce cluster must wait for the full input to be read from HDFS for each query, even if the dataset has already been read by a prior query. There is no shared memory pool between independent jobs that could serve as a cache.

Matei Zaharia and his collaborators at UC Berkeley identified both problems in 2010 and proposed a solution: expose the cluster's aggregate RAM as a distributed, fault-tolerant storage layer that persists across job boundaries. This idea became Apache Spark (Zaharia, Chowdhury, Franklin, et al. 2010).

6.2 Resilient Distributed Datasets

The fundamental abstraction of Apache Spark is the ***Resilient Distributed Dataset*** (RDD), introduced formally by Zaharia et al. at NSDI 2012 (Zaharia,

Chowdhury, Das, et al. 2012). An RDD is the unit of data in Spark in the same way that an HDFS block is the unit of data in Hadoop: every computation operates on RDDs, produces RDDs, and stores RDDs.

Definition | Resilient Distributed Dataset (RDD)

An **RDD** is an immutable, partitioned, fault-tolerant collection of records that can be stored in memory across a cluster of nodes and computed in parallel. Formally, an RDD R is characterised by five properties:

1. **A set of partitions:** $R = \{P_1, P_2, \ldots, P_k\}$, where each partition P_i is a subset of the data assigned to one executor.
2. **A set of dependencies:** a reference to the parent RDD(s) from which R was derived, forming a lineage graph (DAG).
3. **A function to compute each partition** from its parent partition(s).
4. **Partitioning metadata:** the scheme (e.g., hash or range) used to assign records to partitions.
5. **Preferred locations:** hints on which nodes hold the data locally (e.g., HDFS block locations).

6.2.1 Lineage-Based Fault Tolerance

The "Resilient" in RDD refers to Spark's mechanism for recovering from executor failures: ***lineage-based fault tolerance***. This is the most conceptually important property of RDDs, and it distinguishes Spark's approach from both MapReduce (which writes intermediate results to disk) and traditional distributed shared memory (which replicates data in RAM).

When an executor fails and the partitions it held are lost, Spark does not need to restart the entire job. Instead, it consults the RDD's ***lineage graph***—the directed acyclic graph of transformations that produced the lost partition—and *recomputes only the lost partition* on a surviving executor.

This design trades replication overhead for recomputation cost. RDDs backed by data on HDFS (the most common case) can always recompute any lost partition from its HDFS source data, which is itself replicated. For RDDs produced by long chains of transformations, Spark allows users to insert ***checkpoints***—explicit writes of an RDD to HDFS that truncate the lineage graph—preventing unbounded recomputation cost in the event of

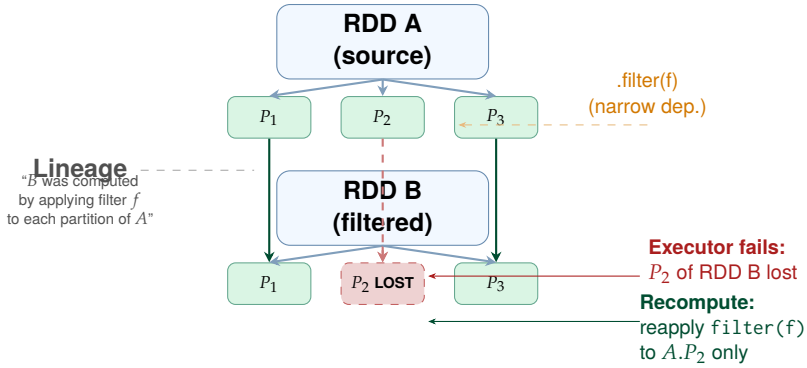

Figure 6.1: RDD lineage-based fault tolerance. When P_2 of RDD B is lost due to executor failure, Spark consults the lineage graph to determine that P_2 of B was produced by applying `filter(f)` to P_2 of RDD A, which is still available. Only the lost partition is recomputed—no full job restart is required.

failure after many transformation stages.

Key Insight | Lineage vs. Replication: A Design Trade-off

Lineage-based fault tolerance costs nothing in memory (no second copy of the data is maintained) but can be expensive in time if the lineage is long. Replication-based fault tolerance (as in HDFS) costs memory for every replica but recovers in constant time regardless of lineage length. Spark's design assumes that executor failures are rare and datasets are large relative to available RAM—so paying memory for replication would defeat the purpose of in-memory computing. For streaming workloads where maintaining lineage across an unbounded stream is impractical, Spark Structured Streaming adds micro-batch replication.

6.2.2 Lazy Evaluation and the RDD DAG

RDDs are computed ***lazily***: calling a transformation method such as `filter()` or `map()` does not immediately execute any computation. It only records the transformation in the RDD's lineage graph. Actual computation is triggered only when the programmer calls an ***action***—a method that returns a non-RDD result to the driver (such as `count()`, `collect()`, or `saveAsTextFile()`).

This laziness is not a limitation; it is a deliberate optimisation opportunity.

When an action is called, Spark's DAG Scheduler can inspect the entire lineage graph of all RDDs involved in producing the result and optimise the computation plan. It can fuse adjacent narrow transformations into a single pass over the data, avoiding intermediate materialisation, and it can identify which partitions are already cached in memory and skip their recomputation.

6.2.3 RDD Persistence

The performance advantage of Spark over MapReduce materialises only when RDDs are *persisted*. When an RDD is used in multiple actions—the common case in iterative algorithms—the programmer must explicitly call `rdd.persist(StorageLevel.MEMORY_ONLY)` (or the equivalent `rdd.cache()`, which is shorthand for `MEMORY_ONLY`) to instruct Spark to retain the RDD's partitions in executor memory after the first computation. Without persistence, each action triggers a full recomputation from the lineage root.

Table 6.1: RDD storage levels and their trade-offs

Storage level	Memory	Disk	When to use
`MEMORY_ONLY`	Yes (raw)	No	Fits in RAM; fast recompute from HDFS
`MEMORY_AND_DISK`	Yes (raw)	Overflow	Larger-than-RAM; avoid full recompute
`MEMORY_ONLY_SER`	Yes (Kryo)	No	Tight on RAM; CPU overhead acceptable
`MEMORY_AND_DISK_SER`	Yes (Kryo)	Overflow	Tight on RAM + large data
`DISK_ONLY`	No	Yes	Rarely accessed; fast recompute is slow
`OFF_HEAP`	Off-heap	No	Avoid GC pauses; Tungsten memory

Spark evicts persisted RDD partitions using a *least-recently-used* (LRU) policy when storage memory is exhausted. Under `MEMORY_AND_DISK`, evicted

partitions spill to the local disk of the executor node rather than being dropped; they are read back from disk if needed by a subsequent action. Under `MEMORY_ONLY`, evicted partitions are simply recomputed from their lineage on demand.

6.3 Spark Architecture

A Spark application consists of a single ***driver program*** and one or more ***executors*** running on worker nodes. The driver and executors communicate through a ***cluster manager***, which allocates the physical resources (CPU cores and RAM) that the application requires.

6.3.1 The Driver Program and SparkSession

The ***driver program*** is the process that contains the user's application code. It runs the `main()` function of the Spark application, constructs the ***SparkSession*** (the unified entry point since Spark 2.0, replacing the earlier `SparkContext` and `HiveContext`), and orchestrates the overall execution.

The driver performs three critical functions:

- **DAG construction:** as the application code calls transformation methods, the driver records each transformation in an RDD lineage DAG. No computation occurs yet.
- **Job submission:** when an action is called, the driver submits a job to the DAG Scheduler. The DAG Scheduler partitions the lineage DAG into ***stages***—groups of transformations that can be pipelined without a data shuffle—and submits each stage as a set of ***tasks*** to the Task Scheduler.
- **Result collection:** for actions that return data to the driver (such as `collect()` or `first()`), the driver receives task results from executors and assembles the final result.

Common Misconception | The Driver Memory Trap

`rdd.collect()` transfers *all* partitions of an RDD to the driver's JVM heap. On a production dataset of even a few hundred gigabytes, this causes an OutOfMemoryError that crashes the driver process and terminates the

entire application. The correct alternatives are `rdd.take(n)` (return only the first n records), `rdd.saveAsTextFile(path)` (write results to HDFS without collecting), or `rdd.sample(false, 0.01)` (return a 1% sample). Use `collect()` only for small result sets that the driver can safely hold in memory—such as the output of a final aggregation.

6.3.2 Executors

Executors are long-lived JVM processes launched on worker nodes at application start-up time. Each executor:

- Runs *tasks* assigned by the Task Scheduler, with multiple tasks running concurrently in separate threads (one thread per available vCPU core allocated to the executor).
- Maintains a local *block store*—the Spark Block Manager— that holds cached RDD partitions and shuffle output blocks.
- Reports task status, progress, and shuffle data locations back to the driver via a heartbeat mechanism.
- Runs for the entire lifetime of the application, enabling in-memory data sharing across multiple Spark jobs within the same application.

The executor's JVM heap is divided between *execution memory* (for active task computation: sort buffers, hash tables for aggregations, shuffle read/write buffers) and *storage memory* (for cached RDD partitions). This division is discussed in depth in Section 6.7.

6.3.3 Cluster Managers

Spark is *cluster-manager-agnostic*: it delegates the allocation of physical resources to a pluggable cluster manager. Three cluster managers are commonly used in production:

- **Spark Standalone:** Spark's built-in cluster manager, suitable for dedicated Spark clusters. Simple to configure but does not support multi-tenant resource sharing or integration with other frameworks. Appropriate for development and single-purpose deployment.
- **YARN:** the most common production deployment (see Chapter 5). Spark

submits an ApplicationMaster to YARN; the AM negotiates executor containers with the ResourceManager. All of YARN's scheduling policies (Capacity, Fair, DRF) apply to Spark executors. Enables multi-framework resource sharing with MapReduce, Tez, and Samza on the same cluster.

- **Kubernetes:** the preferred deployment model on cloud infrastructure. The driver runs as a Kubernetes Pod; each executor runs in its own Pod. Kubernetes manages container scheduling, scaling, and isolation using its standard infrastructure, enabling integration with cloud autoscaling.

6.3.4 DAG Scheduler and Task Scheduler

When an action triggers job execution, the driver's scheduler pipeline proceeds in two phases.

The ***DAG Scheduler*** receives the RDD lineage graph, identifies the *stage boundaries* (discussed in Section 6.4), and decomposes the computation into a DAG of stages. Each stage is a maximal set of consecutive narrow transformations that can be pipelined. Stage boundaries are placed wherever a wide transformation (shuffle) appears. The DAG Scheduler also applies *partition skipping*: any stage whose output partitions are already cached in executor storage memory is skipped entirely, and the cached data is used directly as input to the next stage.

The ***Task Scheduler*** receives the physical task list for each ready stage and submits tasks to executors via the cluster manager. The Task Scheduler applies *delay scheduling*: if the data-local executor is briefly unavailable, the Task Scheduler waits a configurable period (default 3 seconds) before falling back to a rack-local or any-node assignment, maximising data locality.

6.4 Transformations and Actions

Every RDD operation in Spark is classified as either a *transformation* or an *action*. Understanding this distinction, and the further sub-classification of transformations into narrow and wide, is essential for writing correct and performant Spark applications.

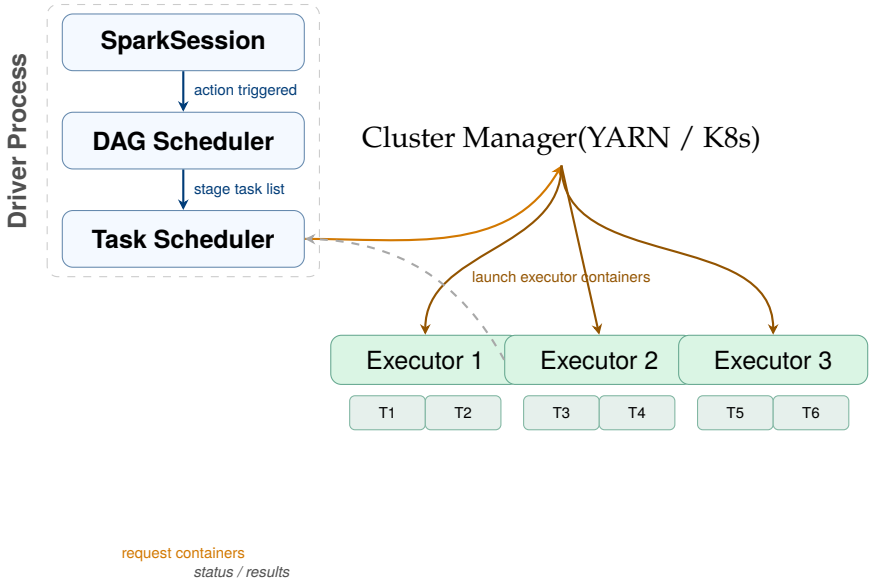

Figure 6.2: Spark architecture. The driver process contains the DAG Scheduler and Task Scheduler. The cluster manager (YARN, Kubernetes, or Standalone) allocates executor containers on worker nodes. Executors run tasks assigned by the driver and report results via heartbeat. Executors remain alive for the full application lifetime, enabling in-memory data sharing across jobs.

6.4.1 Narrow Transformations

A ***narrow transformation*** is one in which each output partition depends on at most one input partition. No data movement across the network is required: each executor can compute its output partitions independently, working only on the input partitions it already holds locally. Consecutive narrow transformations are *pipelined* by the DAG Scheduler into a single stage: a task processes a record through all pipelined transformations without materialising any intermediate RDD.

The most important narrow transformations are:

- `map(f)`: applies function f to each record, producing one output record per input record.
- `filter(p)`: retains only records satisfying predicate p.
- `flatMap(f)`: applies f to each record and flattens the resulting sequences into one output RDD.
- `mapPartitions(f)`: applies f once per partition rather than once per

record, which is more efficient when the function has per-call overhead (e.g., opening a database connection or loading a model).

- `union(other)`: concatenates two RDDs with the same schema, with no data movement (each partition of the result is exactly one partition of an input).
- `sample(fraction)`: samples each partition independently, with no cross-partition communication.

6.4.2 Wide Transformations and the Shuffle

A ***wide transformation*** is one in which each output partition may depend on multiple input partitions, potentially held on different executors. Producing the output requires *moving data across the network*—a ***shuffle***. Every shuffle is a stage boundary: the DAG Scheduler inserts a synchronisation point before the downstream stage begins, ensuring all upstream tasks have completed and written their shuffle output before the downstream tasks begin reading it.

The shuffle is the most expensive operation in Spark. Each shuffle involves:

1. **Map side (shuffle write):** each upstream task hash-partitions its output records by key and writes each partition's records to a local shuffle file on the executor's disk. This write is unavoidable: the data must survive until all downstream tasks have read it.
2. **Network transfer:** each downstream task reads the shuffle blocks it needs from remote executors via HTTP, crossing the network.
3. **Reduce side (shuffle read):** each downstream task receives and potentially sorts its input before applying the aggregation or join function.

The most important wide transformations are:

- `groupByKey()`: collects all values for each key into a single iterable in one partition. Requires a full shuffle.
- `reduceByKey(f)`: applies the commutative and associative function f to reduce values for each key. Crucially, `reduceByKey` applies a partial aggregation *within each partition* before the shuffle (analogous to the Combiner in MapReduce), dramatically reducing the volume of data transferred.

- `aggregateByKey(zeroValue)(seqOp, combOp)`: a generalisation of `reduceByKey` that supports different input and output types.
- `sortByKey(ascending)`: globally sorts the RDD by key, requiring a range-partitioned shuffle.
- `join(other)`: joins two RDDs on their keys, requiring both to be hash-shuffled by key to co-locate matching records.
- `distinct()`: removes duplicates, requiring a shuffle to co-locate all copies of each record.
- `repartition(n)`: redistributes the RDD into exactly n partitions, performing a full shuffle.

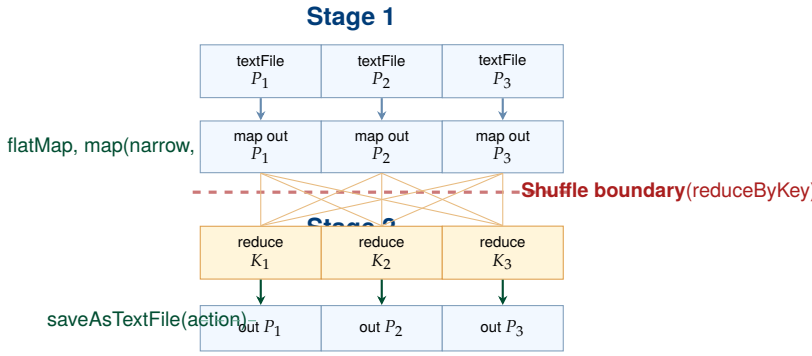

Figure 6.3: Spark stage and shuffle boundary for a word-count job. Stage 1 pipelines `textFile`, `flatMap`, and `map` (all narrow) into a single pass. The `reduceByKey` is a wide transformation, creating a shuffle boundary. Stage 2 performs the shuffle read and final aggregation. The all-to-all arrows represent network transfer of shuffle blocks.

6.4.3 groupByKey vs. reduceByKey: A Critical Distinction

Common Misconception | `groupByKey` Transfers All Data; `reduceByKey` Does Not

The most common performance anti-pattern in Spark is using `groupByKey()` when `reduceByKey()` is applicable. Consider computing the sum of values per key. `groupByKey()` sends *all* raw values for each key across the network to one executor, where the sum is computed. If there are 10^8 records with key "DE" each holding a 16-byte value, that is 1.6 GB of network traffic *just for that one key*.

`reduceByKey(lambda a, b: a + b)` first sums values within each source partition independently (producing one sum per partition per key), then shuffles only these partial sums. If there are 200 partitions, the shuffle transfers 200 numbers for key "DE"—16-byte numbers— instead of 1.6 GB. The network amplification factor is reduced by the partition count (200× in this example).
The general rule: use `reduceByKey`, `aggregateByKey`, or `combineByKey` whenever the aggregation function is associative and commutative. Use `groupByKey` only when all raw values for a key must be available simultaneously (e.g., computing a per-key median).

6.4.4 Actions

Actions are the operations that trigger the execution of the accumulated RDD lineage DAG and return a concrete result. The most important actions are:

- `count()`: returns the number of records in the RDD as a `Long`.
- `collect()`: transfers all partitions to the driver and returns a local array (use only for small results).
- `take(n)`: returns the first n records from the first partition(s), with less data movement than `collect()`.
- `reduce(f)`: applies the associative and commutative function f to all records, returning a single value.
- `foreach(f)`: applies function f to each record on the executor side (e.g., to write records to an external system) without returning data to the driver.
- `saveAsTextFile(path)`: writes the RDD as plain text files to an HDFS path (one file per partition).
- `saveAsSequenceFile(path)`: writes as Hadoop SequenceFile format (key-value pairs).

6.5 Spark SQL: DataFrames, Datasets, and the Catalyst Optimizer

The RDD API provides low-level control over distributed computation but requires the programmer to reason explicitly about types, partitions, and data layouts. For SQL-style analytical workloads, this level of control is unnecessarily cumbersome. Spark SQL, introduced in Spark 1.0 and substantially redesigned through Spark 2.0 and 3.0, provides a structured, schema-aware API that allows the Spark engine to apply far more aggressive automatic optimisations.

6.5.1 DataFrames and Datasets

A ***DataFrame*** is an RDD of `Row` objects with a named, typed schema—conceptually a distributed relational table. The schema is declared explicitly or inferred from structured data sources (Parquet, ORC, JSON, Hive tables, JDBC). Unlike a raw RDD, a DataFrame carries column names and data types, enabling the query engine to reason about the computation without executing it.

A ***Dataset*** is a strongly typed, JVM-object-centric version of the DataFrame, available in Scala and Java. A `Dataset[Person]` stores JVM `Person` objects rather than generic `Row` objects, providing compile-time type safety at the cost of some JVM serialisation overhead. In Python and R, DataFrames are used exclusively (there is no separate Dataset API because these languages are dynamically typed).

The ***SparkSession*** is the unified entry point for all Spark SQL operations:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName("SalesAnalysis") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

# Load a Parquet table from HDFS
sales = spark.read.parquet("hdfs:///data/sales/dt=2024-*")

# DataFrame operations: filter, groupBy, aggregate
result = (sales
```

```
    .filter(col("amount") > 100)
    .groupBy("region", "product_id")
    .agg(
        count("*").alias("num_sales"),
        avg("amount").alias("avg_amount")
    )
    .orderBy(col("avg_amount").desc())
)

result.write.mode("overwrite").parquet("hdfs:///output/
    sales_summary")
```

Listing 6.1: SparkSession creation and DataFrame operations

6.5.2 The Catalyst Optimizer

The key advantage of the DataFrame/SQL API over the raw RDD API is that every DataFrame operation passes through the ***Catalyst optimizer*** before execution. Catalyst is a tree-based query optimizer written in Scala that applies a cascade of rule-based and cost-based transformations to reduce query execution cost. It operates in four phases.

Phase 1 — Analysis. Catalyst resolves attribute names and data types by consulting the ***Spark Catalog***—the metadata registry for tables, views, functions, and databases (backed by the Hive Metastore in production). Unresolved logical plans (trees with unresolved attribute references) become resolved logical plans (trees where every attribute has a confirmed name and data type).

Phase 2 — Logical Optimisation. Catalyst applies rule-based algebraic transformations to the resolved logical plan:

- **Predicate pushdown:** move `WHERE` filter conditions as close to the data source as possible, so that Parquet row groups can be skipped before data is read into memory.
- **Column pruning:** remove columns not referenced by the query from the scan; only referenced column chunks are read from Parquet files.
- **Constant folding:** evaluate expressions that involve only constants at compile time (e.g., `WHERE year = 2020 + 4` becomes `WHERE year = 2024`).
- **Join reordering:** place smaller tables on the build side of hash joins (or as the broadcast side of broadcast hash joins).

Phase 3 — Physical Planning. Catalyst converts the optimised logical plan into one or more candidate physical plans—concrete execution strategies involving Spark operators. For a join, physical plans may include sort-merge join (for large tables), broadcast hash join (when one table fits in executor memory, default threshold 10 MB), or hash join. Catalyst applies a simple cost model (based on estimated row counts from Parquet file statistics) to select the lowest-cost plan.

Phase 4 — Code Generation (Tungsten). The selected physical plan is compiled to JVM bytecode by the ***Tungsten*** execution engine. Tungsten uses *whole-stage code generation*: rather than interpreting a query plan row-by-row through a generic operator tree, Tungsten generates a single JVM method that fuses all operators in a pipeline into a tight loop. This eliminates virtual function call overhead and enables the JIT compiler to produce CPU-efficient native code. Tungsten also uses off-heap *unsafe* memory for sort and aggregate buffers, bypassing JVM garbage collection overhead.

System Design Note | Catalyst and Parquet: An End-to-End Example

A query `SELECT product_id, SUM(amount) FROM sales WHERE region = 'EU' AND dt BETWEEN '2024-01-01' AND '2024-03-31' GROUP BY product_id` on a Parquet table partitioned by `dt` passes through Catalyst as follows:

1. **Partition pruning:** only the subdirectories `/sales/dt=2024-01-01/` through `/sales/dt=2024-03-31/` are scanned. All other partitions are eliminated before any data is read.
2. **Predicate pushdown:** the `region = 'EU'` filter is pushed into the Parquet reader. Row groups where `max(region) < 'EU'` or `min(region) > 'EU'` (lexicographically) are skipped via footer statistics.
3. **Column pruning:** only the `region`, `product_id`, and `amount` column chunks are read; all other column chunks are not opened.
4. **Aggregation:** Catalyst generates a two-phase aggregate plan: a partial sum within each partition (no shuffle), then a shuffle on `product_id`, then a final sum per partition in the reduce stage.

The combined effect of partition pruning, predicate pushdown, and column pruning can reduce the physical I/O from terabytes to gigabytes

on a selective query—without any manual hint or configuration beyond the initial table partitioning.

6.5.3 SparkSQL and Hive Integration

Spark SQL integrates with the Hive Metastore through the Hive Catalog interface, allowing SparkSQL queries to operate on tables defined by Hive DDL statements, including partitioned Parquet and ORC tables (introduced in Chapter 5). This integration allows organisations to migrate from Hive-on-MapReduce to Spark SQL with no changes to table definitions, partition schemes, or ETL job logic—only the execution engine changes.

6.6 MLlib: Distributed Machine Learning

Apache Spark MLlib is Spark's built-in machine learning library. Its central design goal is to express machine learning pipelines—sequences of data preprocessing, feature engineering, and model training steps—as ***Pipeline*** objects that can be fit to training data, saved to disk, and applied to new data with a single API call.

6.6.1 Core Abstractions: Transformer, Estimator, and Pipeline

The MLlib Pipeline API is built from three abstractions.

A ***Transformer*** is an object that implements a `transform(df)` method: it takes a DataFrame as input and produces a new DataFrame with one or more new columns added. Transformers are *stateless*—they do not learn parameters from data. Examples include `Tokenizer` (splits text into words), `HashingTF` (converts token lists to TF-IDF feature vectors), `StandardScaler` after fitting (normalises numerical features using pre-computed mean and standard deviation), and any trained model (which transforms a feature DataFrame into a predictions DataFrame).

An ***Estimator*** is an object that implements a `fit(df)` method: it trains a model on data and produces a *Transformer* (the trained model). Examples include `LogisticRegression`, `RandomForestClassifier`, `KMeans`, and `StandardScaler` (before fitting—the fitted version is a Transformer).

A ***Pipeline*** is an ordered sequence of Transformers and Estimators. `pipeline.fit(df)` calls `fit()` on each Estimator in sequence, using the training DataFrame and passing the output to the next stage. The result is a `PipelineModel`—a sequence of Transformers—which can be applied to new data via `model.transform(test_df)`.

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import (Tokenizer, HashingTF, IDF,
                                StringIndexer, VectorAssembler
                                    )
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import
    MulticlassClassificationEvaluator

# Load labelled text data
df = spark.read.parquet("hdfs:///data/news_articles")

# Build pipeline stages
tokenizer  = Tokenizer(inputCol="text", outputCol="tokens")
hashing_tf = HashingTF(inputCol="tokens", outputCol="
    raw_features",
                       numFeatures=20_000)
idf        = IDF(inputCol="raw_features", outputCol="features")
indexer    = StringIndexer(inputCol="label", outputCol="
    label_idx")
lr         = LogisticRegression(featuresCol="features",
                                labelCol="label_idx",
                                maxIter=100)

pipeline = Pipeline(stages=[tokenizer, hashing_tf, idf, indexer
    , lr])

# Train / test split
train, test = df.randomSplit([0.8, 0.2], seed=42)

# Fit: trains IDF, StringIndexer, LogisticRegression on train
model = pipeline.fit(train)

# Transform: applies all transformations to test
predictions = model.transform(test)

evaluator = MulticlassClassificationEvaluator(
    labelCol="label_idx", predictionCol="prediction",
```

```
    metricName="accuracy")
print(f"Test accuracy: {evaluator.evaluate(predictions):.4f}")

# Persist the trained model to HDFS for serving
model.save("hdfs:///models/news_classifier_v1")
```

Listing 6.2: MLlib Pipeline for text classification

6.6.2 Key MLlib Algorithms

Table 6.2: Representative MLlib algorithms by category

Category	Algorithm	Typical use case
Classification	LogisticRegression, RandomForestClassifier, GBTClassifier, LinearSVC	Spam detection, churn prediction, document classification
Regression	LinearRegression, RandomForestRegressor, GBTRegressor	Price forecasting, demand estimation
Clustering	KMeans, GaussianMixture, BisectingKMeans	Customer segmentation, anomaly detection
Recommendation	ALS (Alternating Least Squares)	Collaborative filtering (Netflix, Spotify)
Feature Eng.	PCA, Word2Vec, MinHashLSH, QuantileDiscretizer	Dimensionality reduction, similarity search

Alternating Least Squares (ALS) deserves particular mention as it illustrates how Spark's in-memory model enables iterative ML algorithms. ALS is a matrix factorisation algorithm for collaborative filtering: given a user-item rating matrix with many missing entries, it alternately holds user factors fixed and solves for item factors (an embarrassingly parallel linear algebra problem), then holds item factors fixed and solves for user factors.

Each alternation reads the full ratings matrix. On a dataset of 10^9 ratings, this read is performed 10–20 times per training run; with the ratings cached in Spark's distributed memory, all subsequent reads are served from RAM at disk-free speed.

6.7 Memory Management in Spark

Spark's performance advantage over MapReduce rests on its ability to use the executor's JVM heap for both active computation and RDD caching. Understanding how Spark divides this memory between competing uses is essential for diagnosing and resolving the two most common production failure modes: spill-to-disk (which degrades performance) and OOM (OutOfMemory) errors (which crash executors).

6.7.1 The Unified Memory Model

Spark 1.6 introduced the ***unified memory model***, which replaced the earlier static partitioning of execution and storage memory with a dynamic boundary. The executor JVM heap is divided into three regions:

1. **Reserved Memory** (300 MB, fixed): reserved for Spark's internal objects. Not configurable.
2. **Spark Memory** (`spark.memory.fraction` $\times$ (heap $-$ 300 MB), default 0.6): the pool shared between execution and storage.
3. **User Memory** (remaining $1 - 0.6 = 0.4$ fraction): available for user-defined data structures, UDF objects, and the Java/Scala collections used in application code.

Within the Spark Memory pool, execution memory and storage memory share the space dynamically:

- **Storage Memory** (initially `spark.memory.storageFraction` $\times$ Spark Memory, default 0.5) caches persisted RDD partitions. Storage memory can expand into unused execution memory, but cannot forcibly evict active execution memory.

- **Execution Memory** (the remainder) holds sort buffers, hash tables for aggregations, and shuffle read/write buffers. Execution memory can borrow from storage memory, and if it does, Spark evicts cached RDD partitions (via LRU) to make room.

Definition | Unified Memory Allocation

For an executor with H bytes of JVM heap:

$$M_{\text{spark}} = \texttt{fraction} \times (H - 300\,\text{MB}) \tag{6.2}$$

$$M_{\text{storage}} \approx \texttt{storageFraction} \times M_{\text{spark}} \tag{6.3}$$

$$M_{\text{exec}} = M_{\text{spark}} - M_{\text{storage}} \quad \text{(initial)} \tag{6.4}$$

For $H = 16\,\text{GB}$, $\texttt{fraction} = 0.6$, $\texttt{storageFraction} = 0.5$:

$$M_{\text{spark}} = 0.6 \times (16\,\text{GB} - 0.3\,\text{GB}) \approx 9.4\,\text{GB}$$

$$M_{\text{storage}} \approx 0.5 \times 9.4\,\text{GB} = 4.7\,\text{GB}$$

$$M_{\text{exec}} \approx 4.7\,\text{GB} \quad \text{(can grow up to } 9.4\,\text{GB)}$$

6.7.2 Serialisation and Kryo

When RDD partitions are persisted with `MEMORY_ONLY_SER` or spill to disk, Spark must serialise Java/Scala objects into a byte stream. By default, Spark uses Java's native serialisation, which is flexible (handles arbitrary Java objects) but slow and space-inefficient.

Kryo serialisation (`spark.serializer = org.apache.spark. serializer.KryoSerialize` is a third-party library that serialises registered classes 2–10× faster and produces 3–5× smaller byte representations than Java serialisation. For RDDs of simple, known types (such as `(String, Int)` pairs or custom case classes), Kryo is almost always the correct choice.

```
from pyspark import SparkConf, SparkContext

conf = SparkConf() \
    .setAppName("KryoDemo") \
    .set("spark.serializer",
         "org.apache.spark.serializer.KryoSerializer") \
    .set("spark.kryo.registrationRequired", "false") \
    .set("spark.kryoserializer.buffer.max", "256m")
```

```
9
10 sc = SparkContext(conf=conf)
```

Listing 6.3: Enabling Kryo serialization and registering classes

6.7.3 Data Skew and Salting

Data skew occurs when the keys of a wide transformation are distributed unevenly: a small number of keys contain a very large fraction of the records. In a `reduceByKey`, all records for the same key must be sent to the same partition. If 40% of the dataset has the key "US", one task will process 40% of the data while the other 199 tasks process the remaining 60% divided evenly. The entire stage cannot complete until the skewed task finishes. The stage latency is dominated by the slowest task, making the overall job wall-clock time proportional to the largest single-key volume.

The standard remedy is ***salting***: appending a random integer suffix (the "salt") to skewed keys to distribute their records across multiple partitions, then performing a two-phase aggregation.

```
1  import random
2
3  SALT_FACTOR = 50    # number of salt buckets for skewed keys
4  SKEWED_KEYS = {"US", "CN", "DE"}   # keys known to be skewed
5
6  def add_salt(record):
7      key, value = record
8      if key in SKEWED_KEYS:
9          salted_key = (key, random.randint(0, SALT_FACTOR - 1))
10     else:
11         salted_key = (key, 0)
12     return (salted_key, value)
13
14 def remove_salt(record):
15     (key, _salt), value = record
16     return (key, value)
17
18 # Phase 1: shuffle by (key, salt) -> 50x more reducers for
       skewed keys
19 partial = (rdd
20     .map(add_salt)
21     .reduceByKey(lambda a, b: a + b))   # partial aggregation
```

```
22
23 # Phase 2: strip salt, re-aggregate to get final per-key totals
24 final = (partial
25     .map(remove_salt)
26     .reduceByKey(lambda a, b: a + b))   # final aggregation
```

Listing 6.4: Salting to resolve data skew in a PySpark RDD

Production Lesson | Skew is Invisible to the Optimizer

Data skew is one of the most common causes of Spark job failure in production, and it is almost entirely invisible to the Catalyst optimizer. Catalyst can pushdown predicates, prune columns, and choose between sort-merge and broadcast joins—but it cannot detect that 40% of the data has the same key unless the user provides statistics or explicit hints. The symptoms are: a stage where 199 out of 200 tasks complete quickly but one task runs for hours, eventually causing the executor to OOM or timeout.

Diagnostic steps: examine the Spark UI "Stages" tab and look for tasks with anomalously large "Input Size" or "Shuffle Read Size." Use `df.groupBy("key").count().orderBy(F.desc("count")).show(20)` to identify skewed keys before designing the aggregation.

6.8 Research Spotlights

Research Spotlight | Zaharia et al. (2010) — Spark: Cluster Computing with Working Sets

Citation: Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., and Stoica, I. (2010). Spark: Cluster Computing with Working Sets. *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 2010)*. (Zaharia, Chowdhury, Franklin, et al. 2010)

Key insight: The paper's thesis is that MapReduce is fundamentally unsuited for two important and growing workload classes: *iterative algorithms* (machine learning, graph processing) and *interactive data analysis* (exploratory SQL on cached datasets). Both require reading the same data multiple times, and MapReduce's forced disk materialisation

between stages makes this prohibitively expensive. The solution is to treat distributed RAM as a first-class abstraction that persists across job boundaries.

Technical contributions:

- Introduced the concept of a *working set*—the subset of data that an application accesses repeatedly—and argued that a cluster computing framework should allow the working set to be stored in distributed memory across the cluster.
- Demonstrated that a logistic regression training job ran 10× faster on Spark than on Hadoop MapReduce by caching the training set in memory after the first iteration.
- Showed that interactive queries over a 39 GB Wikipedia dataset ran in 0.5–1 second on cached data vs. 20–60 seconds on Hadoop, enabling genuine interactive exploration at scale.

Impact today: The HotCloud 2010 paper introduced Spark to the research community and has been cited over 5,000 times. Within three years of this publication, Spark had been adopted by hundreds of organisations and had become the fastest-growing Apache project in history. The working-set insight—that most production ML and analytics workloads repeatedly access the same data—is now the primary justification for in-memory computing frameworks across the industry.

Research Spotlight | Zaharia et al. (2012) — Resilient Distributed Datasets: A Fault-Toler

Citation: Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., and Stoica, I. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012)*, pp. 15–28. **Best Paper Award.** (Zaharia, Chowdhury, Das, et al. 2012)

Key insight: The central contribution is the formalization of the RDD abstraction and its lineage-based fault tolerance model. The paper argues that existing distributed shared memory systems fail for large-scale data processing because they require fine-grained writes to be

tracked for replication—an overhead that is prohibitive at petabyte scale. RDDs restrict mutations to coarse-grained transformations (each transformation applies the same function to all records in a partition), which makes it possible to describe any lost partition's provenance with a short lineage graph rather than a full data log.

Technical contributions:

- Formally defined the RDD as a five-tuple (partitions, dependencies, compute function, partitioner, preferred locations) and proved that lineage-based fault tolerance is sufficient for the class of coarse-grained transformations that covers all common data analytics operations.
- Distinguished narrow from wide dependencies and showed that narrow-dependency lineage recomputation is always partition-local (no cross-executor coordination needed), while wide-dependency recomputation may require reading multiple parent partitions.
- Benchmarked Spark RDDs against Hadoop MapReduce on four workloads: logistic regression (25× speedup on second iteration), PageRank (7× speedup), k-means clustering (40× speedup), and interactive SQL queries (sub-second vs. tens of seconds).

Impact today: The NSDI 2012 paper is the most-cited Spark paper, with over 10,000 citations, and is considered one of the foundational papers of the systems era of big data processing. The RDD abstraction directly influenced the design of DataFrame/Dataset APIs, Apache Flink's DataSet/DataStream APIs, TensorFlow's distributed variable model, and Ray's distributed object store. The narrow/wide dependency distinction is now standard vocabulary in distributed systems courses worldwide.

6.9 Case Study: Alibaba's Spark Deployment for Singles' Day (11.11)

Case Study | Alibaba — Apache Spark at 10,000 Nodes for the World's Largest Shopping

Background. Alibaba's Singles' Day (11.11) shopping festival—held annually on 11 November—is the world's largest single-day commerce event. In 2019, the event generated $38.4 billion USD in gross merchandise value within 24 hours, with a peak transaction rate of 544,000 orders per second. Behind this scale lay one of the largest production deployments of Apache Spark in existence, integrated into Alibaba's MaxCompute (formerly ODPS) and E-MapReduce platforms.

Scale. Alibaba's internal Spark deployment for 11.11 2019 involved:

- Over **10,000 executor nodes**, each with 96 GB of RAM and 24 vCPUs—approximately 960 TB of aggregate RAM and 240,000 vCPUs available to Spark applications.
- Processing over **970 PB** of data in the 24-hour event window, primarily through Spark SQL queries over Parquet tables partitioned by buyer_id, seller_id, and event timestamp.
- Over **1 trillion individual user behaviour events** (page views, searches, cart additions, purchases, refunds) accumulated, processed, and fed into real-time recommendation engines during the event.

The recommendation challenge. The highest-impact use of Spark on 11.11 was powering Taobao's product recommendation system, which generates personalised product lists for 800 million active users. Recommendation model updates during the event must incorporate behaviour signals from the event itself: a user who viewed three laptops in the first hour of the event should be shown accessories in the second hour, not laptops she has already browsed.
This requirement—updating models on streaming data *during* the event, not just in nightly batch jobs—drove Alibaba's engineers to build a hybrid streaming-batch architecture:

1. **Behaviour ingestion** (Alibaba's Blink / Apache Flink): raw user events arrive from Taobao at approximately 11 million events per second at peak. Flink processes them in real time, computing

streaming feature updates (user-product affinity scores, click-through rates, conversion funnels) with a latency of under 500 milliseconds.

2. **Model refresh** (Spark MLlib): every 10 minutes during the 24-hour event, a Spark MLlib ALS collaborative filtering job is triggered, consuming the latest streaming feature snapshots from HDFS (written by Flink) as training data. The 10-minute ALS job runs on 2,000 executor nodes with the rating matrix (approximately 800 million users $\times$ 2 billion items, with 40 billion observed entries) cached in distributed memory across the executors. Retraining from scratch on this matrix would require 6–8 hours with disk-based MapReduce; with Spark and in-memory caching, the job completes in 8–12 minutes.
3. **Recommendation serving**: the refreshed model parameters (user and item latent factors, each a 128-dimensional vector) are pushed to Alibaba's distributed key-value store and served to Taobao's API layer within 2 minutes of the ALS job completing. Users browsing during the refresh window see recommendations based on model parameters that incorporate signals from as recently as 12 minutes ago.

Spark SQL for campaign analytics. Beyond recommendation, Alibaba's marketing analytics team ran Spark SQL queries continuously during 11.11 to power the event's live broadcast analytics dashboard—visible to 800 million TV viewers as a real-time counter of GMV, transaction volume, and category breakdowns. These queries joined three Parquet tables:

Table	Size	Update frequency
`transactions`	~200 TB at event end	Appended per minute
`products`	~3 GB (broadcast)	Static
`sellers`	~5 GB (broadcast)	Static

Catalyst automatically applied *broadcast hash join* for the small dimension tables (products, sellers), avoiding shuffles entirely for those joins. The `transactions` table, partitioned by `event_minute`, was scanned with partition pruning: each dashboard query only read the most recent

5-minute partition (~600 MB) rather than the full 200 TB of accumulated transactions.

Skew management. A critical engineering challenge was the Taobao flagship stores: a single store operated by a major brand might account for hundreds of millions of transaction records in 24 hours. Grouping by `seller_id` without salting would have created severe skew, with one task processing >20% of the entire dataset. Alibaba's engineering team applied salting for known high-volume sellers, reducing the maximum single-task volume to approximately 2% of the dataset.

Outcomes and lessons. Alibaba's 11.11 2019 Spark deployment demonstrated three principles that have since become industry standards for large-scale event analytics:

- **In-memory iterative retraining is viable at billion-user scale.** The ALS rating matrix caching strategy reduced model refresh time by 40× compared to disk-based alternatives, enabling 144 model refreshes during the 24-hour event.
- **Broadcast joins for dimension tables eliminate shuffle.** Alibaba measured a 15× latency reduction for multi-table queries by broadcasting the 3 GB products table to all 10,000 executor nodes, compared to sort-merge joining it through a shuffle.
- **Partition pruning must be designed into the schema.** Partitioning the transactions table by `event_minute` rather than `dt` (which is standard for daily analytics) allowed dashboard queries to scan only the most recent partition. This was a deliberate schema design decision made 6 weeks before the event; retroactively repartitioning 200 TB during the event would have been impossible.

Chapter Summary

- MapReduce's disk I/O bottleneck for iterative algorithms is quantified by the formula: total I/O = $I \times D \times r$, where I is the number of iterations, D is the dataset size, and r is the HDFS replication factor. For 100 iterations on 100 GB with $r = 3$, this produces 30 TB of disk I/O.

- An **RDD** is an immutable, partitioned, lazily computed, fault-tolerant collection of records. Its five defining properties are: partitions, dependencies (lineage), compute function, partitioning metadata, and preferred locations.
- **Lineage-based fault tolerance** recomputes lost partitions from their lineage graph rather than maintaining replicas. This costs nothing in memory but may require recomputation after long lineage chains; **checkpointing** truncates the lineage to bound recomputation cost.
- **Narrow transformations** (map, filter, flatMap) have each output partition depending on at most one input partition and require no shuffle. **Wide transformations** (reduceByKey, groupByKey, join) require all-to-all data exchange (shuffle) and create stage boundaries.
- **`reduceByKey`** applies partial aggregation within each partition before the shuffle, reducing network traffic by approximately the partition count. **`groupByKey`** transfers all raw values across the network and should be avoided whenever an aggregation function is associative and commutative.
- The **Catalyst optimizer** processes DataFrame/SQL plans in four phases: analysis (name resolution), logical optimisation (predicate pushdown, column pruning, constant folding), physical planning (join strategy selection), and code generation (Tungsten whole-stage code generation).
- **RDD persistence levels** range from `MEMORY_ONLY` (fastest, no fault tolerance for evicted partitions) to `DISK_ONLY` (slowest, full fault tolerance). Kryo serialisation reduces the memory footprint and serialisation time of persisted RDDs by 3–10× over Java serialisation.
- The **unified memory model** divides executor heap into Reserved (fixed 300 MB), Spark (0.6 fraction), and User (0.4 fraction) regions. Within the Spark region, execution and storage memory share space dynamically, with execution memory able to evict cached RDD partitions.
- **Data skew** occurs when a small number of keys dominate the dataset. The **salting** remedy appends a random integer to skewed keys, distributing their records across multiple partitions, then removes the salt in a second aggregation pass.
- The MLlib **Pipeline API** expresses preprocessing and training as a sequence of Transformers and Estimators. A `Pipeline.fit()` call trains all

Estimators and produces a `PipelineModel` that can be saved to HDFS and applied to new data.

Exercises

Short Questions

SQ6.1. Explain the MapReduce iteration I/O problem using the formula Total I/O = $I \times D \times r$. For a k-means clustering job that runs 50 iterations on a 200 GB dataset with $r = 3$, calculate the total disk I/O. How does Spark reduce this?

SQ6.2. Define a Resilient Distributed Dataset. List its five defining properties and explain the purpose of each.

SQ6.3. What is lineage-based fault tolerance? Compare it with replication-based fault tolerance on two dimensions: memory overhead and recovery latency.

SQ6.4. Explain the concept of lazy evaluation in Spark. What is the performance benefit of deferring computation until an action is called?

SQ6.5. Distinguish narrow transformations from wide transformations. Give two examples of each, and explain why wide transformations create stage boundaries in the DAG.

SQ6.6. Why is `reduceByKey` almost always preferable to `groupByKey` for aggregation? Give a concrete numerical example showing the difference in network traffic for a dataset with 1 billion records and 200 partitions, where 30% of records share the same key.

SQ6.7. What is a YARN ApplicationMaster for a Spark job? Which component of Spark architecture runs inside the AM container, and what does it do?

SQ6.8. Explain the four phases of the Catalyst optimizer. Which phase is responsible for choosing between a sort-merge join and a broadcast hash join, and what criterion determines the choice?

SQ6.9. What is the unified memory model in Spark? For an executor with 32 GB JVM heap, `spark.memory.fraction = 0.6`, and `spark.memory.storageFraction = 0.5`, calculate the available storage memory, execution memory, and user memory.

SQ6.10. What is Tungsten whole-stage code generation? How does it differ from the interpreted, row-at-a-time Volcano model used by traditional query engines?

SQ6.11. Explain the Spark MLlib abstraction of Transformer vs. Estimator. Give one concrete example of each and explain what `fit(df)` produces for an Estimator.

SQ6.12. What is data skew in a Spark wide transformation? Describe the observable symptom in the Spark UI that indicates a skew problem.

SQ6.13. What does `MEMORY_ONLY_SER` mean as an RDD storage level? Under what circumstances should you choose it over `MEMORY_ONLY`?

SQ6.14. Explain what a checkpoint does in Spark. When is checkpointing necessary, and what are its costs?

SQ6.15. A Spark job with 8 stages takes 4 hours. The Spark UI shows that Stage 3 ran for 3.5 hours, with 199 tasks completing in 2 minutes and one task running for 3.5 hours. What is the most likely cause, and how would you diagnose it?

Analytical Problems

AP6.1. Spark vs. MapReduce: Iterative Algorithm Cost Analysis. A genomics research lab runs a principal component analysis (PCA) on a SNP (single nucleotide polymorphism) dataset. The dataset is 500 GB, stored on HDFS with $r = 3$. PCA requires 80 power-iteration steps.

(a) Calculate the total disk I/O for a MapReduce implementation (one job per iteration, each job reads from and writes to HDFS).

(b) The lab's HDFS cluster sustains 2 GB/s aggregate disk read bandwidth. Estimate the wall-clock time for the MapReduce implementation.

(c) With Spark and `MEMORY_ONLY` persistence, only the first iteration reads

from HDFS. Subsequent iterations read from memory at 50 GB/s aggregate throughput. Estimate the total wall-clock time for the Spark implementation.

(d) Calculate the speedup ratio $T_{\text{MR}}/T_{\text{Spark}}$.

(e) The dataset grows to 2 TB. The cluster has 1 TB of aggregate executor RAM. Explain what happens to the Spark job when the RDD no longer fits in memory under `MEMORY_ONLY` and under `MEMORY_AND_DISK`.

AP6.2. Stage and Shuffle Analysis. A Spark job reads a log file from HDFS, parses each line into a (session_id, page_url, duration_seconds) tuple, filters records with duration > 5 seconds, computes the total duration per session, keeps sessions with total duration > 60 seconds, joins with a 200 MB session-metadata table (session_id, user_id, country), groups by country, and writes to HDFS.

(a) Draw the RDD lineage DAG for this job. Label each RDD with the operation that produced it.

(b) Identify all stage boundaries and explain why each boundary is placed where it is.

(c) Identify any narrow transformations that can be pipelined into a single task pass, and explain the I/O savings this produces.

(d) The session-metadata table is 200 MB. Explain how the Catalyst optimizer would handle the join with this table and why this avoids a shuffle.

(e) The log file has 20 billion records and `spark.sql. shuffle.partitions = 200`. The join produces 150 billion output records. Explain whether 200 shuffle partitions is appropriate and how you would determine the correct value.

AP6.3. Unified Memory Sizing. An e-commerce company runs a Spark job on a YARN cluster. Each executor is allocated 24 GB JVM heap and 6 vCPUs. The job caches a 180 GB RDD (serialised with Kryo) that must be accessed by 20 iterative training passes. The remaining 30 GB of the dataset does not need caching.

(a) With `spark.memory.fraction = 0.6` and `spark.memory.storageFraction = 0.5`, calculate the storage memory and execution memory per execu-

tor.

(b) If the executor count is 40, calculate the total aggregate storage memory across the cluster. Does the 180 GB RDD fit?

(c) If Kryo serialisation achieves a 3× compression ratio over the raw RDD size, recalculate whether the RDD fits in aggregate storage memory.

(d) If the RDD still does not fit, describe two configuration changes (with specific parameter names and values) that could increase the available storage memory.

(e) Explain the performance consequence if the RDD does not fit in `MEMORY_ONLY` storage but the job uses `MEMORY_AND_DISK` instead.

AP6.4. Data Skew Diagnosis and Salting. A retail company runs a Spark job that joins a 5 TB transactions table (partitioned by `store_id`) with a 2 TB products table. The join key is `product_id`. Analysis of the data reveals:

product_id	Transaction share
`IPHONE-15-PRO`	28.3%
`SAMSUNG-S24`	11.7%
`AIRPODS-PRO`	6.2%
All other 4.2M products	53.8%

(a) With 200 shuffle partitions, estimate the number of records assigned to the task handling `IPHONE-15-PRO` vs. the average task (the total dataset has 50 billion records).

(b) The average task runs in 4 minutes. Estimate the stage latency with and without skew mitigation.

(c) Design a salting strategy for the top-3 skewed keys with a salt factor of 30. Show the key before and after salting for a sample record.

(d) Explain the two-phase aggregation required after salting and why it is necessary.

(e) An alternative to salting for join skew is *skew join hint* (`SKEW JOIN` in Spark SQL). Explain conceptually how the optimizer would handle a skew hint differently from a standard sort-merge join.

Design Problems

DP6.1. Spark Architecture for a Real-Time Fraud Detection Platform. A global payment network processes 30,000 transactions per second. Each transaction has 45 features (amount, merchant category, cardholder history, location, time-since-last-transaction, etc.). A fraud detection model must score each transaction within 200 milliseconds of arrival. The ML team retrains the model weekly on 90 days of historical transactions (≈240 TB).

Design the Spark architecture for both the training pipeline and the serving infrastructure. Your design must specify:

(a) The cluster topology for weekly model retraining: number of executor nodes, memory per executor, and YARN queue configuration. Justify the memory sizing based on the 240 TB dataset and the 10-iteration gradient boosting training algorithm.

(b) The MLlib Pipeline stages for feature engineering: which Estimators and Transformers are needed, and in what order.

(c) How the trained model is exported from the Spark cluster to the serving layer, given the 200-millisecond latency constraint (Spark jobs take seconds to initialise; model serving cannot invoke a Spark job per transaction).

(d) The persistence strategy for the feature engineering pipeline: which intermediate RDDs should be cached, at what storage level, and why.

(e) How the design changes if the retraining frequency is increased from weekly to hourly, incorporating the most recent 6 hours of transactions into the model on each run.

DP6.2. Spark SQL Migration from Hive-on-MapReduce. A telecommunications company's analytics team runs 400 Hive-on-MapReduce queries per day against a 500 TB ORC warehouse. Average query latency is 45 minutes; the SLA for analyst-facing queries is 15 minutes; and 50 queries (12.5%) exceed 2 hours. The data engineering team proposes migrating to Spark SQL.

Design the migration. Your design must address:

(a) The Catalyst optimisations (predicate pushdown, column pruning,

broadcast joins) that are most likely to reduce latency for queries over an ORC table partitioned by date and region. Estimate the I/O reduction for a query selecting 8 of 120 columns, filtering on a date range covering 5% of the total data, with an additional predicate that eliminates 70% of the remaining row groups.

(b) The YARN queue configuration changes required to co-run Hive-on-MapReduce (existing nightly ETL) and Spark SQL (new analyst workload) on the same cluster without either workload starving the other.

(c) A compatibility verification strategy: how to confirm that the Spark SQL query results match the Hive-on-MapReduce results for the 400 queries before switching production traffic.

(d) The expected performance change for the 50 slow queries (currently >2 hours). Which of the following root causes would be fixed by Catalyst, and which would still require manual query rewriting: (i) full-table scans that could use partition pruning; (ii) `GROUP BY` on a column with extreme skew; (iii) a three-table join where the smallest table is 400 MB.

(e) A rollback plan if Spark SQL produces incorrect results for a category of queries (e.g., NULL handling differences from Hive's default behaviour).

Programming Exercises

PE6.1. Word Frequency with RDD API (PySpark). Implement a word frequency counter using the Spark RDD API. The program must: (1) read a text file from HDFS; (2) tokenise each line into words (lowercase, strip punctuation); (3) count occurrences of each word; (4) return the top 20 most frequent words and their counts. Measure the performance difference between using `reduceByKey` and `groupByKey`.

```
import re
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("WordFreq").getOrCreate()
sc    = spark.sparkContext
```

```
6
7  def tokenize(line):
8      """Lower-case and split on non-alpha characters."""
9      return [w for w in re.split(r"[^a-z]+", line.lower()) if w]
10
11 # Read text from HDFS (replace with actual path)
12 text_rdd = sc.textFile("hdfs:///data/wikipedia_sample.txt")
13
14 # --- Implementation using reduceByKey (correct approach) ---
15 freq_rdd = (text_rdd
16     .flatMap(tokenize)
17     .map(lambda w: (w, 1))
18     .reduceByKey(lambda a, b: a + b)
19     .sortBy(lambda kv: kv[1], ascending=False)
20 )
21
22 top20 = freq_rdd.take(20)
23 print("Top 20 words (reduceByKey):")
24 for word, count in top20:
25     print(f"  {word:<20} {count:>8,}")
26
27 # --- Implementation using groupByKey (anti-pattern for
       comparison) ---
28 # TODO: implement using groupByKey and compare memory/time
29 # groupby_rdd = (text_rdd
30 #     .flatMap(tokenize)
31 #     .groupBy(lambda w: w)
32 #     .mapValues(lambda vals: sum(1 for _ in vals))
33 #     ...
34 # )
35
36 # Cache and measure second-pass performance
37 freq_rdd.persist()
38 print(f"\nTotal unique words: {freq_rdd.count():,}")
```

Listing 6.5: PE6.1: Word frequency with RDD API

PE6.2. Iterative k-Means with RDD Caching (PySpark). Implement k-means clustering using the raw RDD API (not MLlib) to observe the effect of caching on iterative performance. The implementation must: (1) cache the input data RDD after the first iteration; (2) measure wall-clock time per iteration; (3) report the speedup of cached vs. uncached iterations.

```
import time
import random
import math
from pyspark.sql import SparkSession
from pyspark import StorageLevel

spark = SparkSession.builder.appName("KMeans").getOrCreate()
sc    = spark.sparkContext

def parse_point(line):
    return [float(x) for x in line.strip().split(",")]

def distance(p, c):
    return math.sqrt(sum((a - b)**2 for a, b in zip(p, c)))

def closest_centroid(point, centroids):
    dists = [distance(point, c) for c in centroids]
    return dists.index(min(dists))

def add_vectors(v1, v2):
    return [a + b for a, b in zip(v1, v2)]

def mean_vector(total, count):
    return [x / count for x in total]

K         = 10
MAX_ITERS = 20
SEED      = 42

# Load data
data = sc.textFile("hdfs:///data/clustering_input.csv").map(
    parse_point)

# Cache the input data     this is the key optimisation
data.persist(StorageLevel.MEMORY_ONLY)
data.count()    # trigger caching (materialize into memory)

# Initialise centroids randomly from the first partition
centroids = data.takeSample(False, K, seed=SEED)

iter_times = []
for iteration in range(MAX_ITERS):
    t_start = time.time()

```

```
    # Assign each point to its nearest centroid
    assignments = data.map(
        lambda p: (closest_centroid(p, centroids),
                   (p, 1))
    )

    # Sum vectors and counts per cluster
    cluster_stats = (assignments
        .reduceByKey(lambda a, b: (add_vectors(a[0], b[0]),
                                   a[1] + b[1]))
    )

    # Compute new centroids
    new_centroids_list = (cluster_stats
        .mapValues(lambda vs: mean_vector(vs[0], vs[1]))
        .collect()
    )
    new_centroids = {k: v for k, v in new_centroids_list}
    centroids = [new_centroids.get(i, centroids[i]) for i in
        range(K)]

    elapsed = time.time() - t_start
    iter_times.append(elapsed)
    print(f"Iteration {iteration+1:>2}: {elapsed:.2f}s")

print(f"\nFirst iteration : {iter_times[0]:.2f}s  (HDFS read +
    compute)")
print(f"Mean iter 2-20  : {sum(iter_times[1:])/len(iter_times
    [1:]):.2f}s  (cached)")
print(f"Speedup from cache: {iter_times[0]/sum(iter_times[1:])*
    len(iter_times[1:]):.1f}x")
```

Listing 6.6: PE6.2: Iterative k-means with RDD caching

PE6.3. Salting Benchmark for Skew Mitigation (PySpark). Simulate a skewed dataset and measure the performance difference between a naive `reduceByKey` (with skew) and a salted two-phase aggregation. Report per-task statistics to demonstrate that salting eliminates the long tail.

```
import random
import time
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SkewBenchmark") \
```

```
    .config("spark.sql.shuffle.partitions", "50") \
    .getOrCreate()
sc = spark.sparkContext

TOTAL_RECORDS  = 5_000_000
SKEW_KEY       = "IPHONE-15-PRO"
SKEW_FRACTION  = 0.35           # 35% of all records
SALT_FACTOR    = 20             # number of salt buckets

def generate_record(_):
    """Generate (product_id, sale_amount) with skew."""
    if random.random() < SKEW_FRACTION:
        return (SKEW_KEY, random.randint(500, 1500))
    products = [f"PROD-{i:06d}" for i in range(1, 1001)]
    return (random.choice(products), random.randint(10, 500))

data = sc.parallelize(range(TOTAL_RECORDS), 50).map(
    generate_record)
data.persist()
data.count()    # materialise

# --- Approach 1: naive reduceByKey (skewed) ---
t0 = time.time()
naive_result = data.reduceByKey(lambda a, b: a + b).collect()
t_naive = time.time() - t0
print(f"Naive reduceByKey: {t_naive:.2f}s")

# --- Approach 2: salted two-phase aggregation ---
SKEWED_KEYS = {SKEW_KEY}

def salt_key(record):
    key, val = record
    if key in SKEWED_KEYS:
        return ((key, random.randint(0, SALT_FACTOR - 1)), val)
    return ((key, 0), val)

def desalt_key(record):
    (key, _salt), val = record
    return (key, val)

t1 = time.time()
salted_result = (data
    .map(salt_key)
    .reduceByKey(lambda a, b: a + b)
```

```
    .map(desalt_key)
    .reduceByKey(lambda a, b: a + b)
    .collect()
)
t_salted = time.time() - t1
print(f"Salted 2-phase  : {t_salted:.2f}s")

print(f"Speedup         : {t_naive/t_salted:.2f}x")

# Verify correctness: totals should match
naive_dict  = dict(naive_result)
salted_dict = dict(salted_result)
assert abs(naive_dict.get(SKEW_KEY, 0) -
           salted_dict.get(SKEW_KEY, 0)) < 1e-6, "Results
               differ!"
print("Correctness check: PASSED")
```

Listing 6.7: PE6.3: Salting benchmark for skew mitigation

References

Beyer, M. A. and D. Laney (2012). "The Importance of "Big Data": A Definition". In: *Gartner Research*. Gartner Report G00235055.

Brewer, E. A. (2000). "Towards Robust Distributed Systems". In: *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC 2000)*. Keynote address. Portland, OR, USA: ACM, p. 7. DOI: 10.1145/343477.343502.

Chang, F., J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber (2006). "Bigtable: A Distributed Storage System for Structured Data". In: *Proceedings of the 7th Symposium on Operating System Design and Implementation (OSDI 2006)*. USENIX Association, pp. 205–218.

Dean, J. and S. Ghemawat (2004). "MapReduce: Simplified Data Processing on Large Clusters". In: *Proceedings of the 6th Symposium on Operating System Design and Implementation (OSDI 2004)*. USENIX Association, pp. 137–150.

Ghemawat, S., H. Gobioff, and S.-T. Leung (2003). "The Google File System". In: *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP 2003)*. ACM, pp. 29–43. DOI: 10.1145/945445.945450.

Ghodsi, A., M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica (2011). "Dominant Resource Fairness: Fair Allocation of Multiple Resource Types". In: *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2011)*. USENIX Association, pp. 323–336.

Gilbert, S. and N. A. Lynch (2002). "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services". In: *ACM SIGACT News* 33.2, pp. 51–59. DOI: 10.1145/564585.564601.

Halevy, A., A. Rajaraman, and J. Ordille (2006). "Data Integration: The Teenage Years". In: *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB 2006)*. Seoul, South Korea: VLDB Endowment, pp. 9–16.

Laney, D. (Feb. 2001). *3D Data Management: Controlling Data Volume, Velocity and Variety*. Research Note 949. META Group.

Manyika, J., M. Chui, B. Brown, J. Bughin, R. Dobbs, C. Roxburgh, and A. H. Byers (May 2011). *Big Data: The Next Frontier for Innovation, Competition, and Productivity*. Tech. rep. McKinsey Global Institute.

Reinsel, D., J. Gantz, and J. Rydning (Nov. 2018). *The Digitization of the World: From Edge to Core*. Tech. rep. IDC White Paper, Doc. US44413318. International Data Corporation (IDC).

Thusoo, A., J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, and R. Murthy (2009). "Hive – A Warehousing Solution Over a Map-Reduce Framework". In: *Proceedings of the VLDB Endowment (PVLDB)*. Vol. 2. 2, pp. 1626–1629. DOI: 10.14778/1687553.1687609.

Vavilapalli, V. K., A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O'Malley, S. Radia, B. Reed, and E. Baldeschwieler (2013). "Apache Hadoop YARN: Yet Another Resource Negotiator". In: *Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC 2013)*. ACM, 5:1–5:16. DOI: 10.1145/2523616.2523633.

Zaharia, M., M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica (2012). "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012)*. Best Paper Award. USENIX Association, pp. 15–28.

Zaharia, M., M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica (2010). "Spark: Cluster Computing with Working Sets". In: *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 2010)*. USENIX Association.